IF3070 DASAR INTELEGENSI ARTIFISIAL

TUGAS BESAR I

Optimasi Penataan Kendaraan Geladak Kapal Kargo
PT Intim Line Menggunakan Algoritma Local Search

Disusun Oleh:

Salma Az Zahra	18224015
Belva Charissa Putri Have	18224051
Sherry Eunike	18224075

Dosen Pengampu:

Ir. Windy Gambetta, MBA

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI **1**
DAFTAR TABEL **3**
DAFTAR GAMBAR **4**
BAB I DESKRIPSI PERSOALAN **5**
1.1 2D Vehicle Problem 5
1.2 Permasalahan 7
BAB II PEMBAHASAN **8**
2.1 Pemilihan Objective Function 8
2.1.1 Komponen Fungsi Objektif 8
2.1.2 Kombinasi Skor Akhir 9
2.1.3 Alasan Perancangan 9
2.1.4 Fungsi Objektif Alternatif (Bonus) 10
2.2 Penjelasan Pembangkitan State Awal (Initial State) 13
2.3 Penjelasan Implementasi Algoritma Hill-Climbing 14
2.3.1 Algoritma Steepest Ascent Hill-Climbing 14
2.3.2 Algoritma Hill-Climbing with Sideways Move 18
2.3.3 Algoritma Random Restart Hill-Climbing 24
2.3.4 Algoritma Stochastic Hill-Climbing 29
2.4 Penjelasan Implementasi Algoritma Simulated Annealing 33
2.5 Penjelasan Implementasi Algoritma Genetic Algorithm 40
2.6 Hasil Eksperimen dan Analisis 49
2.6.1 Dataset 49
2.6.2 Hasil Hill-Climbing 49
2.6.3 Hasil Simulated Annealing 72
2.6.4 Hasil Genetic Algorithm 78
2.6.5 Hasil Fungsi Objective Alternatif 94
2.6.6 Analisis Hasil Algoritma dengan Global Optima 119
2.6.7 Analisis Perbandingan Hasil Pencarian Algoritma 120
2.6.8 Analisis Perbandingan Durasi Algoritma 123
2.6.9 Analisis Konsistensi Hasil Algoritma 125
2.6.10 Analisis Variasi Parameter Genetic Algorithm 125
2.6.11 Analisis Perbandingan Penggunaan Fungsi Objektif Alternatif 126
BAB III KESIMPULAN DAN SARAN **128**
3.1 Kesimpulan 128
3.2 Saran 129
BAB IV PEMBAGIAN TUGAS ANGGOTA KELOMPOK **130**

REFERENSI **132**
LAMPIRAN **133**
SURAT PERNYATAAN PENGGUNAAN AI **134**

DAFTAR TABEL

Tabel 1. Hasil Steepest-Ascent Hill-Climbing 1 49
Tabel 2. Hasil Steepest-Ascent Hill-Climbing 2 51
Tabel 3. Hasil Steepest-Ascent Hill-Climbing 3 53
Tabel 4. Hasil Stochastic Hill-Climbing 1 55
Tabel 5. Hasil Stochastic Hill-Climbing 2 57
Tabel 6. Hasil Stochastic Hill-Climbing 3 59
Tabel 7. Hasil Random Restart Hill-Climbing 1 61
Tabel 8. Hasil Random Restart Hill-Climbing 2 63
Tabel 9. Hasil Random Restart Hill-Climbing 3 65
Tabel 10. Hasil Sideways Move Hill-Climbing 1 67
Tabel 11. Hasil Sideways Move Hill-Climbing 2 68
Tabel 12. Hasil Sideways Move Hill-Climbing 3 70
Tabel 13. Hasil Simulated Annealing 1 72
Tabel 14. Hasil Simulated Annealing 2 74
Tabel 15. Hasil Simulated Annealing 3 76
Tabel 16. Hasil Genetic Algorithm 1 78
Tabel 17. Hasil Genetic Algorithm 2 81
Tabel 18. Hasil Genetic Algorithm 3 83
Tabel 19. Hasil Genetic Algorithm 4 86
Tabel 20. Hasil Genetic Algorithm 5 89
Tabel 21. Hasil Genetic Algorithm 6 91
Tabel 22. Hasil ETA Priority dengan Simulated Annealing 1 94
Tabel 23. Hasil ETA Priority dengan Simulated Annealing 2 96
Tabel 24. Hasil ETA Priority dengan Simulated Annealing 3 99
Tabel 25. Hasil ETA Priority dengan Genetic Algorithm 1 101
Tabel 26. Hasil ETA Priority dengan Genetic Algorithm 2 103
Tabel 27. Hasil ETA Priority dengan Genetic Algorithm 3 104
Tabel 28. Hasil Capacity Efficiency dengan Simulated Annealing 1 107
Tabel 29. Hasil Capacity Efficiency dengan Simulated Annealing 2 109
Tabel 30. Hasil Capacity Efficiency dengan Simulated Annealing 3 111
Tabel 31. Hasil Capacity Efficiency dengan Genetic Algorithm 1 113
Tabel 32. Hasil Capacity Efficiency dengan Genetic Algorithm 2 115
Tabel 33. Hasil Capacity Efficiency dengan Genetic Algorithm 3 116
Tabel 34. Durasi Setiap Algoritma 123
Tabel 35. Pembagian Tugas 130

DAFTAR GAMBAR

Gambar 1. Global Optima..... 119
Gambar 2. Kelompok GARUDA..... 133
Gambar 3. Perjuangan Mengerjakan Tubes DAI..... 133

BAB I
DESKRIPSI PERSOALAN

1.1 2D Vehicle Problem

Permasalahan yang dibahas pada tugas besar ini adalah bagaimana menata kendaraan pada geladak satu unit kapal kargo. Permasalahan ini merupakan kombinasi dari *Two-Dimensional Orthogonal Packing Problem*, karena kendaraan dengan dimensi panjang dan lebar tertentu harus ditempatkan secara presisi pada matriks grid area geladak. Selain itu, masalah ini mencakup perluasan dari *Knapsack Problem* karena kapal memiliki kapasitas berat maksimum yang terbatas, sehingga sistem harus menyeleksi kendaraan mana saja yang akan diangkut untuk mengoptimalkan utilitas ruang dan pendapatan.

Sistem harus menentukan tiga keputusan utama untuk setiap kendaraan, yaitu status muat kendaraan (dimuat atau tidak), titik koordinat (x, y) pada geladak, dan orientasinya (Horizontal atau Vertikal). Tujuan akhirnya adalah memaksimalkan nilai fungsi objektif seperti Total Tarif Pengiriman (*Shipping Fee*), Prioritas ETA, atau Efisiensi Kapasitas tanpa melanggar batas ukuran geladak (keluar batas/ *out of bounds*), tanpa adanya tumpang tindih (*overlap*) antar kendaraan, dan tidak melebihi kapasitas berat maksimum kapal.

Setiap kendaraan direpresentasikan dengan ID, dimensi ukuran (w dan l), tarif pengiriman (fee), berat (weight), dan perkiraan waktu kedatangan (ETA). Contoh format data masukan yang sesuai dengan sistem adalah:

```
{
  "ship": { "w": 6, "l": 6, "MaxCapacity": 100 },
  "vehicles": [
    { "v_id": "V1", "w": 2, "l": 2, "fee": 5000, "weight": 20, "eta": 1 },
    { "v_id": "V2", "w": 2, "l": 3, "fee": 7000, "weight": 30, "eta": 1 },
    { "v_id": "V3", "w": 1, "l": 4, "fee": 4000, "weight": 15, "eta": 2 }
  ]
}
```

Setelah proses optimasi dilakukan, sistem menghasilkan rekam jejak tata letak yang menampilkan daftar kendaraan yang berhasil dimuat (berstatus `is_loaded: true`), posisi serta rotasinya, dan representasi penggunaan sel-sel geladak dalam bentuk matriks. Contoh ringkasan keluarannya adalah:

```
{
  "valid": true,
  "total_fee": 16000,
  "total_weight": 65,
  "vehicles": [
    { "id": "V1", "x": 0, "y": 0, "orientation": "Horizontal", "is_loaded": true },
    { "id": "V2", "x": 2, "y": 0, "orientation": "Vertical", "is_loaded": true },
    { "id": "V3", "x": 0, "y": 2, "orientation": "Horizontal", "is_loaded": true }
  ]
}
```


Secara matematis, permasalahan *2D Knapsack* untuk satu area kapal ini diformulasikan sebagai berikut:

$$n = \text{jumlah kendaraan (vehicles)}$$
$$W_{max} = \text{kapasitas maksimum berat kapal (MaxCapacity)}$$
$$w_i = \text{berat kendaraan ke-}i$$
$$c_i = \text{tarif pengiriman kendaraan ke-}i$$

$$x_i = \begin{cases} 1, & \text{jika kendaraan } i \text{ dimuat ke kapal (is_loaded} = \text{True)} \\ 0, & \text{lainnya} \end{cases}$$

Fungsi Objektif (Memaksimalkan total tarif pengiriman):

$$\text{Maximize } Z = \sum_{i=1}^{n} c_i x_i$$

Total berat kendaraan yang dimuat tidak boleh melebihi kapasitas maksimum kapal:

$$\sum_{i=1}^{n} w_i x_i \leq W_{max}$$

Setiap koordinat geladak hanya boleh ditempati maksimal oleh satu kendaraan (Tidak ada tumpang tindih): Jika $G_{a,b}$ adalah jumlah kendaraan yang menempati koordinat sel (a, b) pada geladak kapal, maka:

$$G_{x,y} \leq 1 \quad \forall x \in [0, W_{kapal} - 1], \forall y \in [0, L_{kapal} - 1]$$

Masalah ini termasuk ke dalam kategori *NP-hard problem*, karena jumlah kombinasi penempatan, rotasi orientasi, dan seleksi barang meningkat secara eksponensial seiring bertambahnya jumlah kendaraan. Oleh karena itu, pencarian solusi eksak menjadi tidak efisien. Sebagai gantinya, sistem memanfaatkan pendekatan meta-heuristik dan algoritma stokastik, seperti variasi *Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*. Metode-metode ini mengeksplorasi ruang pencarian dengan melakukan mutasi atau pergerakan kecil (memutar, menggeser, memuat, menurunkan kendaraan) secara iteratif untuk menemukan solusi tata letak yang paling optimal secara waktu.

1.2 Permasalahan

PT Intim Line memerlukan sistem otomatis untuk menentukan penataan kendaraan di kapal kargo secara optimal. Pada tugas besar ini, solusi dicari menggunakan tiga algoritma *local search*:

1. *Hill-Climbing* (HC)

   Melakukan perbaikan solusi dengan berpindah ke neighbor state yang memiliki nilai fungsi objektif lebih baik hingga mencapai kondisi local optima.

2. *Simulated Annealing* (SA)

   Menggunakan konsep probabilistik untuk menerima solusi yang lebih buruk dalam tahapan tertentu, agar dapat keluar dari local optima dan menjelajahi ruang solusi yang lebih luas.

3. *Genetic Algorithm* (GA)

   Menggunakan prinsip evolusi biologis, di mana setiap solusi direpresentasikan sebagai individu dalam populasi yang bereproduksi melalui proses crossover dan mutation untuk menghasilkan generasi baru yang lebih baik.

Setiap algoritma akan beroperasi berdasarkan representasi state berupa pemetaan posisi dan orientasi kendaraan di dalam maupun di luar geladak kapal. Operasi yang diizinkan pada setiap iterasi yaitu menukar dua kendaraan (baik yang berada di dalam maupun di luar kapal), memindahkan posisi kendaraan ke koordinat lain pada geladak, serta mengubah orientasi kendaraan. Tujuan dari setiap algoritma adalah memaksimalkan total tarif pengiriman (*Shipping Fee*) dengan tetap memastikan tidak ada kendaraan yang saling bertumpuk, melampaui batas area geladak, atau melebihi kapasitas muatan maksimum kapal.

BAB II
PEMBAHASAN

2.1 Pemilihan *Objective Function*

Dalam penyelesaian permasalahan penataan kendaraan di geladak kapal, tujuan utama yang ingin dicapai adalah memaksimalkan total pendapatan (*shipping fee*) dari kendaraan yang berhasil dimuat. Suatu penataan hanya dianggap layak jika memenuhi tiga batasan, yaitu:

1. Tidak ada kendaraan yang saling tumpang tindih (*overlap*).
2. Seluruh kendaraan yang dimuat berada sepenuhnya di dalam batas grid geladak.
3. Total berat tidak melebihi MaxCapacity kapal.

Berdasarkan hal tersebut, fungsi objektif dirancang dengan dua komponen, yaitu pencapaian total *shipping fee* serta mekanisme penalti validitas *state*. Implementasi fungsi objektif ditunjukkan sebagai berikut:

```
def calculate_objective(self, obj_type=None):
        if not self.is_valid():
            return -1000000

        return self.get_total_fee()
```

2.1.1 Komponen Fungsi Objektif

a. Total *Shipping Fee* (get_total_fee)

Komponen ini menjumlahkan atribut shipping_fee dari seluruh kendaraan yang memiliki status is_loaded = True. Semakin banyak kendaraan bernilai tinggi yang berhasil dimuat, maka akan semakin tinggi nilai objektifnya.

b. Pengecekan Validitas (is_valid)

Fungsi ini memeriksa tiga batasan, yaitu:

a. Memastikan berat kendaraan yang dimuat tidak melebihi MaxCapacity.
b. Memeriksa posisi kendaraan termuat agar berada sepenuhnya di dalam batas grid geladak (memperhitungkan dimensi efektif w_eff dan l_eff berdasarkan orientasinya).

c. Memastikan tidak ada sel geladak yang ditempati lebih dari satu kendaraan (*overlap*).

c. Penalti State Tidak Valid

Jika state melanggar salah satu batasan, maka fungsi objektif akan langsung mengembalikan nilai -1000000. Nilai ini sengaja dibuat sangat rendah (jauh berada di bawah perkiraan total fee maksimum yang mungkin diperoleh), sehingga state yang tidak valid akan selalu lebih buruk daripada state valid apa pun.

2.1.2 Kombinasi Skor Akhir

Rumusan matemaris dari fungsi objektif $f(s)$ pada *state* s dapat dituliskan sebagai berikut:

$f(s) = F(s)$, jika s valid

$f(s) = -1000000$, jika s tidak valid

Keterangan:

- $F(s)$: total *shipping fee* kendaraan yang dimuat pada state s (self.get_total_fee())

Semakin besar nilai $f(s)$, maka kualitas solusi yang dihasilkan akan semakin baik.

2.1.3 Alasan Perancangan

Terdapat beberapa alasan pemilihan *objective function* ini, yaitu:

a. Pendapatan (*Shipping Fee*) sebagai Tujuan Utama

Fungsi objektif dirancang untuk berfokus pada maksimalisasi total pendapatan dari kendaraan yang berhasil dimuat, tanpa terganggu oleh perhitungan bobot penalti yang rumit di dalam area solusi yang valid.

b. Penalti Mutlak sebagai Jaminan Validitas

Nilai penalti sebesar -1000000 ditetapkan jauh di bawah total fee maksimum yang mungkin diperoleh. Hal ini memastikan bahwa *state* yang melanggar batasan selalu memperoleh *value* yang lebih buruk dibandingkan *state* valid apa pun.

c. Pemisahan Tegas antara Kelayakan dan Optimasi

Dengan memisahkan pemeriksaan validitas secara sejak awal, algoritma dapat fokus sepenuhnya untuk meningkatkan total *fee* di area ruang pencarian *(search space*) yang sudah aman dan layak.

Dengan rancangan ini, algoritma akan menghindari solusi yang tidak valid, berfokus pada pemilihan kombinasi kendaraan yang memberikan *shipping fee* tertinggi, serta menjaga agar seluruh batasan fisik kapal (kapasitas berat, batas grid, dan *overlap*) tetap terpenuhi secara konsisten.

2.1.4 Fungsi Objektif Alternatif (Bonus)

Selain fungsi objektif utama yang memaksimalkan total tarif pengiriman (*shipping fee*), dikembangkan dua fungsi objektif alternatif untuk mengeksplorasi prioritas waktu kedatangan serta pemanfaatan ruang dan kapasitas berat kapal. Keduanya menggunakan atribut kendaraan dan kapal yang tersedia pada dataset tanpa menambahkan variabel baru. Implementasi fungsi objektif ditunjukkan sebagai berikut:

```
def calculate_objective(self, obj_type=None):
        if not self.is_valid():
            return -1000000

        mode = obj_type or getattr(self, "objective_type",
"fee")

        if mode == "eta_priority":
            score = 0
            for v in self.vehicles:
                if v.is_loaded:
                    eta_val = max(1, int(getattr(v, "eta", 1)
or 1))
                    score += (v.shipping_fee / eta_val)
            return round(score)

        elif mode == "cap_efficiency":
            total_fee = self.get_total_fee()
            deck_area = self.ship.w * self.ship.l
            loaded_area = sum(v.w * v.l for v in
self.vehicles if v.is_loaded)
            loaded_weight = self.get_total_weight()

            space_utilization = (loaded_area / deck_area) if
deck_area > 0 else 0
            weight_utilization = (loaded_weight /
self.ship.MaxCapacity) if self.ship.MaxCapacity > 0 else 0
```

```
            bonus = (space_utilization * 5000) +
(weight_utilization * 5000)
            return round(total_fee + bonus)

        return self.get_total_fee()
```

a. Fungsi Objektif 2: ETA-*Weighted Fee* (*ETA Priority*)

Fungsi objektif kedua memberikan skor lebih tinggi kepada kendaraan yang memiliki tarif pengiriman besar dan perkiraan waktu kedatangan (*Estimated Time of Arrival* atau ETA) lebih kecil. Fungsi ini dirumuskan sebagai berikut:

$$f_2(S) = \sum_{v \in loaded(S)} \frac{v.shipping_fee}{v.eta}$$

Keterangan:

- S : state yang merepresentasikan konfigurasi penataan kendaraan
- $loaded(S)$: himpunan kendaraan yang berstatus dimuat pada state S
- $v.shipping_fee$: tarif pengiriman kendaraan v
- $v.eta$: perkiraan waktu kedatangan kendaraan v, dengan nilai positif

Pembagian tarif pengiriman dengan ETA membuat kendaraan dengan waktu kedatangan lebih dekat memperoleh skor lebih besar, jika tarifnya sama. Namun, fungsi ini tidak secara langsung memaksimalkan total tarif pengiriman. Artinya, solusi dengan skor f_2 tertinggi belum tentu menghasilkan pendapatan terbesar. Satuan ETA juga harus konsisten karena dapat memengaruhi nilai skor.

b. Fungsi Objektif 3: *Capacity Utilization (Capacity Efficiency)*

Fungsi objektif ketiga menggabungkan total tarif pengiriman dengan bonus berdasarkan luas kendaraan yang dimuat dan kapasitas berat kapal yang digunakan. Dengan fungsi ini, penilaian solusi tidak hanya bergantung pada total tarif pengiriman, tetapi juga pada penggunaan sumber daya kapal. Fungsi objektif ini dirumuskan sebagai berikut:

$$f_3(S) = F(S) + \left(\frac{A_{loaded}(S)}{A_{ship}} \times 5000\right) + \left(\frac{W_{loaded}(S)}{MaxCapacity} \times 5000\right)$$

dengan:

$$F(S) = \sum_{v \in loaded(S)} v.shipping_fee$$

$$A_{loaded}(S) = \sum_{v \in loaded(S)} (v.w \times v.l)$$

$$A_{ship} = ship.w \times ship.l$$

$$W_{loaded}(S) = \sum_{v \in loaded(S)} v.weight$$

Keterangan:

- $F(S)$: total tarif pengiriman kendaraan yang dimuat
- $A_{loaded}(S)$: luas kendaraan berdasarkan dimensi lebar (w) dan panjang (l)
- A_{ship} : luas geladak kapal
- $W_{loaded}(S)$: total berat kendaraan yang dimuat
- ship.MaxCapacity : maksimum kapasitas berat kapal
- 5000 : konstanta bobot bonus untuk setiap komponen utilisasi

Kedua bonus memberikan skor tambahan berdasarkan perbandingan luas kendaraan dengan luas geladak serta total berat kendaraan dengan kapasitas maksimum kapal. Konstanta digunakan untuk mengatur besarnya pengaruh kedua komponen tersebut terhadap skor akhir, bukan sebagai pendapatan tambahan. Besarnya bonus perlu diperiksa agar pengaruhnya tidak terlalu kecil atau terlalu besar dibandingkan total tarif pengiriman.

Namun, total luas kendaraan belum tentu menunjukkan bahwa geladak telah digunakan secara optimal. Bentuk dan posisi kendaraan masih dapat menyisakan ruang yang sulit diisi. Karena itu, validitas susunan kendaraan tetap diperiksa secara terpisah untuk memastikan tidak ada kendaraan yang bertumpang tindih, keluar dari batas geladak, atau menyebabkan berat total melebihi kapasitas kapal.

2.2 Penjelasan Pembangkitan *State* Awal (*Initial State*)

State awal dibangkitkan secara acak melalui `initial_random_state()`, kemudian diperbaiki agar valid sebelum dipakai oleh algoritma *local search*.

```
def initial_random_state(ship, vehicles_data, objective_type="fee"):
    vehicles = [Vehicle(**data) for data in vehicles_data]
    indices = list(range(len(vehicles)))
    random.shuffle(indices)

    for i in indices:
        v = vehicles[i]
        if random.random() < 0.7:
            v.orientation = random.choice(["Horizontal", "Vertical"])
            w_eff, l_eff = v.get_dimensions()
            max_x = max(0, ship.w - w_eff)
            max_y = max(0, ship.l - l_eff)
            v.x = random.randint(0, max_x)
            v.y = random.randint(0, max_y)
            v.is_loaded = True
        else:
            v.is_loaded = False

    state = State(ship, vehicles, objective_type=objective_type)
    state.repair_to_valid()
    return state
```

Cara Kerja:

1. Seluruh data kendaraan (vehicle_data) diubah menjadi objek Vehicle, kemudian urutan indeksnya diacak menggunakan random.shuffle(indices).
2. Setiap kendaraan diberi peluang 70% (`random.random() < 0.7`) untuk dimuat ke geladak kapal. Jika dipilih, kendaraan yang dimuat mendapat orientasi acak dan koordinat posisi (x,y) acak yang dibatasi agar tidak keluar dari grid kapal. Kendaraan sisanya akan ditempatkan di luar geladak (`is_loaded = False`).
3. Objek State yang terbentuk kemudian memanggil fungsi `repair_to_valid()`. Fungsi ini memeriksa kendaran yang termuat secara berurutan dan menurunkan status muatan (`is_loaded = False`) kendaraan jika ditemukan pelanggaran batasan.

Keuntungan Pendekatan Ini:

1. ***State* awal selalu valid,** sehingga seluruh pencarian dimulai dari titik yang layak dan nilai objektif awal yang tidak pernah menyentuh skor penalti.

2. ***State* awal bervariasi** karena pemilihan kendaraan, orientasi, dan posisi dilakukan secara acak (*random shuffle*). Hal ini penting bagi *Random Restart Hill-climbing* (*state* tiap *restart* berbeda), populasi *Genetic Algorithm* (k individu valid yang beragam), dan *Simulated Annealing* (agar pencarian tidak selalu berawal dari pola yang sama).

2.3 Penjelasan Implementasi Algoritma *Hill-Climbing*

Hill-Climbing adalah salah satu metode *local search* yang dipakai untuk mencari solusi yang lebih baik secara bertahap, berdasarkan nilai fungsi objektif. Di implementasi ini, pencarian dimulai dari *initial state* yang dibuat secara acak lalu diperbaiki (*repair*) agar valid. Kemudian, *neighbor state* dicari dengan tiga cara, yaitu menukar dua kendaraan (yang di dalam maupun di luar kapal), memindahkan kendaraan ke koordinat lain di geladak, atau mengubah orientasinya.

Di setiap iterasi, algoritma memeriksa semua *neighbor state* lalu memilih satu yang nilainya paling bagus (total *fee* yang lebih tinggi). *Neighbor* yang tidak valid akan otomatis mendapatkan skor -1000000, jadi tidak akan terpilih. Jika *neighbor value* lebih baik dari *current state*, maka algoritma akan berpindah ke *neighbor*. Proses ini terus diulang sampai tidak ada lagi *neighbor* yang memiliki *value* lebih baik. Di kondisi itu lah algoritma sudah sampai di *local optimum*.

2.3.1 Algoritma *Steepest Ascent Hill-Climbing*

a. Deskripsi *Steepest Ascent Hill-Climbing*

Steepest Ascent Hill-Climbing mengevaluasi seluruh *neighbor* valid dari *current state,* lalu berpindah ke *neighbor* dengan nilai objektif tertinggi, dengan syarat nilainya lebih besar dari *current state*. Jika beberapa *neighbor* memiliki nilai terbaik yang sama, satu diantaranya dipilih secara acak. Algoritma ini berhenti saat tidak ada *neighbor* yang nilainya lebih baik.

Pada kasus ini, algoritma mencari kombinasi kendaraan yang dimuat beserta posisi dan orientasinya agar total *shipping fee* setinggi mungkin, tanpa melanggar kapasitas berat, batas grid, dan *overlap* kendaraan.

b. *Source Code Steepest Ascent Hill-Climbing*

```
def steepest_ascent_hill_climbing(initial_state,
max_iterations=1000):
    start_time = time.time()
    current_state = initial_state.copy()
```

```
    current_val = current_state.calculate_objective()

    history_objective = [current_val]
    iterations = 0
    total_evaluations = 0

    while iterations < max_iterations:
        valid_neighbors, trial_evals =
generate_all_neighbors(current_state)
        total_evaluations += trial_evals

        if not valid_neighbors:
            break

        best_val = max(n.calculate_objective() for n in
valid_neighbors)
        if best_val <= current_val:
            break

        best_candidates = [n for n in valid_neighbors if
n.calculate_objective() == best_val]
        current_state = random.choice(best_candidates)
        current_val = best_val
        iterations += 1
        history_objective.append(current_val)

    return {
        "initial_state": initial_state,
        "final_state": current_state,
        "best_objective": current_val,
        "history_objective": history_objective,
        "total_iterations": iterations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": current_state.is_valid()
    }
```

c. Penjelasan *Source Code Steepest Ascent Hill-Climbing*

- **Fungsi Algoritma *Steepest Ascent Hill-Climbing***

  Fungsi `steepest_ascent_hill_climbing()` menerima beberapa parameter utama:

  - `initial_state`: *State* awal berupa penataan kendaraan di geladak.
  - `max_iterations`: Batas maksimum iterasi perpindahan yang berhasil (`iterations`) (*default*: 1000).

Fungsi mengembalikan sebuah *dictionary* berisi:

- `initial_state`: *State* awal yang pertama kali digunakan.
- `final_state`: Solusi terbaik keseluruhan (*global best*) yang ditemukan setelah seluruh proses pencarian selesai.
- `best_objective`: Nilai objektif tertinggi yang berhasil dicapai (*shipping fee* maksimum)
- `history_objective`: Riwayat nilai objektif di setiap langkah perpindahan.
- `total_iterations & total_evaluations`: Akumulasi jumlah perpindahan yang berhasil (`iterations`) dan jumlah seluruh kandidat *move* yang dicoba (`total_evaluations`).
- `execution_time`: Total waktu eksekusi program dalam satuan detik.
- `valid`: Status validitas dari *state* akhir.

- **Deklarasi Variabel Algoritma *Steepest Ascent Hill-Climbing***

  Beberapa variabel penting yang digunakan antara lain:

  - `current_state & current_val`: Salinan *state* yang sedang ditelusuri beserta nilai fungsi objektifnya saat ini.
  - `history_objective`: Daftar riwayat nilai objektif untuk grafik *trajectory*.
  - `iterations & total_evaluations`: Penghitung perpindahan yang berhasil (`iterations`) dan akumulasi *trial_evals* dari seluruh iterasi (`total_evaluations`).
  - `best_val`: Nilai objektif tertinggi yang ditemukan di antara seluruh *neighbor* valid pada iterasi tersebut.
  - `best_candidates`: Daftar *neighbor* yang memiliki nilai setara dengan `best_val` agar pemilihan di antara nilai yang sama dapat dilakukan secara acak menggunakan `random.choice`.

- **Inisialisasi Nilai Awal Algoritma *Steepest Ascent Hill-Climbing***

  Proses pencarian diawali dengan mencatat waktu mulai menggunakan time.time(), menyalin *state* awal agar objek asli tidak berubah (`initial_state.copy()`),

menghitung nilai objektif awalnya, serta menginisialisasi variabel pelacak metrik dan riwayat objektif.

```
    start_time = time.time()
    current_state = initial_state.copy()
    current_val = current_state.calculate_objective()

    history_objective = [current_val]
    iterations = 0
    total_evaluations = 0
```

- **Proses Perulangan (*Loop*) Pencarian Algoritma *Steepest Ascent Hill-Climbing***

  Algoritma menjalankan perulangan `while` selama jumlah iterasi masih di bawah `max_iterations.` Setiap iterasi memanggil fungsi `generate_all_neighbors()` untuk mendapatkan seluruh *neighbor* valid beserta jumlah *trial*-nya.

```
while iterations < max_iterations:
        valid_neighbors, trial_evals =
generate_all_neighbors(current_state)
        total_evaluations += trial_evals

        if not valid_neighbors:
            break
```

- **Membangkitkan Tetangga dan Mencari Tetangga dengan Nilai Terbaik**

  Variabel `best_val` dicari secara efisien menggunakan fungsi `max()` yang mengevaluasi seluruh *neighbor* valid. Jikai nilai terbaik tersebut tidak lebih besar dari *state* saat ini (`best_val <= current_val`), algoritma langsung berhenti (*break*). Jika ditemukan peningkatan, *neighbor* yang nilainya sama persis dengan `best_val` dikumpulkan ke dalam `best_candidates.`

```
       best_val = max(n.calculate_objective() for n in
valid_neighbors)
        if best_val <= current_val:
            break

        best_candidates = [n for n in valid_neighbors if
n.calculate_objective() == best_val]
        current_state = random.choice(best_candidates)
        current_val = best_val
```

```
        iterations += 1
        history_objective.append(current_val)
```

- **Menyimpan Hasil Akhir Algoritma *Steepest Ascent Hill-Climbing***

  Setelah perulangan selesai, waktu eksekusi total dihitung menggunakan time.time() - start_time dan algoritma mengembalikan sleuruh ringkasan hasil pencarian dalam bentuk *dictionary*.

```
   return {
        "initial_state": initial_state,
        "final_state": current_state,
        "best_objective": current_val,
        "history_objective": history_objective,
        "total_iterations": iterations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": current_state.is_valid()
    }
```

d. Kesimpulan *Steepest Ascent Hill-Climbing*

Algoritma *Steepest Ascent Hill-Climbing* adalah pendekatan *local search* yang sistematis karena selalu mengevaluasi seluruh *neighbor* valid sebelum berpindah, sehingga setiap langkah mengambil peningkatan terbesar yang tersedia. Pemilihan secara acak hanya terjadi ketika ada beberapa *neighbor* yang nilainya setara.

Kelemahannya, algoritma ini dapat terjebak pada *local optimum* atau *plateau*. Pada kasus penataan kendaraan di geladak ini, *plateau* mudah terbentuk karena banyak pergerakan (*rotate, translate, swap*) yang tidak mengubah total *fee*, sedangkan peningkatan *fee* biasanya memerlukan pemuatan kendaraan baru yang harus tetap mematuhi batasan ruang dan kapasitas berat yang tersedia.

2.3.2 Algoritma *Hill-Climbing with Sideways Move*

a. Deskripsi *Hill-Climbing with Sideways Move*

Hill-Climbing with Sideways Move merupakan varian dari algoritma *Hill-Climbing* yang dirancang untuk mengatasi permasalahan terjebak pada plateau atau

daerah datar dalam ruang solusi, yaitu kondisi di mana beberapa *neighbor state* memiliki nilai fungsi objektif yang sama dengan *current state*.

Berbeda dengan *Steepest Ascent Hill-Climbing* yang hanya berpindah jika ditemukan neighbor dengan nilai fungsi objektif lebih baik, algoritma ini juga mengizinkan perpindahan ke *neighbor* dengan nilai yang sama, selama jumlah perpindahan tersebut belum melebihi batas maksimum yang telah ditentukan (*maximum sideways move*). Dengan demikian, algoritma dapat terus menjelajahi area pencarian yang datar untuk mencari peluang peningkatan nilai di iterasi berikutnya.

b. Source Code Hill-Climbing with Sideways Move

```
def sideways_move_hill_climbing(initial_state,
max_sideways_moves=50, max_iterations=1000):
    start_time = time.time()
    current_state = initial_state.copy()
    current_val = current_state.calculate_objective()

    history_objective = [current_val]
    total_iterations = 0
    total_evaluations = 0
    sideways_count = 0

    while total_iterations < max_iterations:
        neighbors_res = generate_all_neighbors(current_state)
        if isinstance(neighbors_res, tuple):
            neighbors, trial_evals = neighbors_res[0],
neighbors_res[1]
            total_evaluations += trial_evals
        else:
            neighbors = neighbors_res

        if not neighbors:
            break

        best_val = float("-inf")
        best_candidates = []
        for neighbor in neighbors:
            val = neighbor.calculate_objective()
            if val > best_val:
                best_val = val
                best_candidates = [neighbor]
            elif val == best_val:
                best_candidates.append(neighbor)
```

```
        best_neighbor = random.choice(best_candidates)

        if best_val > current_val:
            sideways_count = 0
        elif best_val == current_val and sideways_count <
max_sideways_moves:
            sideways_count += 1
        else:
            break

        current_state = best_neighbor
        current_val = best_val
        total_iterations += 1
        history_objective.append(current_val)

    return {
        "initial_state": initial_state,
        "final_state": current_state,
        "best_objective": current_val,
        "history_objective": history_objective,
        "total_iterations": total_iterations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": current_state.is_valid(),
        "sideways_moves_count": sideways_count
    }
```

c. Penjelasan *Source Code Hill-Climbing with Sideways Move*

- **Fungsi Algoritma *Hill-Climbing with Sideways Move***

  Fungsi `sideways_move_hill_climbing()` digunakan untuk mengimplementasikan *Hill-Climbing with Sideways Move.*

  Fungsi menerima parameter:

  - initial_state: objek state awal (konfigurasi awal) yang akan dievaluasi.
  - max_sideways_moves : jumlah maksimum pergerakan menyamping (nilai sama) yang diizinkan (default 50).
  - max_iterations: batas maksimum iterasi algoritma secara keseluruhan (default: 1000).

  Hasil yang dikembalikan berupa *dictionary* (kamus) yang memuat berbagai informasi evaluasi seperti state awal dan akhir, nilai objektif terbaik, riwayat pencarian, waktu eksekusi, serta validitas solusi akhir.

- **Deklarasi Variabel Algoritma *Hill-Climbing with Sideways Move***

  Variabel utama yang digunakan antara lain:

  - current_state: menyimpan state yang sedang dievaluasi.
  - current_val : nilai fungsi objektif dari current state
  - sideways_count: menghitung jumlah pergerakan samping (stagnan) beruntun yang sudah dilakukan.
  - best_val: menyimpan skor nilai evaluasi terbaik sementara dari neighbor yang dibangkitkan.
  - best_candidates: daftar (list) neighbor yang memiliki skor objektif terbaik yang sama (setara).
  - best_neighbor: neighbor yang dipilih secara acak (stokastik) dari daftar best_candidates.
  - history_objective: menyimpan riwayat keseluruhan nilai objektif tiap iterasi.
  - total_iterations dan total_evaluations: penghitung total iterasi dan proses evaluasi neighbor.
  - start_time dan execution_time: digunakan untuk mengukur total waktu eksekusi algoritma.

- **Inisialisasi Nilai Awal Algoritma *Hill-Climbing with Sideways Move***

  Tahap awal dilakukan dengan menyalin *initial state* ke `current_state`, menghitung nilai objektifnya, dan mengatur penghitung iterasi serta pergerakan samping.

```
start_time = time.time()

current_state = initial_state.copy()
current_val = current_state.calculate_objective()

history_objective = [current_val]
total_iterations = 0
total_evaluations = 0
sideways_count = 0
```

- **Proses Perulangan (*Loop*) Pencarian Algoritma *Hill-Climbing with Sideways Move***

  Proses pencarian dilakukan selama iterasi belum melampaui batas maksimum (`max_iterations`). Setiap iterasi akan membangkitkan semua neighbor state dari state saat ini untuk menilai kualitas masing-masing.

```
   while total_iterations < max_iterations:
        neighbors_res =
generate_all_neighbors(current_state)
        if isinstance(neighbors_res, tuple):
            neighbors, trial_evals = neighbors_res[0],
neighbors_res[1]
            total_evaluations += trial_evals
        else:
            neighbors = neighbors_res

        if not neighbors:
            break
```

- **Membangkitkan Tetangga dan Mencari Tetangga Terbaik**

  Setiap neighbor yang dihasilkan dihitung nilai objektifnya. Jika ditemukan neighbor dengan skor objektif lebih besar, ia akan menimpa nilai terbaik dan menjadikannya kandidat tunggal. Jika ditemukan neighbor dengan nilai yang sama dengan skor terbaik sementara, neighbor tersebut ditambahkan ke daftar kandidat (`best_candidates`). Setelah evaluasi selesai, satu tetangga terbaik dipilih secara acak untuk meminimalisasi pola perpindahan yang deterministik.

```
best_val = float("-inf")
best_candidates = []
for neighbor in neighbors:
    val = neighbor.calculate_objective()
    if val > best_val:
        best_val = val
        best_candidates = [neighbor]
    elif val == best_val:
        best_candidates.append(neighbor)

best_neighbor = random.choice(best_candidates)
```

- **Memperbarui State Saat Ini Jika Ada Peningkatan Value**

Apabila terdapat pergerakan yang meningkatkan nilai objektif, algoritma mereset penghitung pergerakan samping menjadi nol. Jika tidak ada peningkatan namun nilainya sejajar (plateau) dan batas `max_sideways_moves` belum terlampaui, algoritma menambah nilai sideways_count. Jika tidak ada satupun kondisi yang terpenuhi, loop dihentikan. Setelah pengecekan arah selesai, iterasi dan state diperbarui secara serempak.

```
if best_val > current_val:
    sideways_count = 0
elif best_val == current_val and sideways_count <
max_sideways_moves:
    sideways_count += 1
else:
    break

current_state = best_neighbor
current_val = best_val
total_iterations += 1
history_objective.append(current_val)
```

- **Menyimpan Hasil Akhir Algoritma *Hill-Climbing with Sideways Move***

  Setelah proses pencarian berakhir akibat batas iterasi atau tidak tersedianya neighbor yang menguntungkan, waktu eksekusi dikalkulasi dan seluruh matriks evaluasi serta solusi akhir dikembalikan dalam bentuk *dictionary*.

```
   return {
        "initial_state": initial_state,
        "final_state": current_state,
        "best_objective": current_val,
        "history_objective": history_objective,
        "total_iterations": total_iterations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": current_state.is_valid(),
        "sideways_moves_count": sideways_count
    }
```

d. Kesimpulan *Hill-Climbing with Sideways Move*

Algoritma *Hill-Climbing with Sideways Move* merupakan pengembangan dari *Hill-Climbing* klasik yang menambahkan kemampuan untuk berpindah ke *neighbor state* dengan nilai objektif yang sama. Fitur ini memungkinkan algoritma tetap menjelajahi ruang

solusi saat menghadapi *plateau* atau area datar, di mana tidak ada peningkatan nilai fungsi objektif yang langsung terlihat. Dengan memberikan batasan jumlah pergerakan samping (*sideways moves*), algoritma tetap menjaga efisiensi pencarian sekaligus memperluas peluang menemukan solusi tata letak yang lebih baik di iterasi berikutnya. Pendekatan ini menjadikan algoritma lebih fleksibel dan adaptif dalam menghadapi kondisi stagnasi lokal.

Dalam konteks *2D Vehicle Packing* pada satu geladak kapal, metode ini membantu meningkatkan efisiensi penempatan kendaraan untuk memaksimalkan total tarif pengiriman (*shipping fee*) tanpa melanggar batas koordinat dan kapasitas berat maksimum. Perpindahan samping memungkinkan algoritma menggeser atau memutar kendaraan di dalam dek untuk membuka ruang baru yang mungkin sebelumnya terlewatkan oleh metode konvensional. Meskipun tidak sepenuhnya menghindari risiko *local optimum*, *Hill-Climbing with Sideways Move* menawarkan keseimbangan yang baik antara eksploitasi dan eksplorasi, menghasilkan pencarian konfigurasi muatan yang lebih dinamis dan berpotensi menemukan solusi yang lebih mendekati optimum global.

2.3.3 Algoritma *Random Restart Hill-Climbing*

a. Deskripsi *Random Restart Hill-Climbing*

Random Restart Hill-Climbing merupakan varian dari algoritma *Hill-Climbing* yang dirancang untuk mengurangi risiko terjebak pada *local optimum*. Pendekatan ini dilakukan dengan cara mengulang proses pencarian dari beberapa *initial state* yang berbeda secara acak.

Eksekusi (*run*) pertama dijalankan menggunakan *initial state* yang telah ditentukan, sedangkan *run* berikutnya akan memanfaatkan *state* acak yang baru dari `initial_random_state`. Dari seluruh rangkaian proses *run* tersebut, *state* dengan nilai objektif tertinggi akan dipilih dan disimpan sebagai hasil akhir.

b. *Source Code Random Restart Hill-Climbing*

```
def random_restart_hill_climbing(initial_state=None,
ship=None, vehicles_data=None, max_restarts=5,
max_iterations_per_restart=1000):
    start_time = time.time()

    if initial_state is not None:
        ship = ship or initial_state.ship
```

```
        vehicles_data = vehicles_data or [{"v_id": v.id, "w":
v.w, "l": v.l, "fee": v.shipping_fee, "weight": v.weight,
"eta": v.eta} for v in initial_state.vehicles]

    if ship is None or vehicles_data is None:
        raise ValueError("Must provide either initial_state
or (ship, vehicles_data).")

    best_overall_state = None
    best_overall_value = float("-inf")
    combined_history = []
    total_iterations_all = 0
    total_evaluations_all = 0
    first_initial_state = initial_state.copy() if
initial_state is not None else None
    obj_type = getattr(initial_state, "objective_type",
"fee") if initial_state is not None else "fee"

    for r in range(max_restarts):
        if r == 0 and initial_state is not None:
            restart_init = initial_state.copy()
        else:
            restart_init = initial_random_state(ship,
vehicles_data, objective_type=obj_type)

        if first_initial_state is None:
            first_initial_state = restart_init.copy()

        result = steepest_ascent_hill_climbing(restart_init,
max_iterations=max_iterations_per_restart)

        total_iterations_all += result["total_iterations"]
        total_evaluations_all += result["total_evaluations"]
        combined_history.extend(result["history_objective"])

        if result["best_objective"] > best_overall_value:
            best_overall_value = result["best_objective"]
            best_overall_state = result["final_state"]

    return {
        "initial_state": first_initial_state,
        "final_state": best_overall_state,
        "best_objective": best_overall_value,
        "history_objective": combined_history,
        "total_iterations": total_iterations_all,
        "total_evaluations": total_evaluations_all,
        "restarts_count": max_restarts,
        "execution_time": time.time() - start_time,
        "valid": best_overall_state.is_valid() if
best_overall_state else True
```

```
    }
```

c. Penjelasan *Source Code Random Restart Hill-Climbing*

- **Fungsi Algoritma *Random Restart Hill-Climbing***

  Fungsi `random_restart_hill_climbing()` menerima beberapa parameter utama:

  - `initial_state`: *State* awal yang digunakan khusus untuk *run* pertama.
  - `ship & vehicles_data`: Konfigurasi kapal dan data kendaraan yang digunakan untuk membangkitkan *state* acak melalui `initial_random_state()`. Jika kosong, keduanya dapat diturunkan otomatis dari `initial_state`.
  - `max_restarts`: Jumlah total *run* (percobaan ulang) pencarian.
  - `max_iterations_per_restart`: Batas maksimum iterasi untuk setiap satu kali siklus *run* (*default*: 1000).

  Fungsi mengembalikan sebuah *dictionary* yang berisi:

  - `initial_state`: *State* awal yang pertama kali digunakan.
  - `final_state`: Solusi terbaik keseluruhan (*global best*) yang ditemukan dari seluruh rangkaian *restart*.
  - `best_objective`: Nilai objektif tertinggi yang berhasil dicapai.
  - `history_objective`: Kumpulan riwayat nilai objektif dari seluruh proses iterasi.
  - `total_iterations & total_evaluations`: Akumulasi jumlah iterasi dan evaluasi dari awal hingga akhir proses.
  - `restarts_count`: Jumlah total siklus *restart* yang dilakukan.
  - `execution_time`: Total waktu eksekusi program dalam satuan detik.
  - `valid`: Status validitas dari *state* akhir.

- **Deklarasi dan Inisialisasi Variabel Utama Algoritma *Random Restart Hill-Climbing***

  Beberapa variabel penting yang digunakan antara lain:

  - `best_overall_state`: Menyimpan referensi objek *state* terbaik secara keseluruhan.
  - `best_overall_value`: Diinisialisasi dengan nilai tak hingga negatif `(float('-inf'))` agar nilai objektif pertama pasti bisa memperbaruinya.
  - `combined_history`: Mengumpulkan dan menggabungkan riwayat nilai objektif

dari setiap proses *run*.

- `total_iterations_all & total_evaluation_all:` Akumulator untuk menghitung total beban komputasi secara keseluruhan.
- `first_initital_state`: State awal pada *run* pertama, digunakan sebagai acuan `initial_state` pada hasil akhir.
- `restart_init`: *State* awal yang digunakan khusus untuk proses *run* yang sedang berjalan.

- **Inisialisasi Nilai Awal Algoritma *Random Restart Hill-Climbing***

  Proses diawali dengan mencatat waktu mulai menggunakan time.time(). Jika `initial_state` diberikan, fungsi akan mengekstrak data `ship` dan `vehicles_data`, kemudian menginisialisasi variabel pelacak metrik.

```
   start_time = time.time()

    if initial_state is not None:
        ship = ship or initial_state.ship
        vehicles_data = vehicles_data or [{"v_id": v.id, "w":
v.w, "l": v.l, "fee": v.shipping_fee, "weight": v.weight,
"eta": v.eta} for v in initial_state.vehicles]

    if ship is None or vehicles_data is None:
        raise ValueError("Must provide either initial_state
or (ship, vehicles_data).")

    best_overall_state = None
    best_overall_value = float("-inf")
    combined_history = []
    total_iterations_all = 0
    total_evaluations_all = 0
    first_initial_state = initial_state.copy() if
initial_state is not None else None
    obj_type = getattr(initial_state, "objective_type",
"fee") if initial_state is not None else "fee"
```

- **Proses Perulangan (*Loop*) Pencarian Algoritma *Random Restart Hill-Climbing***

  Algoritma menjalankan perulangan sebanyak `max_restarts` kali. Pada *run* pertama (r == 0) dan initial_state tersedia, algoritma memakai salinan dari `initial_state`. Pada run berikutnya (r > 0), algoritma membangkitkan *initial state* acak yang baru melalui `initial_random_state`.

```
   for r in range(max_restarts):
```

```
        if r == 0 and initial_state is not None:
            restart_init = initial_state.copy()
        else:
            restart_init = initial_random_state(ship,
vehicles_data, objective_type=obj_type)

        if first_initial_state is None:
            first_initial_state = restart_init.copy()
```

- **Menjalankan *Steepest Ascent* dan Memperbarui State Terbaik**

  Setiap *state* awal pada masing-masing *restart* dieksekusi menggunakan algoritma lokal (*Steepest Ascent*). Hasil dari pencarian itu diakumulasikan ke dalam statistik lokal dan jika ditemukan nilai yang lebih tinggi dari riwayat sebelumnya, maka `best_objective & best_overall_value` akan diperbarui.

```
       result = steepest_ascent_hill_climbing(restart_init,
max_iterations=max_iterations_per_restart)

        total_iterations_all += result["total_iterations"]
        total_evaluations_all += result["total_evaluations"]
        combined_history.extend(result["history_objective"])

        if result["best_objective"] > best_overall_value:
            best_overall_value = result["best_objective"]
            best_overall_state = result["final_state"]
```

- **Menyimpan dan Mengembalikan Hasil Akhir**

  Setelah seluruh proses *restart* selesai, total waktu eksekusi akan dihitung dan algoritma mengembalikan struktur data terpusat (*dictionary*) yang berisi seluruh metrik pencarian.

```
   return {
        "initial_state": first_initial_state,
        "final_state": best_overall_state,
        "best_objective": best_overall_value,
        "history_objective": combined_history,
        "total_iterations": total_iterations_all,
        "total_evaluations": total_evaluations_all,
        "restarts_count": max_restarts,
        "execution_time": time.time() - start_time,
        "valid": best_overall_state.is_valid() if
best_overall_state else True
    }
```

d. Kesimpulan Algoritma *Random Restart Hill-Climbing*

Algoritma *Random Restart Hill-Climbing* merupakan pengembangan dari metode *Hill-Climbing* standar yang dirancang untuk mengatasi keterbatasan dalam menghadapi *local optimum*. Dengan menambahkan mekanisme *restart* secara acak, di mana proses *run* pertama dimulai dari *initial state* tertentu dan *run* berikutnya menggunakan *initial random state,* algoritma ini memungkinkan pencarian dilakukan berulang kali dari titik awal berbeda.

Setiap kali pencarian selesai pada suatu proses *run*, *state* dengan nilai objektif tertinggi akan diseleksi dan disimpan sebagai hasil akhir. Pendekatan ini efektif untuk memperluas jangkauan eksplorasi dan meningkatkan peluang menemukan solusi yang mendekati optimal pada permasalahan kompleks, meskipun memerlukan manajemen batas iterasi dan jumlah *restart* yang seimbang agar efisiensi waktu komputasi tetap terjaga.

2.3.4 Algoritma *Stochastic Hill-Climbing*

a. Deskripsi *Stochastic Hill-Climbing*

Stochastic Hill-Climbing merupakan varian dari algoritma *Hill-Climbing* yang dirancang untuk memperkenalkan unsur acak pada pemilihan *neighbor* (tetangga). Berbeda dengan *Steepest Ascent* yang selalu memilih *neighbor* terbaik secara deterministik, algoritma ini menyaring seluruh *neighbor* valid yang memiliki nilai fungsi objektif lebih baik dari *current state*.

Dari kumpulan *neighbor* yang memenuhi kriteria peningkatan (*better neighbors*), algoritma memilih salah satu secara acak menggunakan fungsi pemilihan acak (random.choice). Algoritma ini hanya menerima *neighbor* dengan fungsi objektif yang lebih baik dan tidak menerima *neighbor* yang bernilai sama atau lebih rendah.

b. *Source Code Stochastic Hill-Climbing*

```
def stochastic_hill_climbing(initial_state,
max_iterations=1000):
    start_time = time.time()
    current_state = initial_state.copy()
```

```
    current_val = current_state.calculate_objective()

    history_objective = [current_val]
    official_iterations = 0
    total_evaluations = 0

    for _ in range(max_iterations):
        valid_neighbors, trial_evals =
generate_all_neighbors(current_state)
        total_evaluations += trial_evals

        if not valid_neighbors:
            break

        better_neighbors = []
        for n in valid_neighbors:
            val = n.calculate_objective()
            if val > current_val:
                better_neighbors.append((n, val))

        if not better_neighbors:
            break

        current_state, current_val =
random.choice(better_neighbors)
        official_iterations += 1
        history_objective.append(current_val)

    return {
        "initial_state": initial_state,
        "final_state": current_state,
        "best_objective": current_val,
        "history_objective": history_objective,
        "total_iterations": official_iterations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": current_state.is_valid()
    }
```

c. Penjelasan *Source Code Stochastic Hill-Climbing*

- **Fungsi Algoritma *Stochastic Hill-Climbing***

  Fungsi `stochastic_hill_climbing()` menerima beberapa parameter utama:
  - `initial_state`: *State* awal yang menjadi titik awal pencarian solusi.
  - `max_iterations`: Batas maksimum iterasi perulangan yang diizinkan (*default*: 1000).

Fungsi mengembalikan sebuah *dictionary* yang berisi:

- `initial_state`: State awal yang pertama kali digunakan.
- `final_state`: Solusi terbaik (*best state*) yang ditemukan setelah seluruh proses pencarian selesai.
- `best_objective`: Nilai objektif tertinggi yang berhasil dicapai.
- `history_objective`: Riwayat nilai objekif di setiap langkah perpindahan.
- `total_iterations`: Jumlah total perpindahan *state* yang berhasil dilakukan.
- `total_evaluations`: Akumulasi jumlah evaluasi fungsi objektif dari awal hingga akhir.
- `execution_time`: Total waktu eksekusi program dalam satuan detik.
- `valid`: Status kevalidan dari *final state*.

- **Deklarasi Variabel Algoritma *Stochastic Hill-Climbing***

  Beberapa variabel penting yang digunakan dalam fungsi ini antara lain:

  - `start_time`: Mencatat waktu awal eksekusi program (dalam detik) menggunakan time.time() untuk menghitung total durasi proses pencarian di akhir.
  - `current_state & current_val`: Menyimpan state yang sedang dievaluasi saat ini beserta nilai fungsi objektifnya. *State* awal disalin menggunakan .copy() agar objek aslinya tidak ikut berubah.
  - `history_objective`: Berbentuk list yang berfungsi untuk mencatat riwayat perubahan nilai objektif di setiap langkah perpindahan.
  - `official_iterations`: Penghitung jumlah perpindahan *state* yang berhasil dilakukan (*neighbor value* > *current value*) dari awal sampai perulangan berhenti.
  - `total_evaluations`: Akumulasi total evaluasi yang dilakukan selama proses pencarian berlangsung.
  - `valid_neighbors & trial_evals`: Mengumpulkan daftar *neighbor* valid hasil pembangkitan (generate_all_neighbors) beserta jumlah evaluasi yang dilakukan pada iterasi tersebut.
  - `better_neighbors`: Daftar sementara yang berisi pasangan *neighbor* valid yang nilainya lebih besar (*val* > *current val*) dari nilai *state* saat ini.

- **Inisialisasi Nilai Awal Algoritma *Stochastic Hill-Climbing***

Proses pencarian diawali dengan mencatat waktu mulai menggunakan time.time(), menyalin state awal agar objek asli tidak berubah (initial_state.copy()), menghitung nilai objektif awalnya, serta menginisialisasi variabel pelacak metrik.

```
start_time = time.time()
current_state = initial_state.copy()
current_val = current_state.calculate_objective()

history_objective = [current_val]
official_iterations = 0
total_evaluations = 0
```

- **Proses Perulangan (*Loop*) Pencarian Algoritma *Stochastic Hill-Climbing***

  Algoritma menjalankan perulangan `for` sebanyak max.iterations. Pada setiap iterasi, seluruh *neighbor* valid dibangkitkan beserta jumlah evaluasinya.

```
   for _ in range(max_iterations):
        valid_neighbors, trial_evals =
generate_all_neighbors(current_state)
        total_evaluations += trial_evals

        if not valid_neighbors:
            break
```

- **Menyaring dan Memilih *Neighbor* yang Lebih Baik Secara Acak**

  Algoritma melakukan iterasi pada seluruh *neighbor* valid untuk mengevaluasi nilainya. Jika nilai suatu *neighbor* lebih besar dari nilai *state* saat ini (`val > current_val`), *neighbor* tersebut akan dimasukkan ke dalam daftar `better_neighbors`.

```
better_neighbors = []
    for n in valid_neighbors:
      val = n.calculate_objective()
      if val > current_val:
        better_neighbors.append((n, val))

    if not better_neighbors:
      break
```

- **Memilih Secara Acak dan Memperbarui State Saat Ini**

  Jika ditemukan satu atau lebih kandidat yang nilainya lebih baik, algoritma akan memilih salah satu secara acak menggunakan `random.choice(better_neighbors)` untuk memperbarui *state* saat ini.

```
 current_state, current_val =
random.choice(better_neighbors)
    official_iterations += 1
    history_objective.append(current_val)
```

- **Menyimpan Hasil Akhir Algoritma *Stochastic Hill-Climbing***

  Setelah perulangan selesai (sudah mencapai batas maksimum atau tidak ada lagi *neighbor* dengan nilai yang lebih baik), algoritma mengembalikan seluruh ringkasan hasil pencarian dalam bentuk *dictionary*.

```
   return {
        "initial_state": initial_state,
        "final_state": current_state,
        "best_objective": current_val,
        "history_objective": history_objective,
        "total_iterations": official_iterations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": current_state.is_valid()
    }
```

d. Kesimpulan Fungsi *Stochastic Hill-Climbing*

Algoritma *Stochastic Hill-Climbing* memberikan variasi pada jalur pencarian dengan memilih secara acak di antara semua kandidat solusi peningkatan yang tersedia. Kelebihannya adalah jalur yang dilewati tidak deterministik, sehingga hasil atau lintasan bisa berbeda pada setiap eksekusi. Namun, karena algoritma ini hanya menerima *neighbor* dengan *value* lebih baik dan tidak menerima *neighbor* dengan *value* yang sama atau lebih rendah, algoritma ini masih memiliki risiko terjebak di kondisi lokal optimum.

2.4 Penjelasan Implementasi Algoritma *Simulated Annealing*

a. Deskripsi Algoritma *Simulated Annealing*

Algoritma Simulated Annealing (SA) merupakan metode *local search* berbasis probabilistik yang terinspirasi dari proses pendinginan logam (*annealing*) dalam fisika. Ide dasarnya adalah mencari solusi optimal global dengan cara mengizinkan transisi ke solusi yang lebih buruk pada tahap awal pencarian (saat "suhu" masih tinggi) dengan probabilitas tertentu. Seiring berjalannya iterasi, suhu (*temperature*) akan diturunkan secara bertahap, sehingga probabilitas menerima solusi buruk akan semakin kecil, dan algoritma akan berkonvergensi ke solusi optimal.

Pada kasus pemuatan kargo (*cargo loading*), algoritma ini bertujuan untuk menempatkan kendaraan-kendaraan ke dalam geladak kapal sedemikian rupa untuk memaksimalkan total *ShippingFee* dari kendaraan yang diangkut. Proses utama dari Simulated Annealing dalam konteks ini meliputi:

1. Menginisialisasi *state* awal berupa penempatan kendaraan secara acak (*random*).
2. Menghitung nilai *objective function* (*ShippingFee*) dari *state* tersebut.
3. Membangkitkan *neighbor state* (solusi tetangga) yang valid dengan memindahkan kendaraan, menukar kendaraan, atau mengubah orientasinya.
4. Mengevaluasi selisih keuntungan (*delta_e*). Jika *neighbor* memberikan keuntungan lebih besar, solusi langsung diterima. Jika lebih kecil, solusi tetap bisa diterima berdasarkan perhitungan probabilitas *eksponensial* terhadap suhu saat ini.
5. Menurunkan suhu secara bertahap (*cooling schedule*) hingga proses berhenti (suhu mencapai minimum atau iterasi maksimal tercapai).
6. Menyimpan dan mengembalikan konfigurasi pemetaan geladak terbaik (*best state*) yang ditemukan.

b. *Source Code* Algoritma Simulated Annealing

```
import math
import random
import time
from tetangga import generate_random_valid_neighbor

def simulated_annealing(initial_state,
```

```
initial_temperature=1000.0, cooling_rate=0.95,
min_temperature=0.01, max_iterations=1500):
    start_time = time.time()

    current_state = initial_state.copy()
    current_val = current_state.calculate_objective()

    best_state = current_state.copy()
    best_val = current_val

    history_objective = [current_val]
    history_acceptance_prob = [1.0]

    iterations = 0
    total_evaluations = 0
    stuck_count = 0
    t = float(initial_temperature)

    step = 0
    while t > min_temperature and step < max_iterations:
        step += 1

        next_state, trial_evals =
generate_random_valid_neighbor(current_state)
        total_evaluations += trial_evals

        if next_state is None:
            stuck_count += 1
            history_acceptance_prob.append(0.0)
            t *= cooling_rate
            continue

        next_val = next_state.calculate_objective()
        delta_e = next_val - current_val

        if delta_e > 0:
            prob = 1.0
        else:
            try:
                prob = math.exp(delta_e / max(1e-9, t))
            except OverflowError:
                prob = 0.0

        history_acceptance_prob.append(round(prob, 4))

        if delta_e > 0 or random.random() < prob:
            current_state = next_state
            current_val = next_val
            iterations += 1
            history_objective.append(current_val)
```

```
            if current_val > best_val:
                best_val = current_val
                best_state = current_state.copy()
        else:
            stuck_count += 1

        t *= cooling_rate

    return {
        "initial_state": initial_state,
        "final_state": best_state,
        "best_objective": best_val,
        "history_objective": history_objective,
        "history_acceptance_prob": history_acceptance_prob,
        "stuck_count": stuck_count,
        "stuck_frequency": round(stuck_count / max(1, step),
4),
        "total_iterations": iterations,
        "total_evaluations": total_evaluations,
        "final_temperature": t,
        "execution_time": time.time() - start_time,
        "valid": best_state.is_valid()
    }
```

c. Penjelasan Source Code Algoritma *Simulated Annealing*

- **Fungsi Algoritma *Simulated Annealing***

  Fungsi Algoritma Simulated Annealing Fungsi simulated_annealing() digunakan untuk melakukan pencarian solusi penataan kargo optimal menggunakan metode SA. Parameter yang diterima oleh fungsi ini:

  - initial_state: konfigurasi acak penempatan kendaraan pada geladak kapal sebagai titik awal.
  - initial_temperature: parameter suhu awal (default: 1000.0).
  - cooling_rate: laju penurunan suhu pada setiap iterasi (default: 0.95).
  - min_temperature: batas suhu minimum untuk menghentikan pencarian.
  - max_iterations: batas langkah/iterasi maksimum yang diperbolehkan.

  Fungsi ini mengembalikan *dictionary* yang berisi *state* awal, *state* akhir (*best_state*), nilai objektif terbaik, riwayat probabilitas penerimaan, riwayat pencapaian fungsi objektif, metrik kemandekan pencarian, metrik evaluasi total, serta informasi durasi komputasi.

- **Deklarasi Variabel Algoritma *Simulated Annealing***

  Variabel-variabel utama yang digunakan meliputi:

  - current_state dan current_val: menyimpan konfigurasi saat ini dan nilai objektifnya (*ShippingFee*).
  - best_state dan best_val: menyimpan konfigurasi dengan keuntungan tertinggi yang ditemukan sejauh ini.
  - history_objective: *list* untuk mencatat perubahan nilai fungsi objektif terhadap banyak iterasi yang dilewati.
  - t: menyimpan suhu iterasi saat ini.
  - delta_e: selisih nilai keuntungan antara *neighbor state* dengan *current state*.
  - stuck_count dan total_evaluations: penghitung frekuensi algoritma terjebak (*stuck*) di *local optima* dan total fungsi evaluasi tetangga yang dipanggil.

- **Inisialisasi Nilai Awal Algoritma *Simulated Annealing***

  Pada tahap awal, state awal disalin ke variabel `current_state`. Nilai objektif dihitung dan disalin ke `best_val` sebagai pembanding awal. Suhu awal ditetapkan dari parameter `initial_temperature` dan akan menurun setiap iterasi dengan faktor `cooling_rate`.

```
    start_time = time.time()

    current_state = initial_state.copy()
    current_val = current_state.calculate_objective()

    best_state = current_state.copy()
    best_val = current_val

    history_objective = [current_val]
    history_acceptance_prob = [1.0]

    iterations = 0
    total_evaluations = 0
    stuck_count = 0
    t = float(initial_temperature)

    step = 0
```

- **Proses Perulangan (*Loop*) Pencarian Algoritma *Simulated Annealing***

Algoritma melakukan pencarian selama suhu `t` masih di atas batas `min_temperature` dan jumlah langkah pencarian (`step`) belum melebihi `max_iterations`. Pada setiap iterasi, algoritma menghasilkan *neighbor state* yang valid, menghitung nilai objektifnya, membandingkan dengan *state* saat ini, dan menentukan apakah akan berpindah ke *neighbor state* atau tidak.

```
   while t > min_temperature and step < max_iterations:
        step += 1

        next_state, trial_evals =
generate_random_valid_neighbor(current_state)
        total_evaluations += trial_evals

        if next_state is None:
            stuck_count += 1
            history_acceptance_prob.append(0.0)
            t *= cooling_rate
            continue

        next_val = next_state.calculate_objective()
```

- **Membangkitkan Tetangga dan Mencari Tetangga Terbaik**

  Fungsi `generate_random_valid_neighbor()` digunakan untuk membangkitkan *state* tetangga (misalnya memindahkan koordinat, menukar kendaraan, atau merotasi orientasi kendaraan). Selisih nilai objektif dihitung:

  ```
  delta_e = next_val - current_val
  ```

  Karena tujuannya memaksimalkan *ShippingFee*, jika `delta_e > 0`, artinya solusi baru lebih baik dan langsung diterima. Jika `delta_e <= 0`, solusi tetap bisa diterima dengan probabilitas $P = e^{\Delta E/T}$ yang nilainya akan semakin kecil seiring menurunnya suhu.

  ```
  if delta_e > 0:
      prob = 1.0
  else:
      try:
          prob = math.exp(delta_e / max(1e-9, t))
      except OverflowError:
          prob = 0.0
  ```

- **Memperbarui State Saat Ini Jika Ada Peningkatan Value**

  Apabila solusi tetangga lebih baik atau diterima secara probabilistik, maka *state* saat ini diperbarui. Jika solusi tersebut memiliki nilai *ShippingFee* tertinggi sejauh ini, rekor `best_state` juga ikut diperbarui:

```
if delta_e > 0 or random.random() < prob:
    current_state = next_state
    current_val = next_val
    iterations += 1
    history_objective.append(current_val)

    if current_val > best_val:
        best_val = current_val
        best_state = current_state.copy()
else:
    stuck_count += 1

t *= cooling_rate
```

  Langkah ini memastikan algoritma mampu keluar dari *local optimum* di tahap awal pencarian, namun tetap berfokus pada eksploitasi solusi terbaik seiring penurunan suhu.

- **Menyimpan Hasil Akhir Algoritma *Simulated Annealing***

  Pada setiap iterasi, nilai objektif dan probabilitas penerimaan disimpan untuk keperluan analisis kelak (seperti plot grafik). Selain itu, mekanisme `stuck_count` diterapkan untuk mencatat seberapa sering solusi ditolak atau gagal menghasilkan *neighbor* yang valid, yang kemudian dirasiokan menjadi persentase `stuck_frequency`. Setelah iterasi selesai atau suhu di bawah ambang batas, fungsi mengembalikan hasil pencarian berupa *dictionary* yang berisi *best state*, status validitas, dan metrik-metrik terkait pencarian.

```
   return {
        "initial_state": initial_state,
        "final_state": best_state,
        "best_objective": best_val,
        "history_objective": history_objective,
        "history_acceptance_prob": history_acceptance_prob,
        "stuck_count": stuck_count,
        "stuck_frequency": round(stuck_count / max(1, step),
```

```
4),
        "total_iterations": iterations,
        "total_evaluations": total_evaluations,
        "final_temperature": t,
        "execution_time": time.time() - start_time,
        "valid": best_state.is_valid()
    }
```

2.5 Penjelasan Implementasi Algoritma *Genetic Algorithm*

a. Deskripsi Algoritma *Genetic Algorithm*

Genetic Algorithm (GA) merupakan metode optimasi metaheuristik berbasis populasi yang meniru proses evolusi biologis. Algoritma ini menggunakan konsep seleksi alam untuk menghasilkan solusi tata letak yang semakin baik melalui mekanisme *selection* (seleksi), *crossover* (persilangan), dan *mutation* (mutasi). Dalam konteks pemuatan kargo geladak kapal, setiap solusi potensial (individu) direpresentasikan sebagai himpunan status kendaraan mencakup status muat, posisi koordinat (x, y), dan orientasinya pada grid geladak kapal. Setiap individu memiliki nilai *fitness* (fungsi objektif) yang menggambarkan total *Shipping Fee* yang dihasilkan.

Proses pencarian dimulai dengan membangkitkan populasi awal berisi sejumlah konfigurasi acak yang valid. Setiap individu dievaluasi *fitness*-nya. Individu dengan *Shipping Fee* lebih tinggi memiliki probabilitas lebih besar untuk terpilih sebagai induk (*parent*) melalui metode *Roulette Wheel Selection*. Dua induk terpilih digabungkan melalui *crossover* untuk menghasilkan keturunan baru yang mewarisi gabungan tata letak keduanya, lalu divalidasi agar tidak ada kendaraan yang tumpang tindih. Selanjutnya, operasi *mutation* dilakukan dengan peluang tertentu untuk mengubah letak, orientasi, atau status muat kendaraan secara acak guna menjaga keragaman genetik dan mencegah konvergensi dini di *local* optima.

Evolusi ini berlangsung iteratif hingga generasi maksimal tercapai, dengan selalu mempertahankan individu terbaik (*elitism*) ke generasi berikutnya.

b. *Source Code* Algoritma *Genetic Algorithm*

```
import time
import random
from model import State, Vehicle
from tetangga import initial_random_state
```

```
def crossover(parent1, parent2, return_unrepaired=False):
    n = len(parent1.vehicles)
    if n <= 1:
        child = parent1.copy()
        return (child, None) if return_unrepaired else child

    cut_point = random.randint(1, n - 1)
    child_vehicles = []

    for i in range(n):
        if i < cut_point:

child_vehicles.append(parent1.vehicles[i].copy())
        else:

child_vehicles.append(parent2.vehicles[i].copy())

    obj_type = getattr(parent1, "objective_type", "fee")
    child_state = State(parent1.ship, child_vehicles,
objective_type=obj_type)
    unrepaired = child_state.copy() if return_unrepaired and
not child_state.is_valid() else None
    child_state.repair_to_valid()

    return (child_state, unrepaired) if return_unrepaired
else child_state

def mutate(state, mutation_rate=0.15):
    if random.random() > mutation_rate or not
state.vehicles:
        return state

    idx = random.randint(0, len(state.vehicles) - 1)
    target = state.vehicles[idx]

    if target.is_loaded:
        action = random.choice(["move", "rotate", "unload"])
        if action == "move":
            target.x = max(0, min(state.ship.w - 1, target.x
+ random.choice([-1, 1])))
            target.y = max(0, min(state.ship.l - 1, target.y
+ random.choice([-1, 1])))
        elif action == "rotate":
            target.orientation = "Vertical" if
target.orientation == "Horizontal" else "Horizontal"
        elif action == "unload":
            target.is_loaded = False
    else:
        target.is_loaded = True
        target.orientation = random.choice(["Horizontal",
```

```
"Vertical"])
        w_eff, l_eff = target.get_dimensions()
        max_x = max(0, state.ship.w - w_eff)
        max_y = max(0, state.ship.l - l_eff)
        target.x = random.randint(0, max_x)
        target.y = random.randint(0, max_y)

    state.repair_to_valid()
    return state

def select_parent_roulette(population, fitness_scores):
    total_fitness = sum(fitness_scores)
    if total_fitness <= 0:
        return random.choice(population)

    pick = random.uniform(0, total_fitness)
    current = 0
    for individual, fitness in zip(population,
fitness_scores):
        current += fitness
        if current >= pick:
            return individual
    return population[-1]

def genetic_algorithm(ship, vehicles_data,
population_size=20, max_generations=50, mutation_rate=0.15,
initial_population=None, objective_type="fee"):
    start_time = time.time()

    if initial_population is not None and
len(initial_population) >= population_size:
        population = [s.copy() for s in
initial_population[:population_size]]
    else:
        population = [initial_random_state(ship,
vehicles_data, objective_type=objective_type) for _ in
range(population_size)]

    fitness_scores = [ind.calculate_objective() for ind in
population]
    total_evaluations = len(population)

    best_idx = max(range(len(fitness_scores)), key=lambda i:
fitness_scores[i])
    best_overall_state = population[best_idx].copy()
    best_overall_fitness = fitness_scores[best_idx]
    first_initial_state = best_overall_state.copy()

    history_best_objective = [best_overall_fitness]
    history_avg_objective = [sum(fitness_scores) /
```

```
len(fitness_scores)]

    for gen in range(1, max_generations + 1):
        new_population = [best_overall_state.copy()]

        while len(new_population) < population_size:
            parent1 = select_parent_roulette(population,
fitness_scores)
            parent2 = select_parent_roulette(population,
fitness_scores)

            child = crossover(parent1, parent2)
            child = mutate(child,
mutation_rate=mutation_rate)
            new_population.append(child)

        population = new_population
        fitness_scores = [ind.calculate_objective() for ind
in population]
        total_evaluations += (len(population) - 1)

        gen_best_idx = max(range(len(fitness_scores)),
key=lambda i: fitness_scores[i])
        gen_best_fitness = fitness_scores[gen_best_idx]
        gen_avg_fitness = sum(fitness_scores) /
len(fitness_scores)

        if gen_best_fitness > best_overall_fitness:
            best_overall_fitness = gen_best_fitness
            best_overall_state =
population[gen_best_idx].copy()

        history_best_objective.append(gen_best_fitness)
        history_avg_objective.append(gen_avg_fitness)

    return {
        "initial_state": first_initial_state,
        "final_state": best_overall_state,
        "best_objective": best_overall_fitness,
        "history_objective": history_best_objective,
        "history_best_objective": history_best_objective,
        "history_avg_objective": history_avg_objective,
        "total_generations": max_generations,
        "total_iterations": max_generations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": best_overall_state.is_valid()
    }
```

c. Penjelasan Source Code Algoritma *Genetic Algorithm*

- **Fungsi Algoritma *Genetic Algorithm***

  Fungsi `genetic_algorithm()` mencari solusi tata letak optimal. Parameter yang diterima meliputi:

  - ship dan vehicles_data: Data dimensi geladak kapal dan kendaraan.
  - population_size: Jumlah individu per generasi (default: 20).
  - max_generations: Batas generasi evolusi (default: 50).
  - mutation_rate: Peluang terjadinya mutasi pada keturunan (default: 0.15).
  - objective_type: Jenis perhitungan *fitness* (default: "fee").

  Hasil dari fungsi ini mengembalikan *dictionary* berisi:

  - initial_state dan final_state: Konfigurasi dek awal dan konfigurasi dek terbaik yang ditemukan.
  - best_objective: Skor *fitness* tertinggi dari *final_state*.
  - history_best_objective: Riwayat nilai *fitness* tertinggi di setiap generasi untuk grafik konvergensi.
  - history_avg_objective: Nilai rata-rata *fitness* populasi di setiap generasi untuk memantau peningkatan kualitas populasi.
  - total_evaluations: Total pemanggilan fungsi objektif selama pencarian.
  - execution_time: Waktu komputasi yang dihabiskan algoritma.

- **Deklarasi Variabel Algoritma *Genetic Algorithm***

  Variabel-variabel utama yang beroperasi di dalam fungsi genetic_algorithm() meliputi:

  - population: Daftar (*list*) yang berisi sekumpulan individu (state konfigurasi tata letak geladak) dalam satu generasi pencarian.
  - fitness_scores: Daftar (*list*) yang menyimpan nilai evaluasi objektif (*fitness*) dari masing-masing individu di dalam population yang bersesuaian.
  - best_overall_state: Menyimpan individu atau konfigurasi dek terbaik yang pernah ditemukan sepanjang keseluruhan proses evolusi (berfungsi menjaga sifat *elitism*).
  - best_overall_fitness: Menyimpan skor objektif tertinggi yang dimiliki oleh best_overall_state.

- new_population: Daftar sementara (*list*) untuk menampung individu-individu generasi baru (keturunan) hasil seleksi, *crossover*, dan mutasi sebelum menggantikan populasi lama di iterasi berikutnya.
- history_best_objective dan history_avg_objective: Daftar (*list*) yang mencatat rekam jejak nilai *fitness* terbaik dan rata-rata *fitness* populasi pada setiap generasi untuk keperluan analisis atau visualisasi grafik.
- total_evaluations: Penghitung jumlah total evaluasi fungsi objektif (pembangkitan/penilaian *state*) yang dilakukan sejak algoritma dijalankan.
- parent1 dan parent2: Menyimpan dua individu induk terpilih dari populasi berdasarkan metode *Roulette Wheel Selection*.
- child: Menyimpan individu keturunan baru yang merupakan hasil dari operasi *crossover* antara dua *parent* dan modifikasi dari fungsi *mutate*.

- **Inisialisasi Nilai Awal Algoritma *Genetic Algorithm***

  Algoritma membangkitkan populasi awal berisi konfigurasi acak menggunakan fungsi `initial_random_state()`. Evaluasi *fitness* dilakukan untuk setiap individu pada populasi awal, dan individu dengan skor *fitness* tertinggi langsung disimpan ke dalam variabel `best_overall_state` sebagai solusi elit (*elite*).

```
   start_time = time.time()

    if initial_population is not None and
len(initial_population) >= population_size:
        population = [s.copy() for s in
initial_population[:population_size]]
    else:
        population = [initial_random_state(ship,
vehicles_data, objective_type=objective_type) for _ in
range(population_size)]

    fitness_scores = [ind.calculate_objective() for ind in
population]
    total_evaluations = len(population)

    best_idx = max(range(len(fitness_scores)), key=lambda i:
fitness_scores[i])
    best_overall_state = population[best_idx].copy()
    best_overall_fitness = fitness_scores[best_idx]
    first_initial_state = best_overall_state.copy()
```

```
    history_best_objective = [best_overall_fitness]
    history_avg_objective = [sum(fitness_scores) /
len(fitness_scores)]
```

- **Proses Perulangan (*Loop*) Pencarian Algoritma *Genetic Algorithm***

  Evolusi berlangsung sebanyak `max_generations`. Pada setiap awal pembentukan generasi baru, algoritma menerapkan prinsip *elitism* dengan menyalin `best_overall_state` ke dalam `new_population` untuk memastikan solusi terbaik tidak hilang akibat mutasi yang merusak. Sisa populasi baru dibentuk melalui seleksi dan rekombinasi hingga populasinya kembali mencapai batas `population_size`.

```
   for gen in range(1, max_generations + 1):
        new_population = [best_overall_state.copy()]

        while len(new_population) < population_size:
            parent1 = select_parent_roulette(population,
fitness_scores)
            parent2 = select_parent_roulette(population,
fitness_scores)

            child = crossover(parent1, parent2)
            child = mutate(child,
mutation_rate=mutation_rate)
            new_population.append(child)
```

- **Membangkitkan Tetangga dan Mencari Tetangga Terbaik**

  Proses pembangkitan "tetangga" dalam konteks GA dilakukan melalui seleksi dan kombinasi dua individu (*parent*) untuk menghasilkan individu baru (*offspring*).

  1. **Seleksi:** Menggunakan fungsi select_parent_roulette(). Probabilitas terpilihnya sebuah state berbanding lurus dengan nilai objektifnya terhadap total *fitness* populasi.
  2. **Crossover:** Menggunakan titik potong acak (*1-point crossover*) pada daftar kendaraan. Kendaraan sebelum titik potong diwarisi dari parent1, sisanya dari parent2. Jika penggabungan ini menghasilkan tumpang tindih (*overlap*), fungsi repair_to_valid() otomatis membersihkan kendaraan yang bertabrakan.

3. **Mutasi:** Fungsi mutate() memilih satu kendaraan secara acak. Jika kendaraan sedang dimuat, sistem mungkin menggeser koordinat (x/y), memutar orientasi, atau menurunkannya (unload). Jika kendaraan di luar kapal, sistem akan mencoba memuatnya (load) di posisi acak. Setelah mutasi, validitas dek kembali diperbaiki (*repair*).

Langkah-langkah ini memastikan algoritma mampu mengeksplorasi ruang solusi secara luas sambil mempertahankan individu yang memiliki *fitness* unggul.

- **Memperbarui State Saat Ini Jika Ada Peningkatan Value**

  Di akhir setiap generasi, algoritma mencari individu dengan skor *fitness* (nilai objektif) tertinggi pada populasi yang baru terbentuk. Jika *fitness* tertinggi dari generasi tersebut melampaui rekor nilai *fitness* terbaik secara keseluruhan (`best_overall_fitness`), maka individu tersebut disimpan sebagai solusi elit (terbaik) yang baru.

```
if gen_best_fitness > best_overall_fitness:
            best_overall_fitness = gen_best_fitness
            best_overall_state =
population[gen_best_idx].copy()
```

  Pembaruan ini menjamin bahwa algoritma selalu mempertahankan solusi penataan dek terbaik yang pernah ditemukan sepanjang proses evolusi (memaksimalkan *Shipping Fee*).

- **Menyimpan Hasil Akhir Algoritma *Genetic Algorithm***

  Pada setiap iterasi generasi, algoritma merekam nilai *fitness* terbaik (`history_best_objective`) dan rata-rata *fitness* populasi (`history_avg_objective`) untuk keperluan analisis konvergensi. Setelah batas iterasi maksimal (`max_generations`) tercapai, waktu eksekusi total dikalkulasi menggunakan `time.time()`, dan hasil akhir dikembalikan dalam bentuk *dictionary*:

```
   return {
```

```
        "initial_state": first_initial_state,
        "final_state": best_overall_state,
        "best_objective": best_overall_fitness,
        "history_objective": history_best_objective,
        "history_best_objective": history_best_objective,
        "history_avg_objective": history_avg_objective,
        "total_generations": max_generations,
        "total_iterations": max_generations,
        "total_evaluations": total_evaluations,
        "execution_time": time.time() - start_time,
        "valid": best_overall_state.is_valid()
    }
```

Hasil akhir ini mencakup representasi objek konfigurasi geladak terbaik, catatan perkembangan nilai objektif, metrik evaluasi fungsi, status validitas dek, dan durasi komputasi. Dengan pendekatan berbasis populasi ini, *Genetic Algorithm* mampu menghasilkan solusi penempatan kendaraan yang mendekati optimal secara masif dan terstruktur pada persoalan kompleks seperti *2D Vehicle Packing Problem*.

d. Kesimpulan *Genetic Algorithm*

Secara keseluruhan, *Genetic Algorithm* (GA) merupakan pendekatan metaheuristik berbasis evolusi yang terbukti dinamis dalam menangani optimasi tata letak spasial (*2D Vehicle Packing Problem*). Dengan mempresentasikan setiap penempatan kendaraan di atas geladak sebagai "kromosom", mekanisme hibridisasi sifat induk (*crossover*) memungkinkan algoritma menggabungkan bagian tata letak yang baik dari dua solusi yang berbeda. Selain itu, mekanisme seleksi *Roulette Wheel* memastikan pencarian difokuskan pada area ruang solusi yang paling menguntungkan, sementara mutasi pergerakan/rotasi kendaraan memastikan algoritma tidak mengalami stagnasi lokal.

Berbeda dengan algoritma turunan *Hill-Climbing* yang menavigasi ruang solusi dari satu titik saja (lintasan tunggal), GA memproses lusinan kombinasi konfigurasi dek secara serempak di setiap generasi (pencarian paralel). Meskipun menuntut waktu komputasi yang relatif lebih komprehensif, pendekatan *elitism* pada algoritma ini menjamin bahwa konfigurasi geladak dengan tarif (*Shipping Fee*) maksimum yang

pernah ditemukan tidak akan hilang, menjadikannya algoritma yang sangat tangguh untuk permasalahan tata letak kapasitas ruang berskala besar.

2.6 Hasil Eksperimen dan Analisis

2.6.1 Dataset

Pengujian seluruh algoritma pencarian lokal untuk optimasi tata letak geladak kapal kargo menggunakan dataset kendaraan default yang dibaca dari berkas vehicles.json. Dataset ini berisi 8 kendaraan (V1 hingga V8) dan digunakan sebagai data awal agar setiap algoritma dapat diuji pada kondisi yang sama. Data kendaraan ditentukan secara *hardcoded*, tetapi jumlah kendaraan dapat ditambah melalui penyesuaian pada kode program.

Setiap kendaraan dalam dataset ini tidak hanya memiliki informasi ID, melainkan dilengkapi parameter dimensi lebar (w) dan panjang (l) untuk validasi spasial, tarif pengiriman (fee) sebagai dasar perhitungan fungsi objektif, bobot kendaraan (*weight*) untuk pengecekan kapasitas maksimum kapal, serta prioritas waktu (eta). Berikut adalah struktur dataset yang digunakan untuk pengujian algoritma:

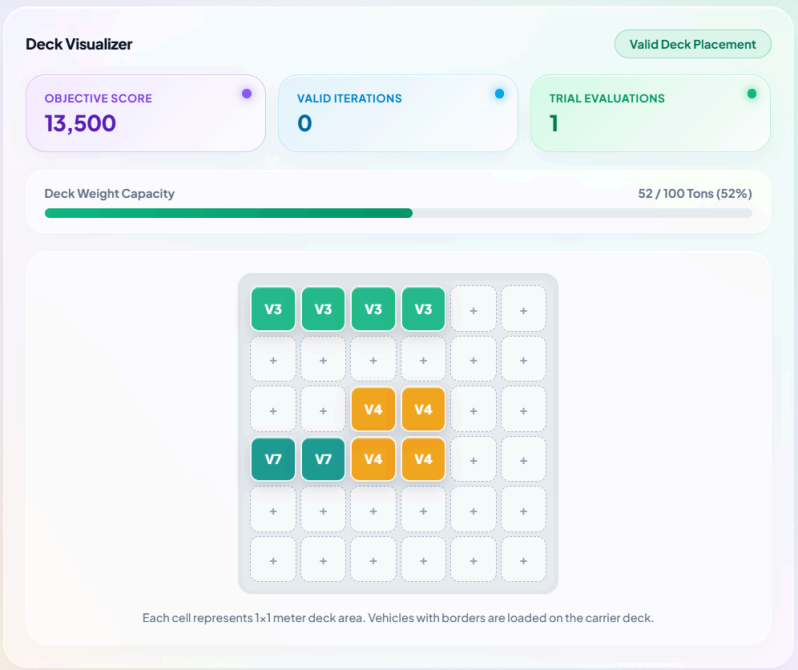
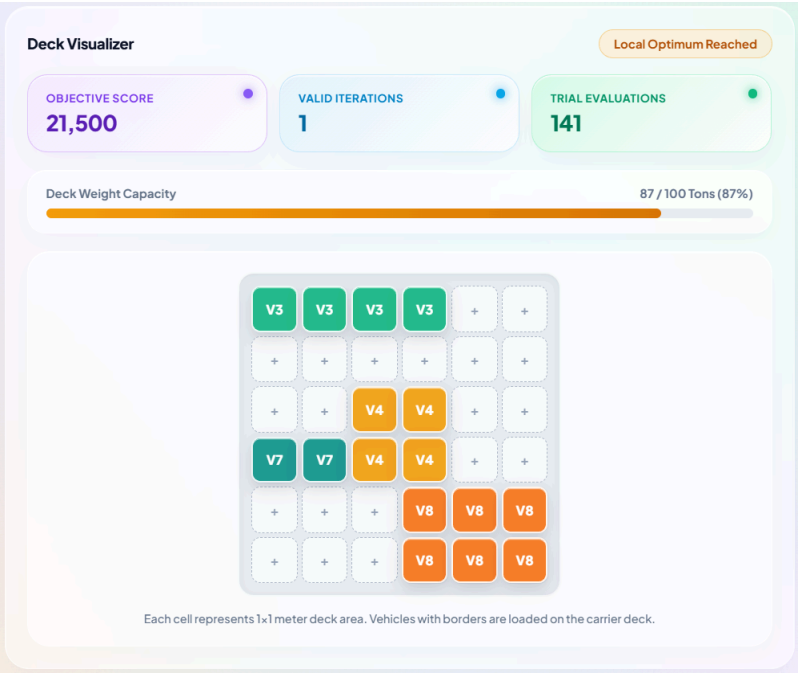
```
[
  { "v_id": "V1", "w": 2, "l": 2, "fee": 5000, "weight": 20, "eta": 1 },
  { "v_id": "V2", "w": 2, "l": 3, "fee": 7000, "weight": 30, "eta": 1 },
  { "v_id": "V3", "w": 1, "l": 4, "fee": 4000, "weight": 15, "eta": 2 },
  { "v_id": "V4", "w": 2, "l": 2, "fee": 6000, "weight": 25, "eta": 3 },
  { "v_id": "V5", "w": 3, "l": 1, "fee": 5500, "weight": 18, "eta": 2 },
  { "v_id": "V6", "w": 1, "l": 2, "fee": 3000, "weight": 10, "eta": 4 },
  { "v_id": "V7", "w": 2, "l": 1, "fee": 3500, "weight": 12, "eta": 5 },
  { "v_id": "V8", "w": 3, "l": 2, "fee": 8000, "weight": 35, "eta": 3 }
]
```

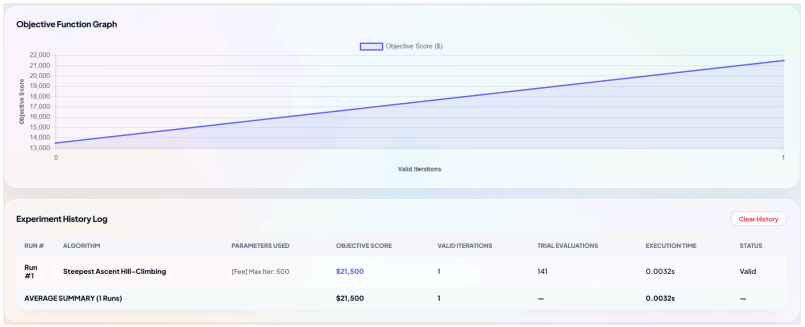
2.6.2 Hasil *Hill-Climbing*

a. Hasil *Steepest-Ascent Hill-Climbing*

Tabel 1. Hasil *Steepest-Ascent Hill-Climbing* 1

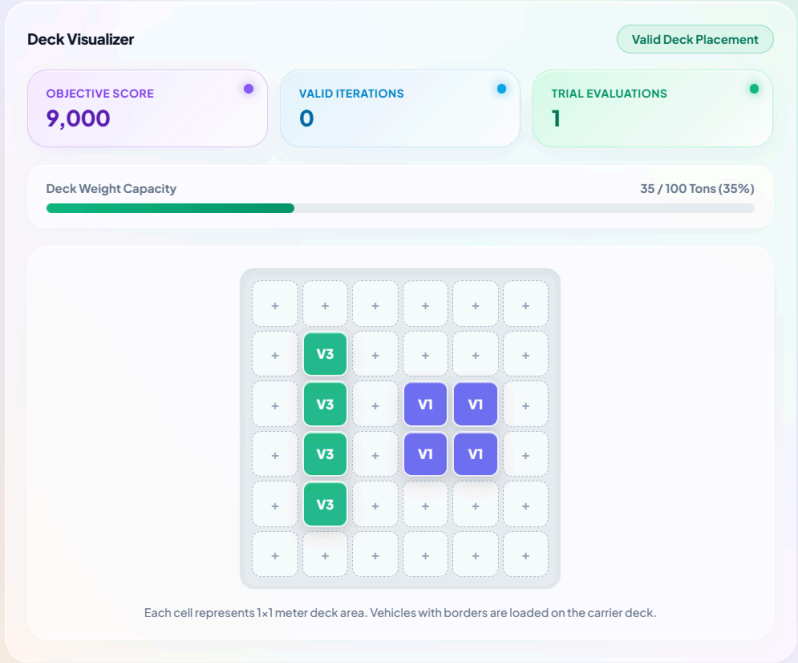
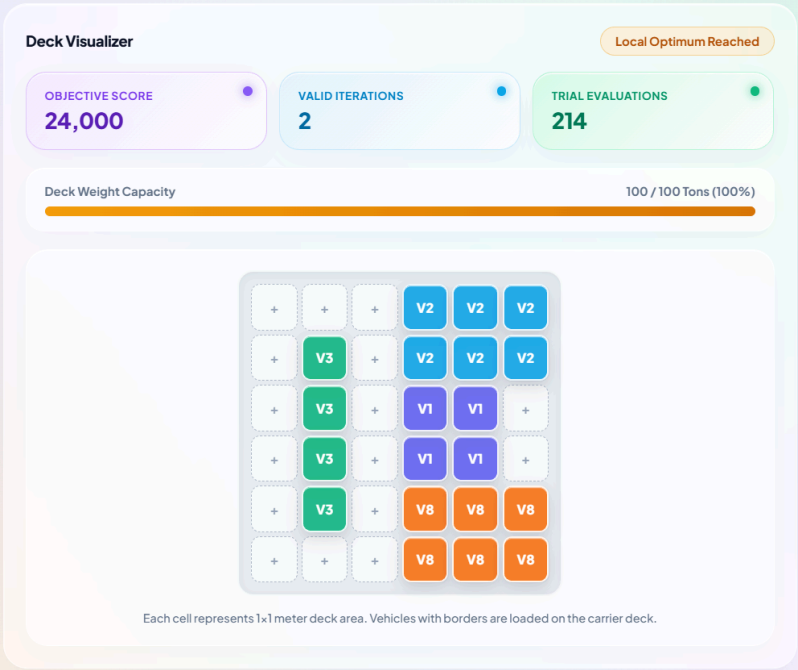
Iterasi 1

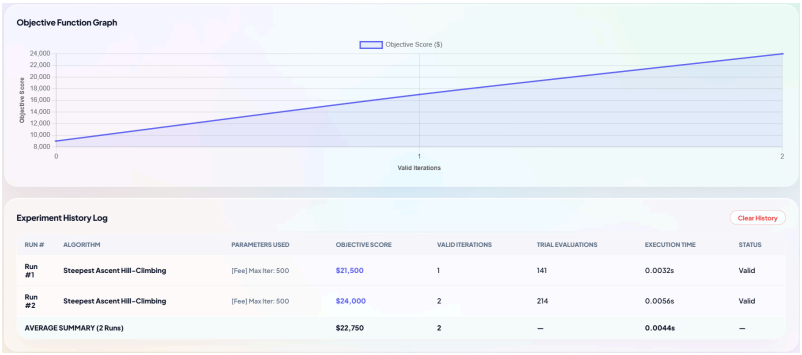
Initial State	
Best State	

Plot Objective Function over All Successors	
Objective Score (Fee)	$21,500
Execution Time (s)	0.0032s
Total Kontainer yang Digunakan	4 (V3, V4, V7, V8)
Valid Iterations	1
Trial Evaluations	141

Tabel 2. Hasil *Steepest-Ascent Hill-Climbing* 2

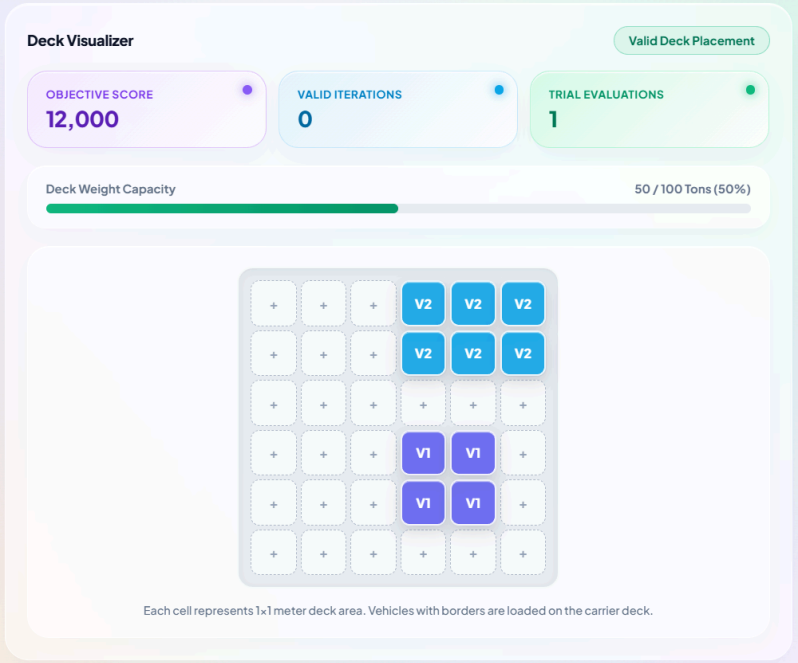
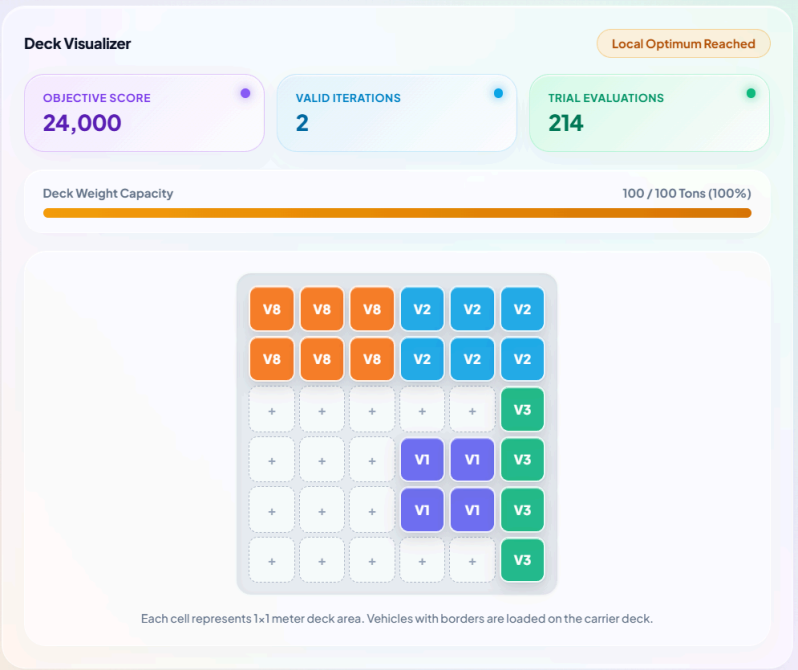
Iterasi 2

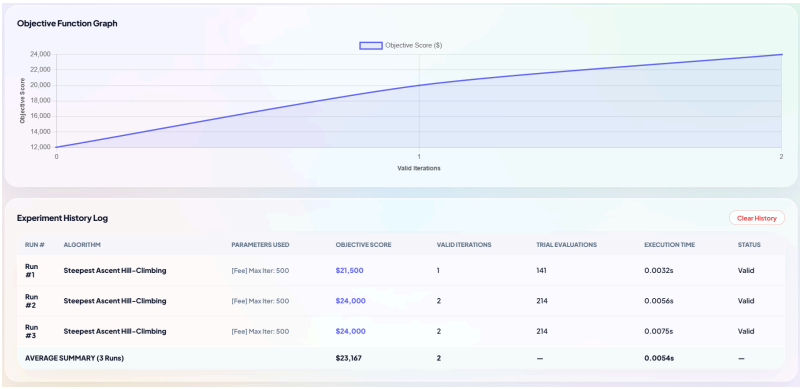
Initial State	
Best State	

Plot Objective Function over All Successors	
Objective Score (Fee)	$24,000
Execution Time (s)	0.0056s
Total Kontainer yang Digunakan	4 (V1, V2, V3, V8)
Valid Iterations	2
Trial Evaluations	214

Tabel 3. Hasil *Steepest-Ascent Hill-Climbing* 3

Iterasi 3

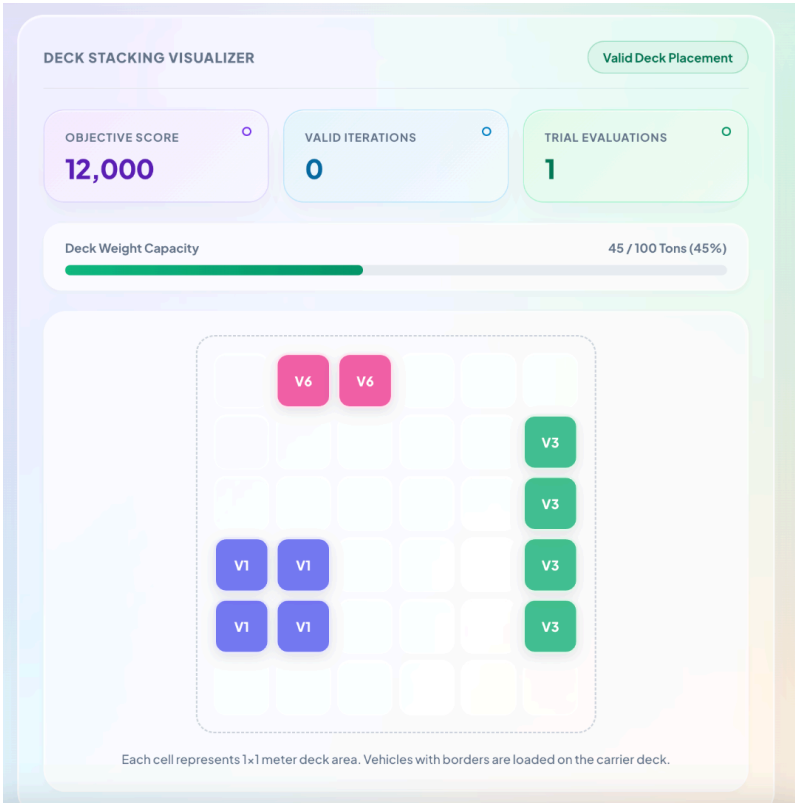
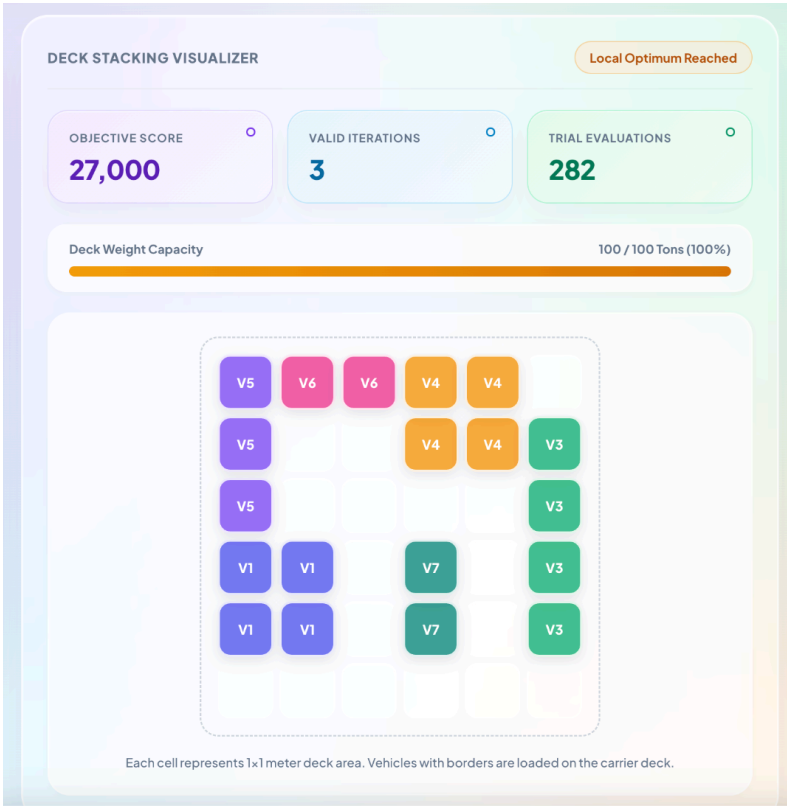
Initial State	
Best State	

Plot Objective Function over All Successors	
Objective Score (Fee)	$24,000
Execution Time (s)	0.0075s
Total Kontainer yang Digunakan	4 (V1, V2, V3, V8)
Valid Iterations	2
Trial Evaluations	214

b. Hasil *Stochastic Hill-Climbing*

Tabel 4. Hasil *Stochastic Hill-Climbing* 1

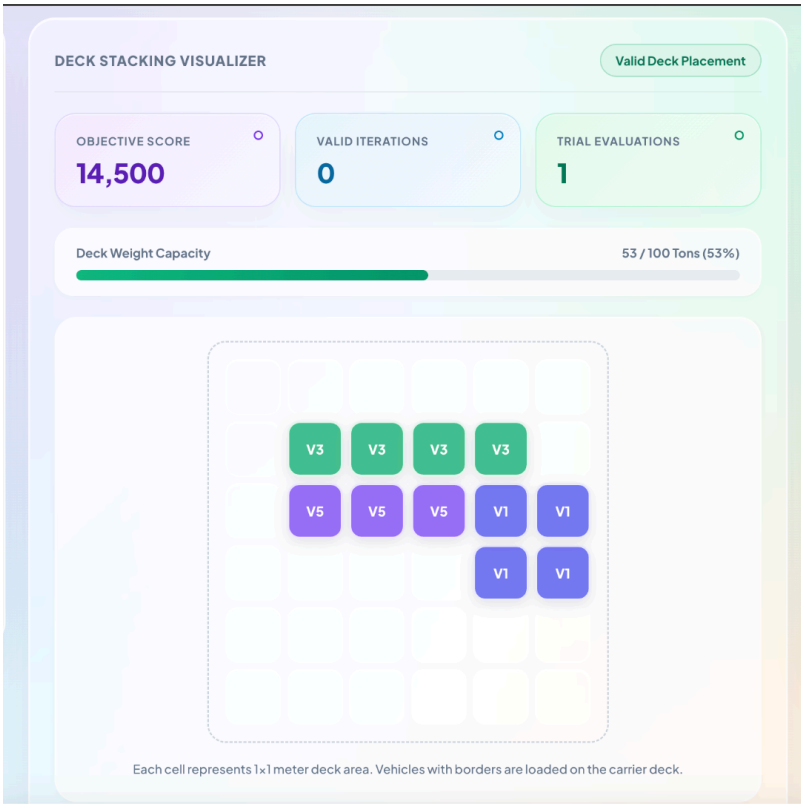
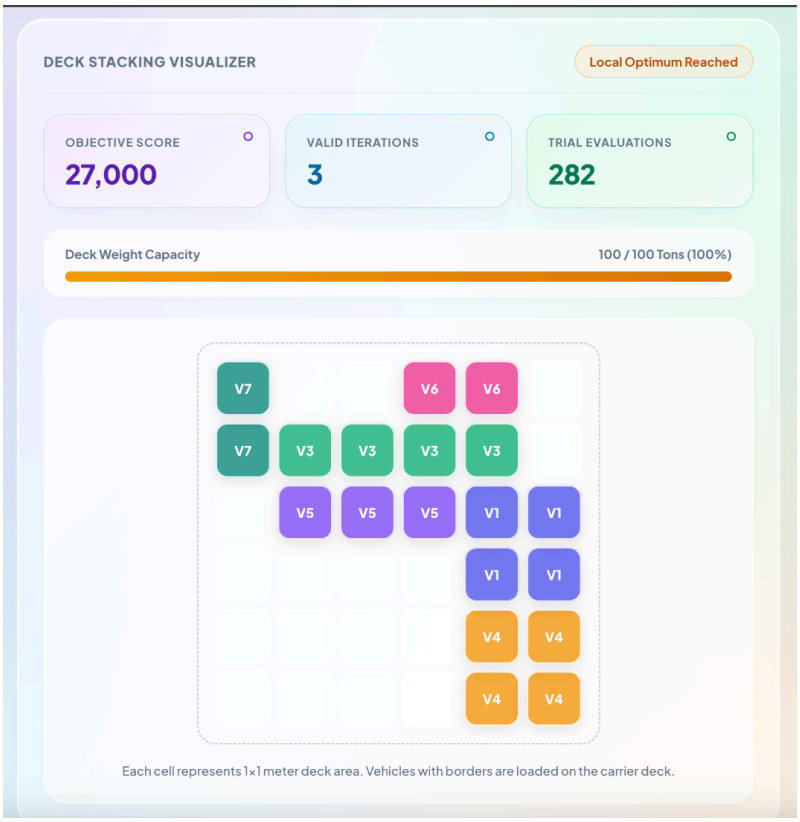
Iterasi 1

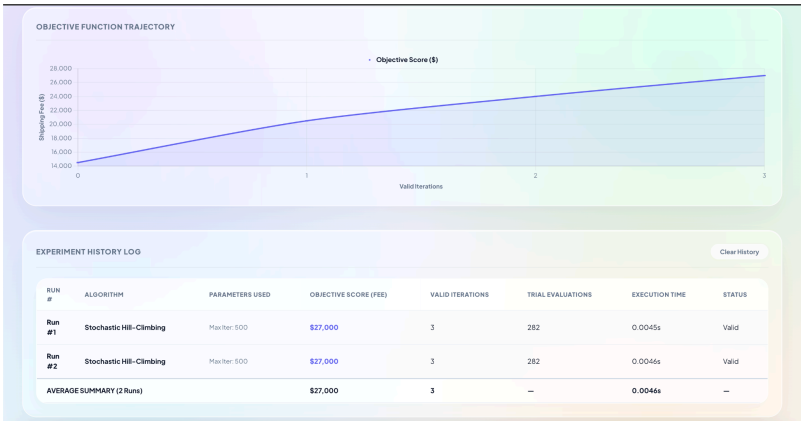
Initial State	
Best State	

Plot Objective Function over All Successors	
Objective Score (Fee)	$27,000
Execution Time (s)	0.0045s
Total Kontainer yang Digunakan	6 (V1, V3, V4, V5, V6, V7)
Valid Iterations	3
Trial Evaluations	282

Tabel 5. Hasil *Stochastic Hill-Climbing* 2

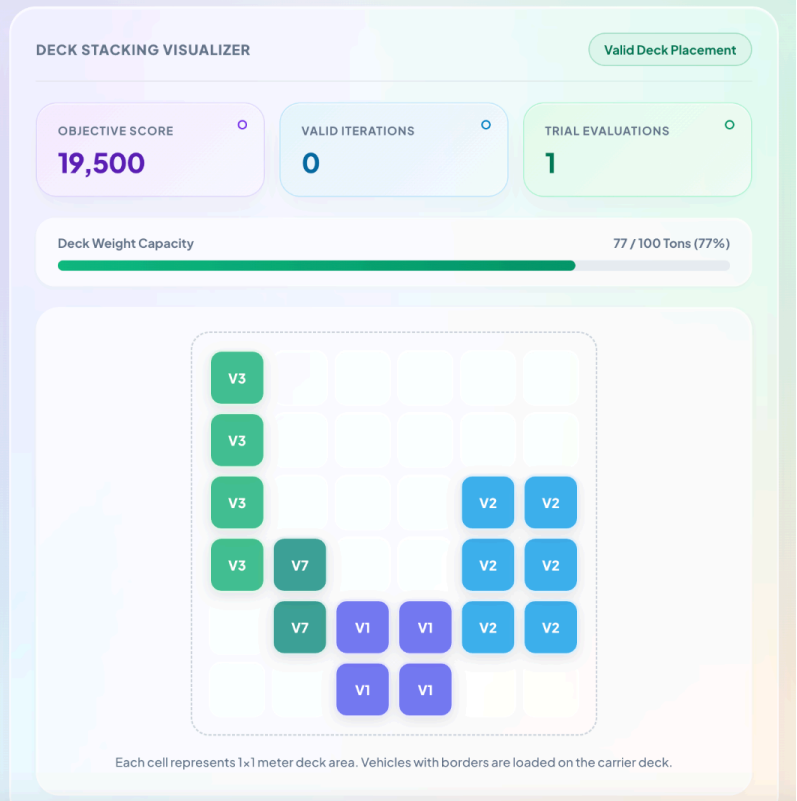
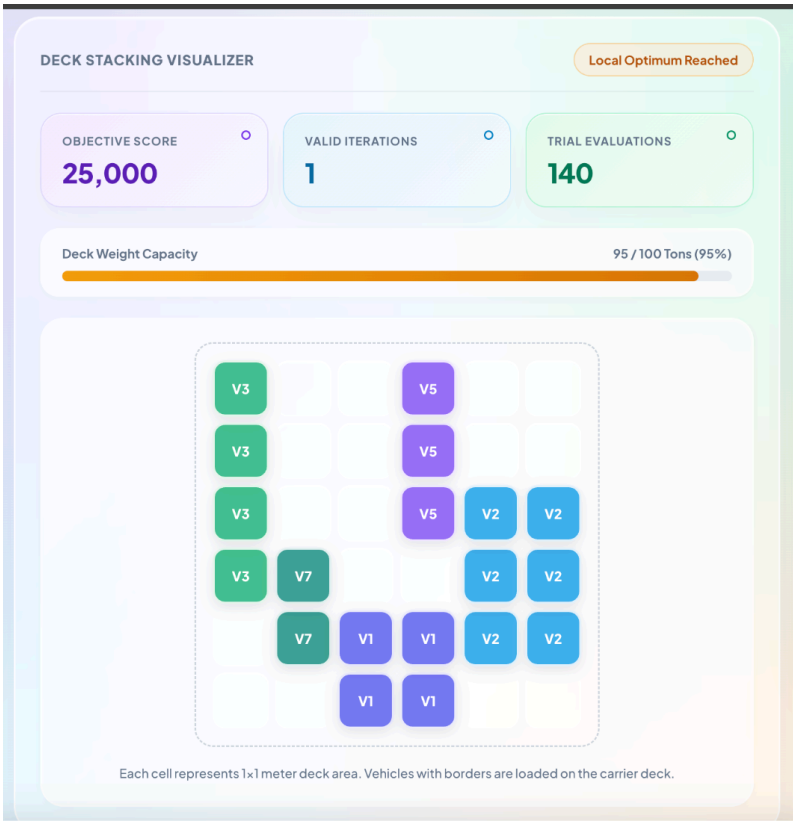
Iterasi 2

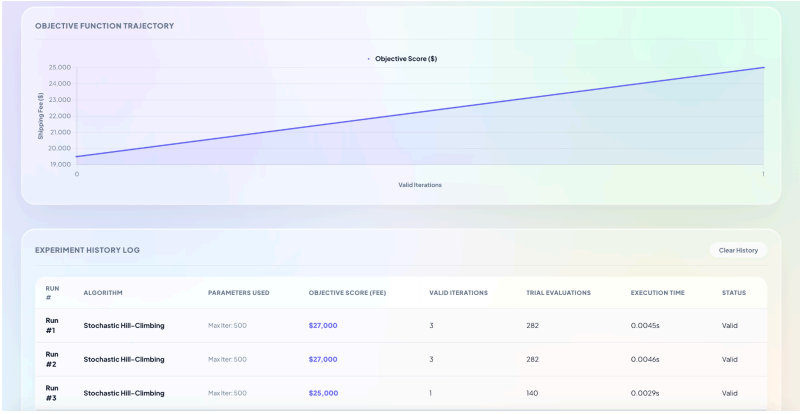
Initial State	
Best State	

Plot Objective Function over All Successors	
Objective Score (Fee)	$27,000
Execution Time (s)	0.0046s
Total Kontainer yang Digunakan	6 (V1, V3, V4, V5, V6, V7)
Valid Iterations	3
Trial Evaluations	282

Tabel 6. Hasil *Stochastic Hill-Climbing* 3

Iterasi 3

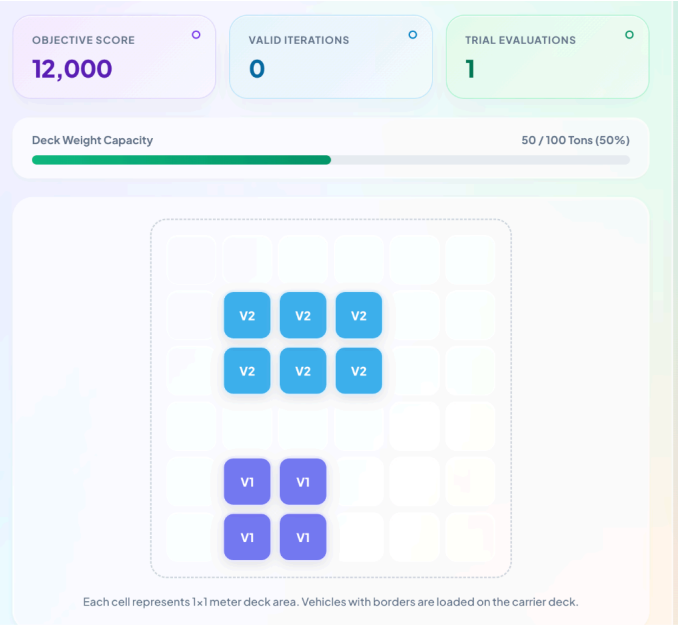
Initial State	
Best State	

Plot Objective Function over All Successors	
Objective Score (Fee)	$25,000
Execution Time (s)	0.0029s
Total Kontainer yang Digunakan	5 (V1, V2, V3, V5, V7)
Valid Iterations	1
Trial Evaluations	140

c. Hasil *Random Restart Hill-Climbing*

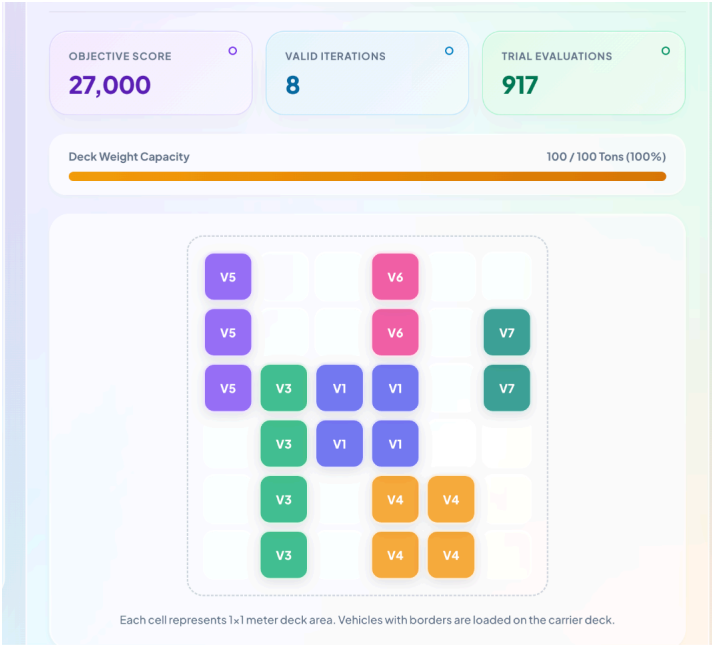
Tabel 7. Hasil *Random Restart Hill-Climbing* 1

Iterasi 1
Initial State

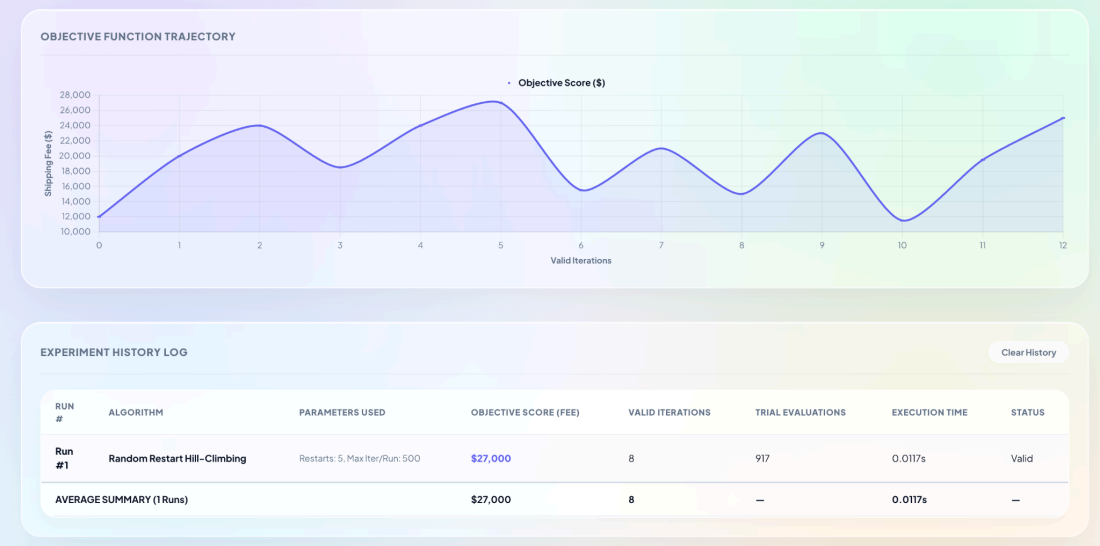


Total Kontainer yang Digunakan	Max Restart	Initial Objective Score
2 (V1, V2)	5	$12,000

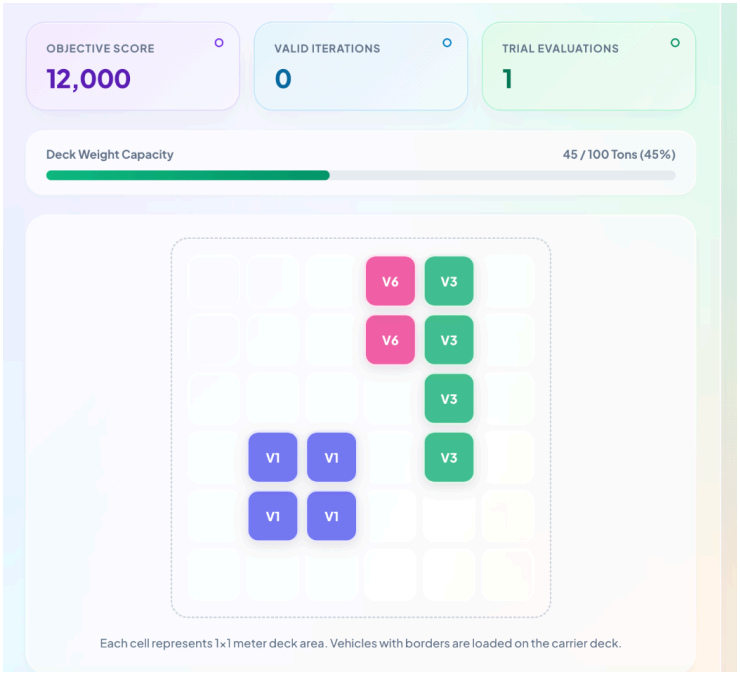
Best State



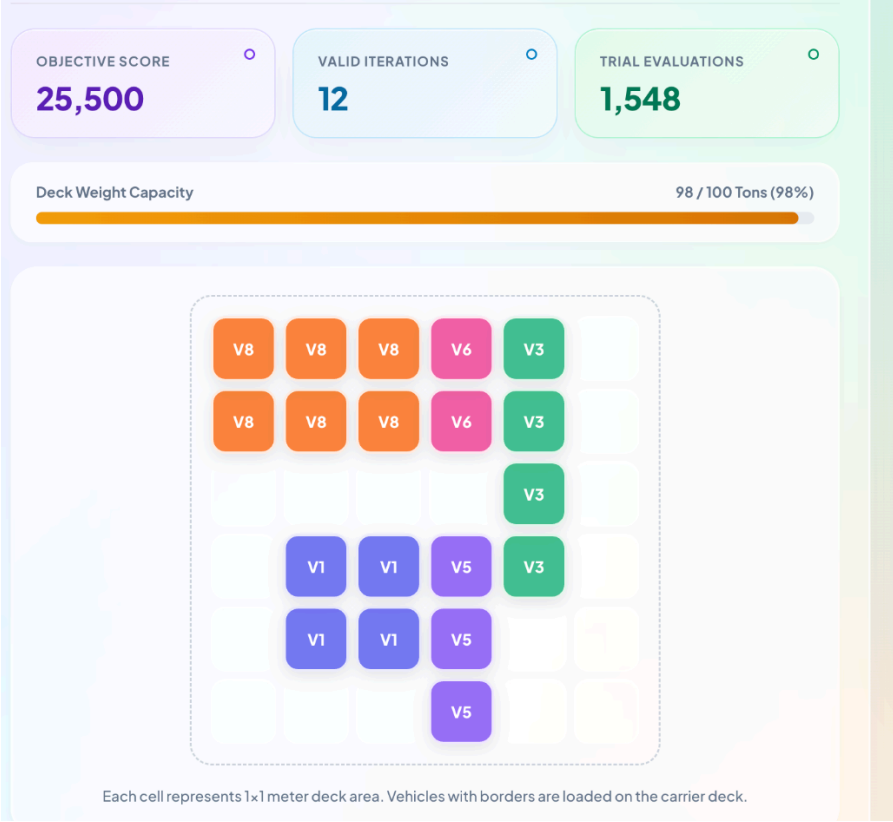
Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Valid Iterations

6 (V1, V3, V4, V5, V6, V7)	$27,000	0.0117s	8
Plot Best Objective Function over Iterations			
			

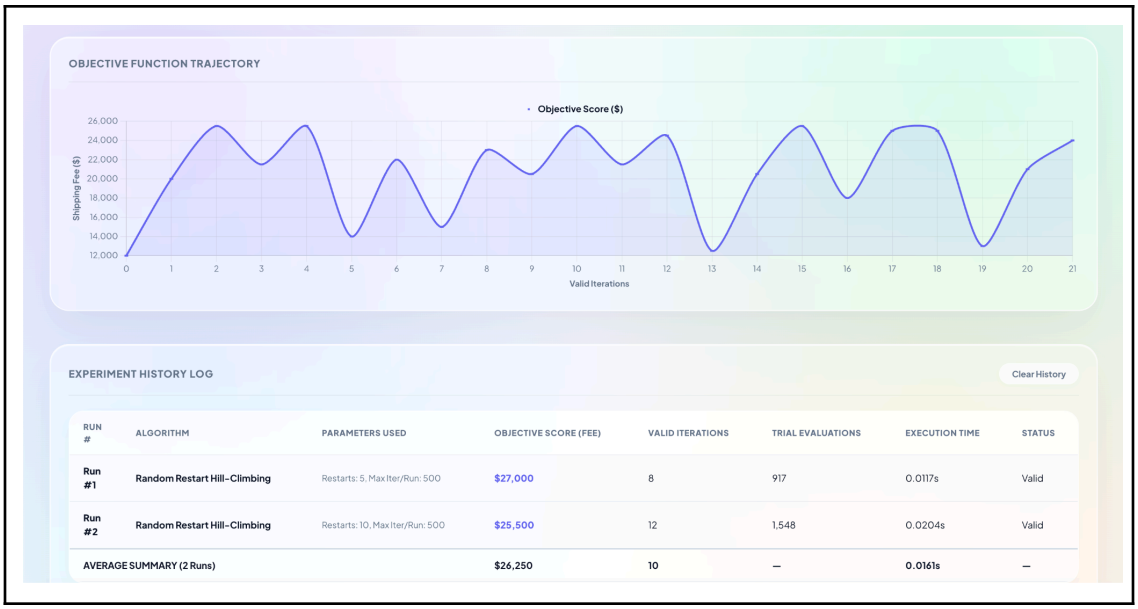
Tabel 8. Hasil *Random Restart Hill-Climbing* 2

Iterasi 2
Initial State


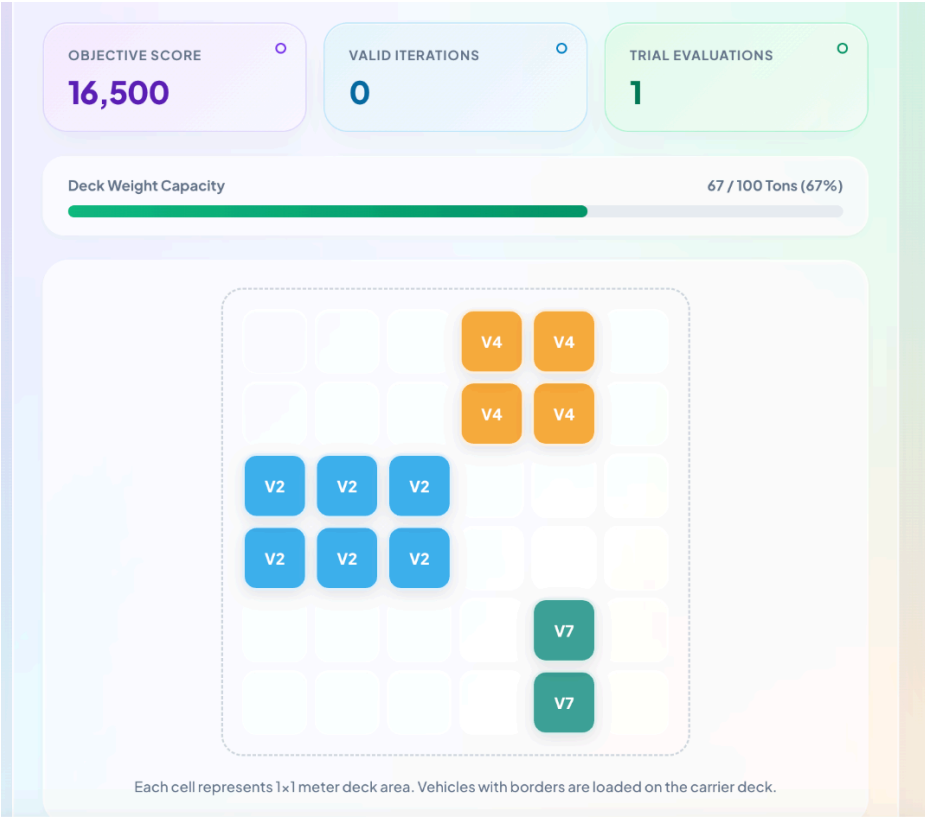
Total Kontainer yang Digunakan	Max Restart	Initial Objective Score
3 (V1, V3, V6)	10	$12,000
Best State		



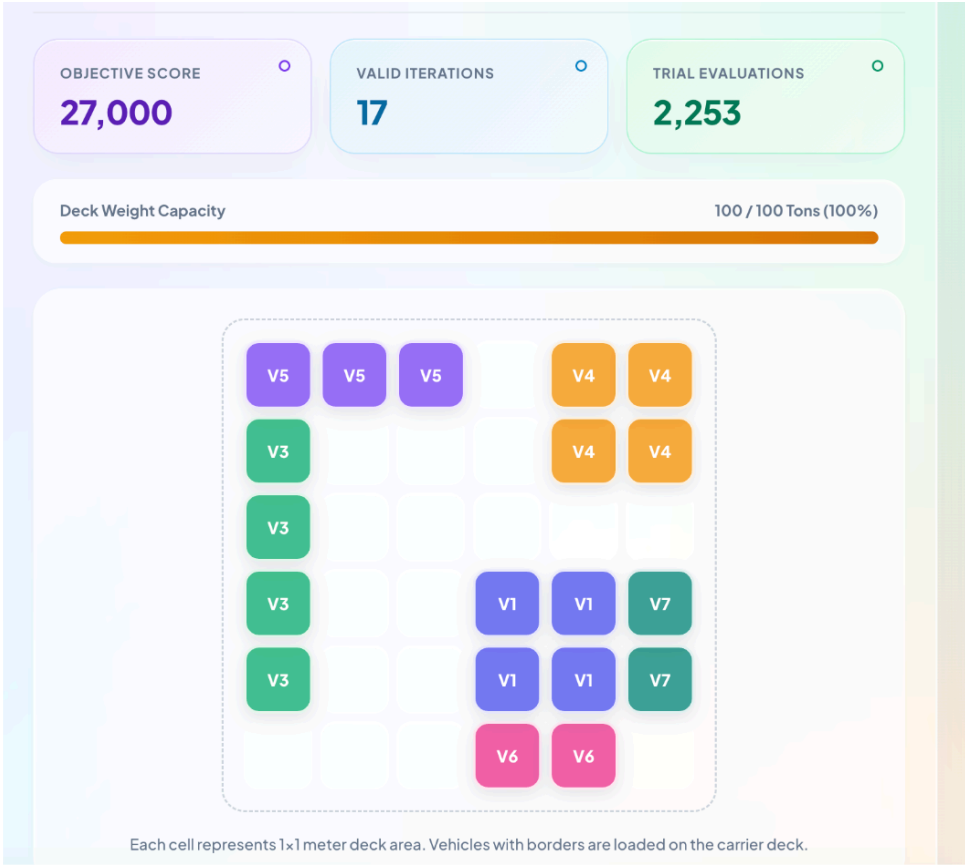
Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Valid Iterations
5 (V1, V3, V5, V6, V8)	$25,500	0.0204s	12
Plot Best Objective Function over Iterations			



Tabel 9. Hasil *Random Restart Hill-Climbing* 3

Iterasi 3
Initial State


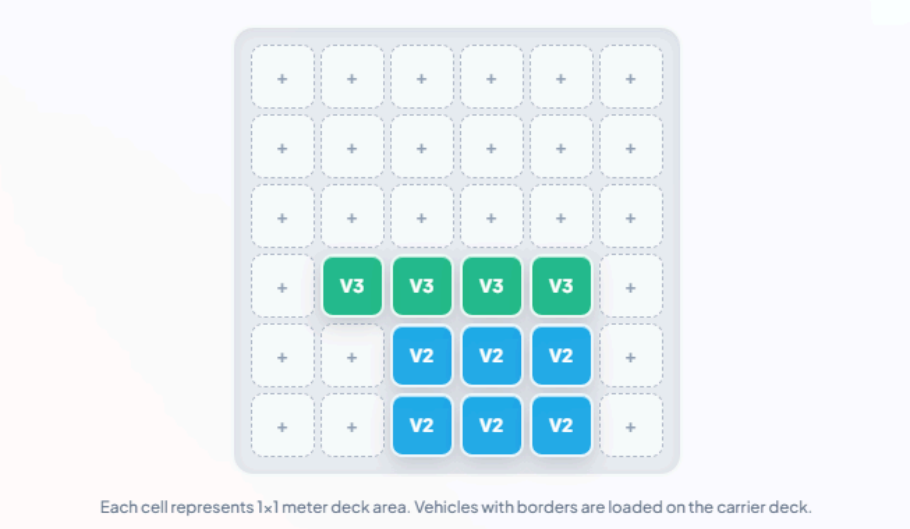
Total Kontainer yang Digunakan	Max Restart	Initial Objective Score
3 (V2, V4, V7)	15	$16,500
Best State		

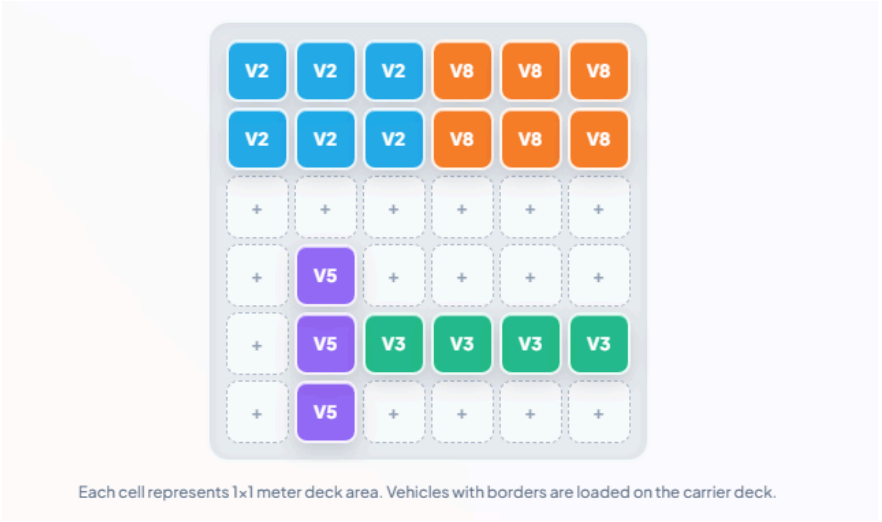


Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Valid Iterations
6 (V1, V3, V4, V5, V6, V7)	$27,000	0.0212s	17
Plot Best Objective Function over Iterations			

d. Hasil *Sideways Move Hill-Climbing*

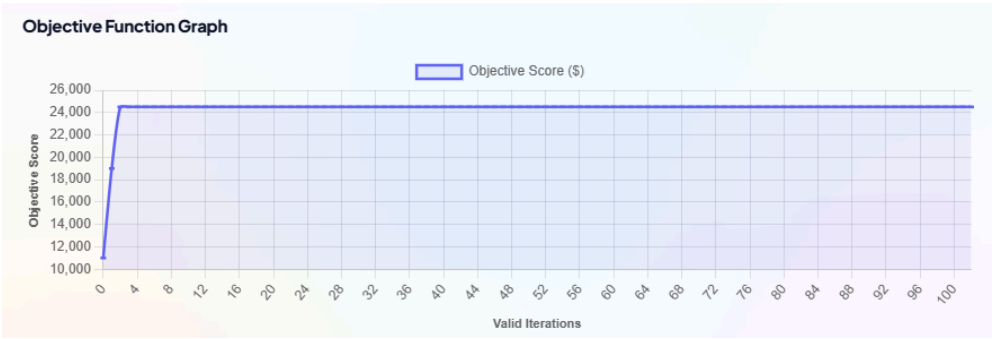
Tabel 10. Hasil *Sideways Move Hill-Climbing* 1

Iterasi 1		
Initial State		
		
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Objective Score**
2 (V2, V3)	500	$11,000
Best State		



Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Valid Iterations
4 (V2, V3, V5, V8)	$24,500	0.2716s	102

Plot Best Objective Function over Iterations

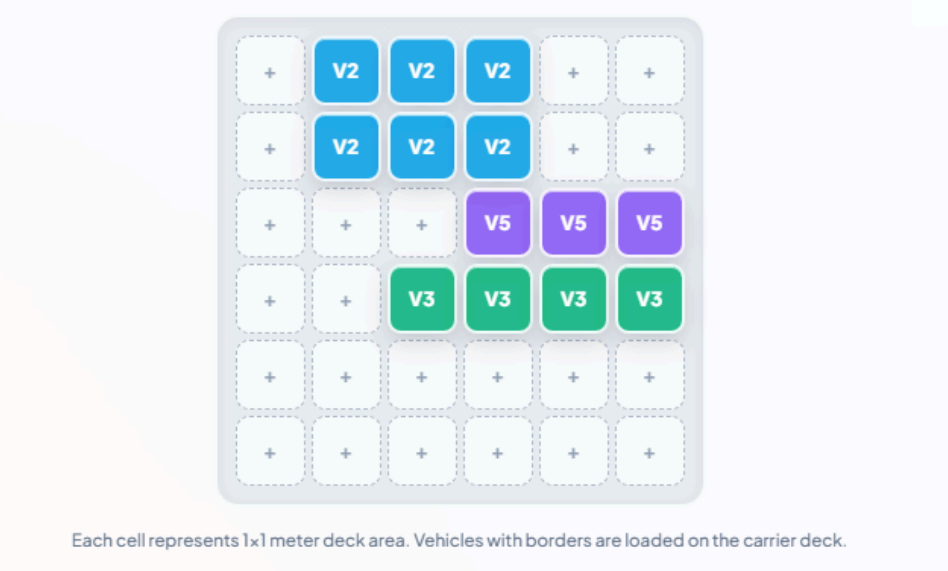


Experiment History Log Clear History

RUN #	ALGORITHM	PARAMETERS USED	OBJECTIVE SCORE	VALID ITERATIONS	TRIAL EVALUATIONS	EXECUTION TIME	STATUS
Run #1	Hill-Climbing with Sideways Move	[Fee] Max Sideways: 100, Max Iter: 500	$24,500	102	7,214	0.2716s	Valid
AVERAGE SUMMARY (1 Runs)			$24,500	102	—	0.2716s	—

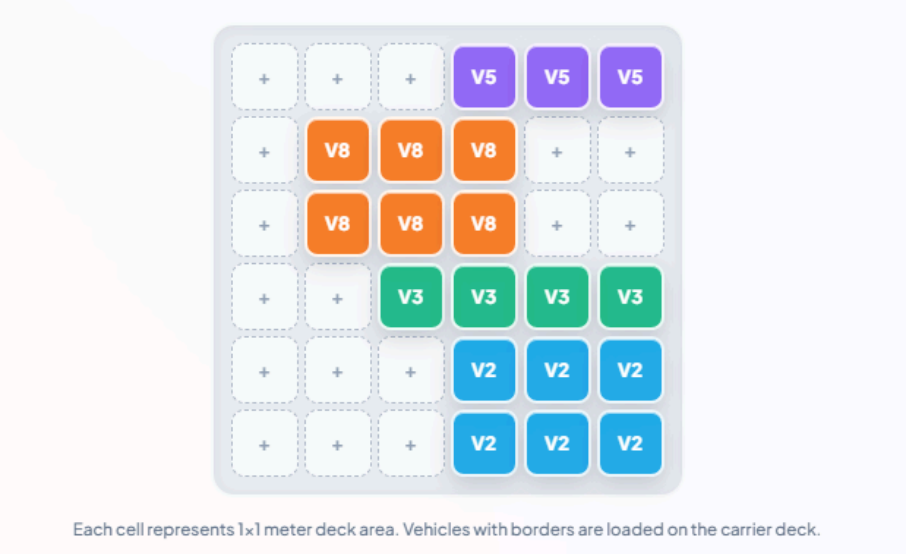
Tabel 11. Hasil *Sideways Move Hill-Climbing* 2

Iterasi 2
Initial State



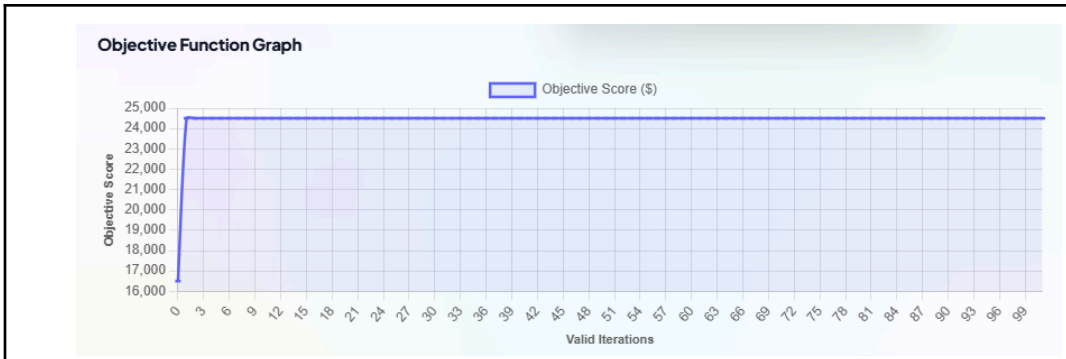
Total Kontainer yang Digunakan	Max Iterations	Initial Objective Score
3 (V2, V3, V5)	500	$16,500

Best State



Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Valid Iterations
4 (V2, V3, V5, V8)	$24,500	0.2993s	101

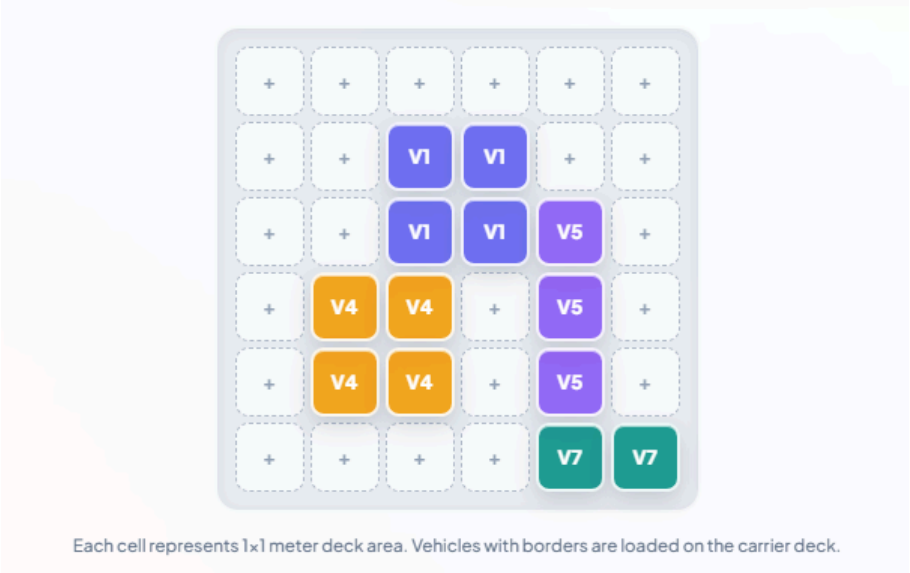
Plot Best Objective Function over Iterations



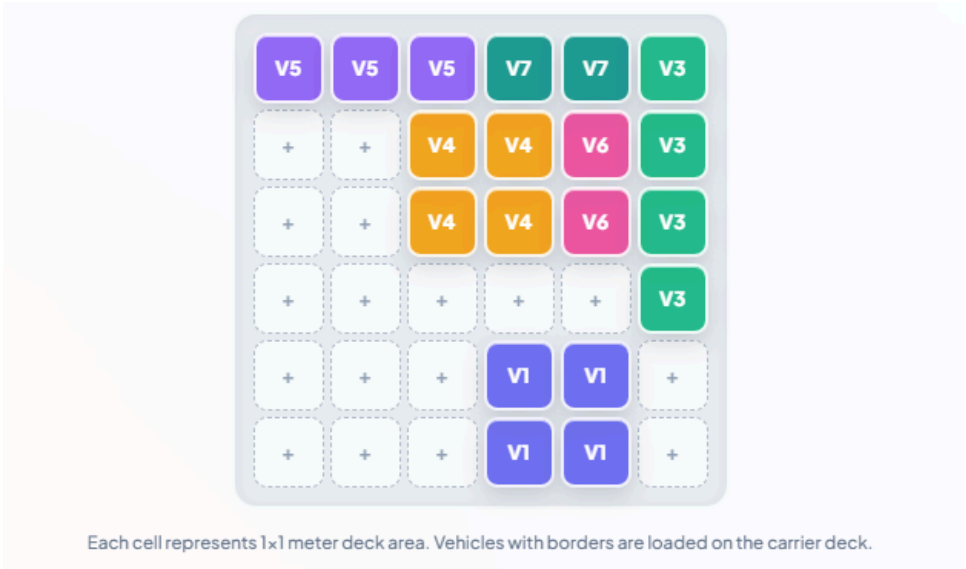
Experiment History Log Clear History

RUN #	ALGORITHM	PARAMETERS USED	OBJECTIVE SCORE	VALID ITERATIONS	TRIAL EVALUATIONS	EXECUTION TIME	STATUS
Run #1	Hill-Climbing with Sideways Move	[Fee] Max Sideways: 100, Max Iter: 500	$24,500	102	7,214	0.2716s	Valid
Run #2	Hill-Climbing with Sideways Move	[Fee] Max Sideways: 100, Max Iter: 500	$24,500	101	7,141	0.2993s	Valid
AVERAGE SUMMARY (2 Runs)			$24,500	102	—	0.2854s	—

Tabel 12. Hasil *Sideways Move Hill-Climbing* 3

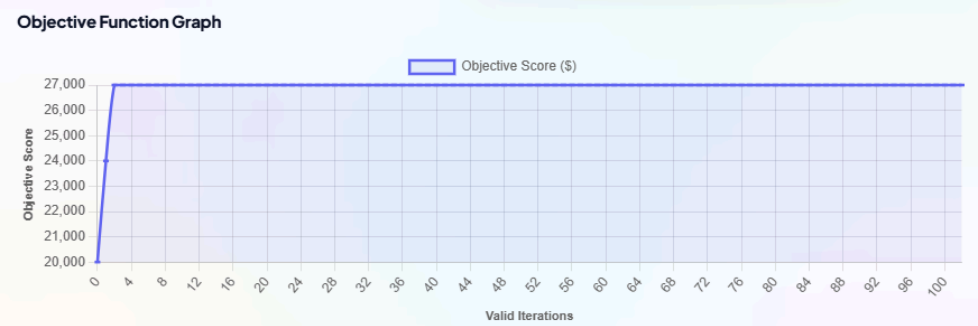
Iterasi 3		
Initial State		
		
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Objective Score**
4 (V1, V4, V5, V7)	500	$20,000

Best State



Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Valid Iterations
6 (V1, V3, V4, V5, V6, V7)	$27,000	0.4066s	102

Plot Best Objective Function over Iterations

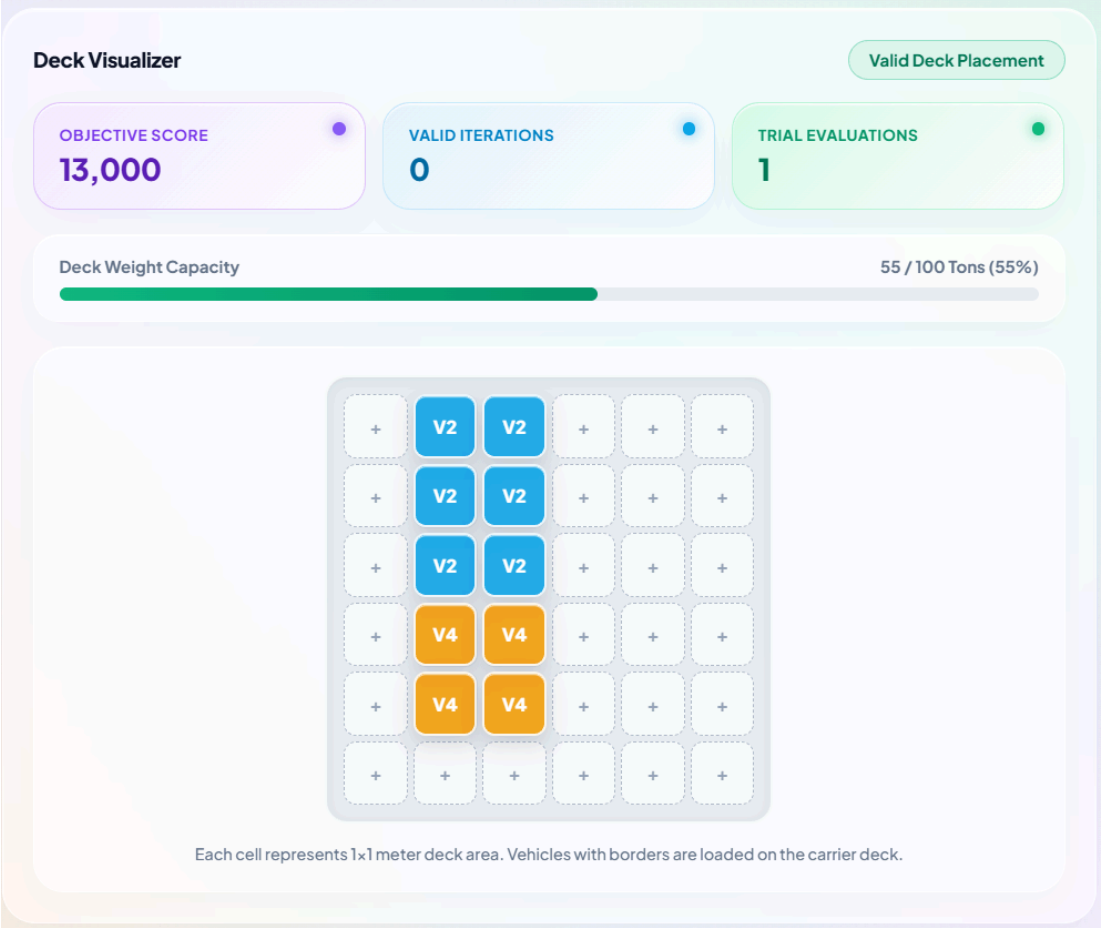


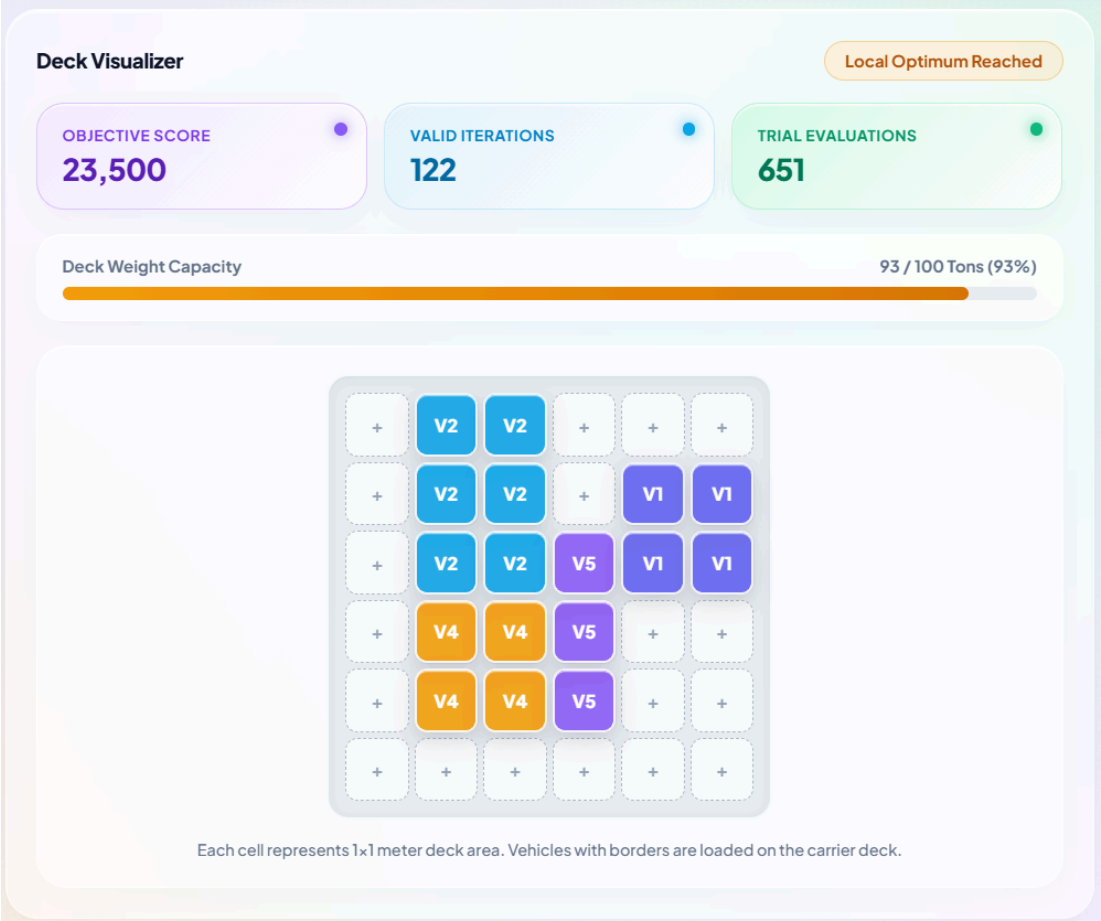
Experiment History Log Clear History

RUN #	ALGORITHM	PARAMETERS USED	OBJECTIVE SCORE	VALID ITERATIONS	TRIAL EVALUATIONS	EXECUTION TIME	STATUS
Run #1	Hill-Climbing with Sideways Move	[Fee] Max Sideways: 100, Max Iter: 500	$24,500	102	7,214	0.2716s	Valid
Run #2	Hill-Climbing with Sideways Move	[Fee] Max Sideways: 100, Max Iter: 500	$24,500	101	7,141	0.2993s	Valid
Run #3	Hill-Climbing with Sideways Move	[Fee] Max Sideways: 100, Max Iter: 500	$27,000	102	7,311	0.4066s	Valid
AVERAGE SUMMARY (3 Runs)			$25,333	102	—	0.3258s	—

2.6.3 Hasil *Simulated Annealing*

Tabel 13. Hasil *Simulated Annealing* 1

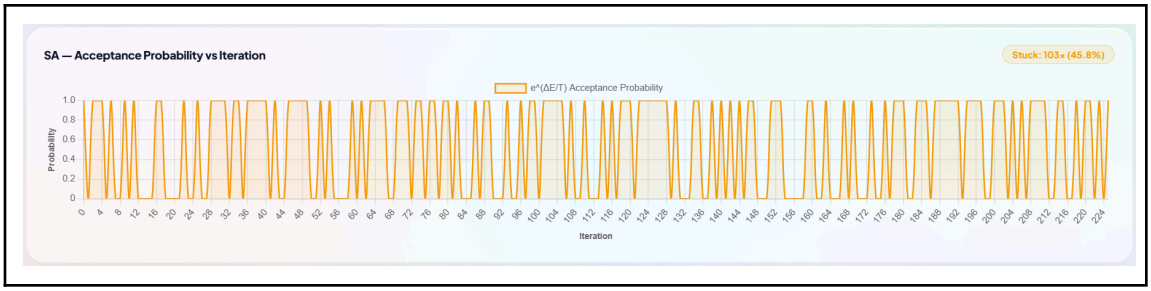
Iterasi 1				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Temperature**	**Cooling Rate (Alpha)**	**Initial Objective Score**
2 (V2, V4)	500	1000	0,95	$13,000
Best State				



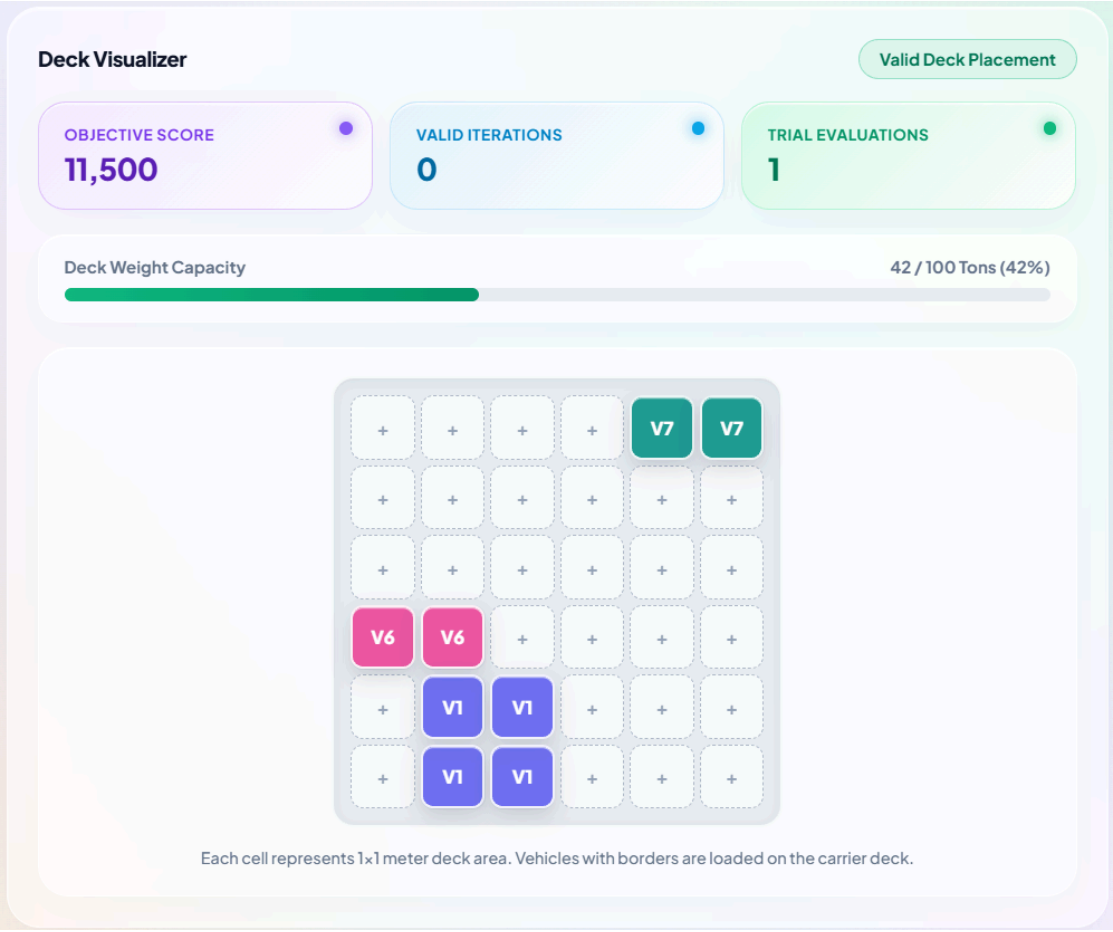
Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
4 (V1, V2, V4, V5)	$23,500	0.0225s	103 (45.8%)	122

Plot Objective Function over Iterations

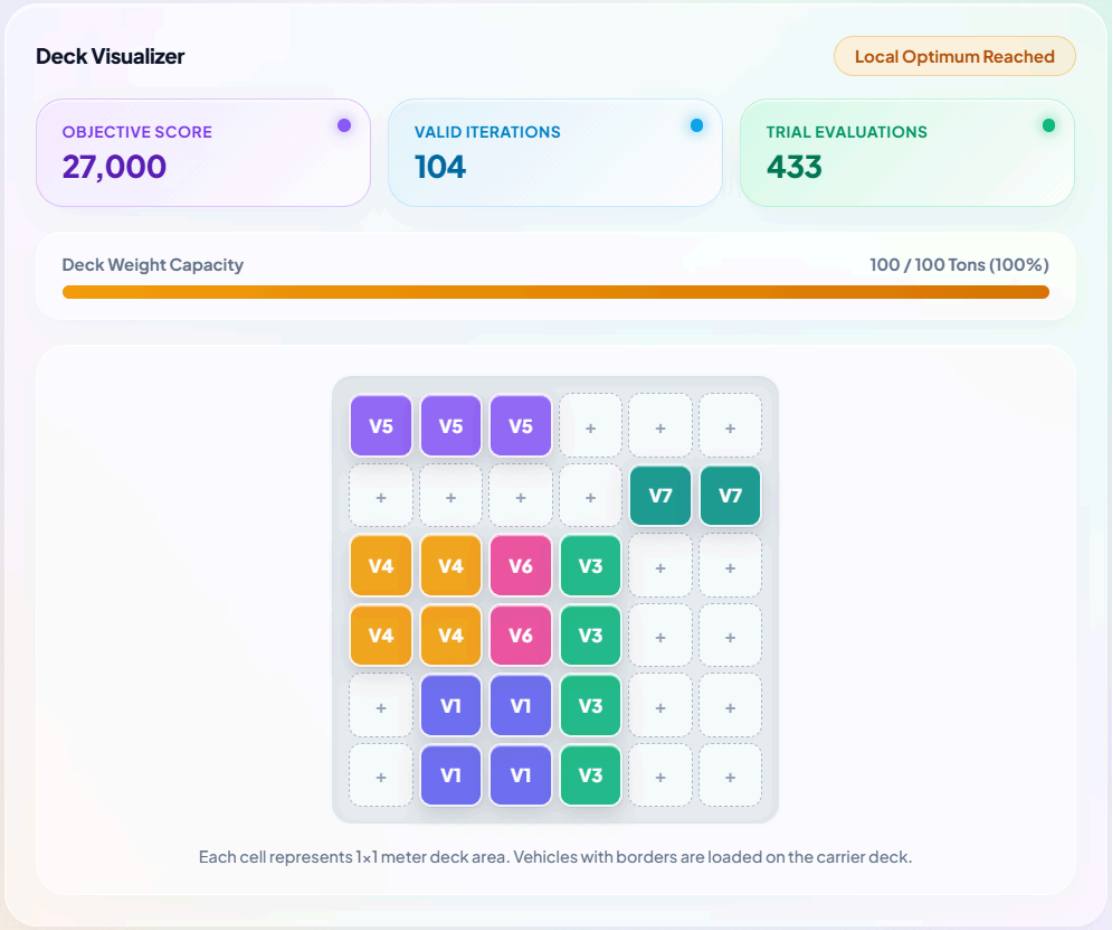
Plot probability $e^{\Delta E/T}$ over Iterations



Tabel 14. Hasil *Simulated Annealing* 2

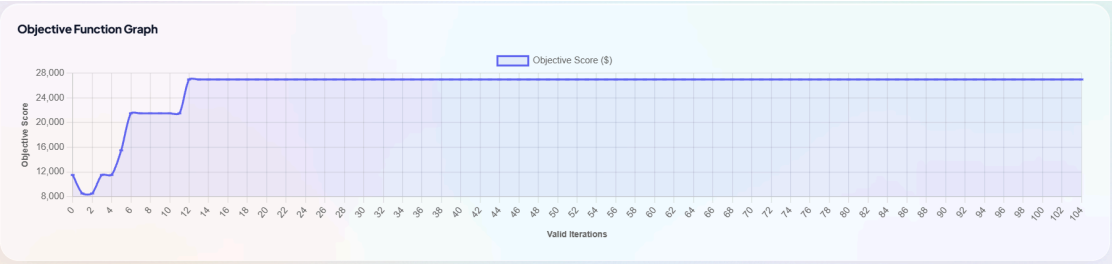
Iterasi 2				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Temperature**	**Cooling Rate (Alpha)**	**Initial Objective Score**
3 (V1, V6, V7)	500	1000	0,95	$11,500

Best State

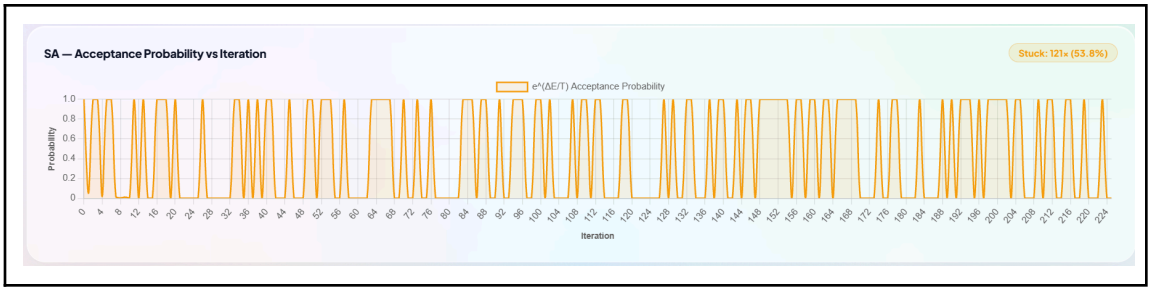


Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
6 (V1, V3, V4, V5, V6, V7)	$27,000	0.0180s	121 (53.8%)	104

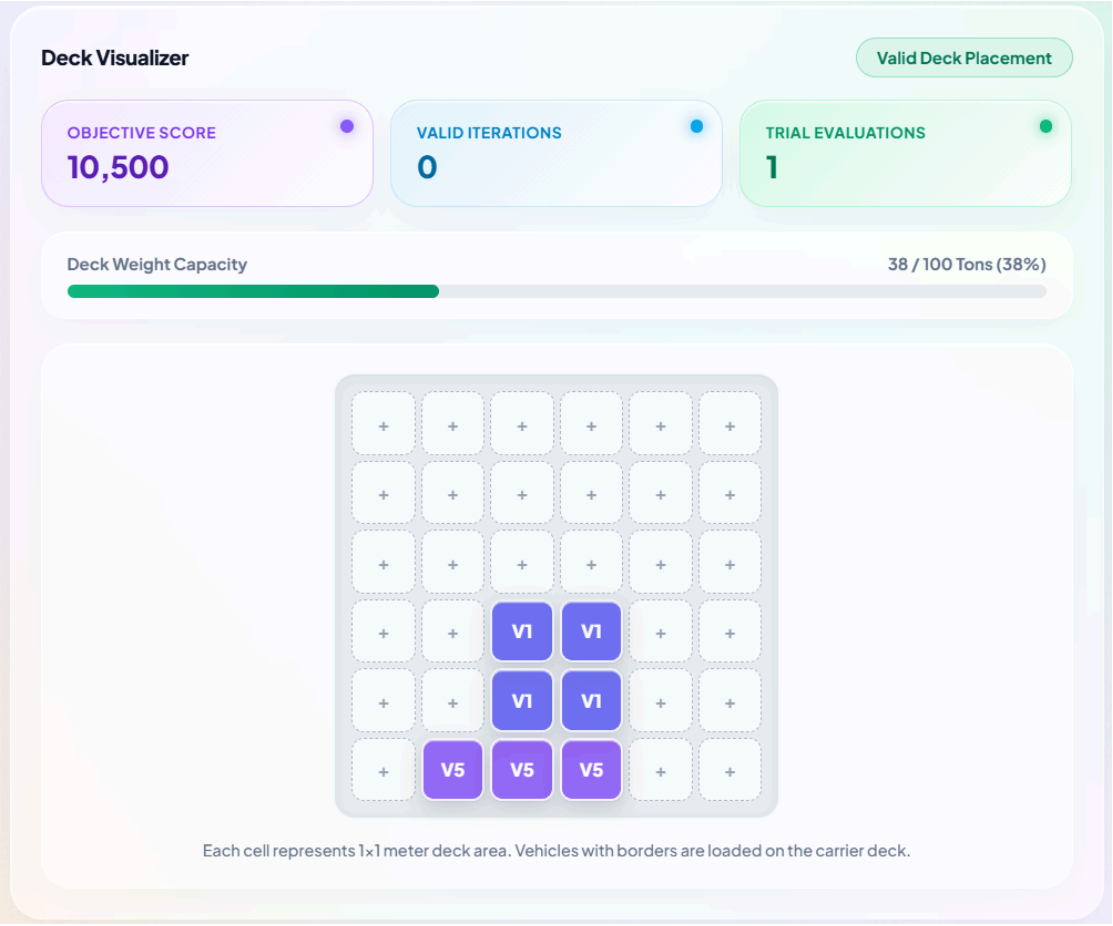
Plot Objective Function over Iterations



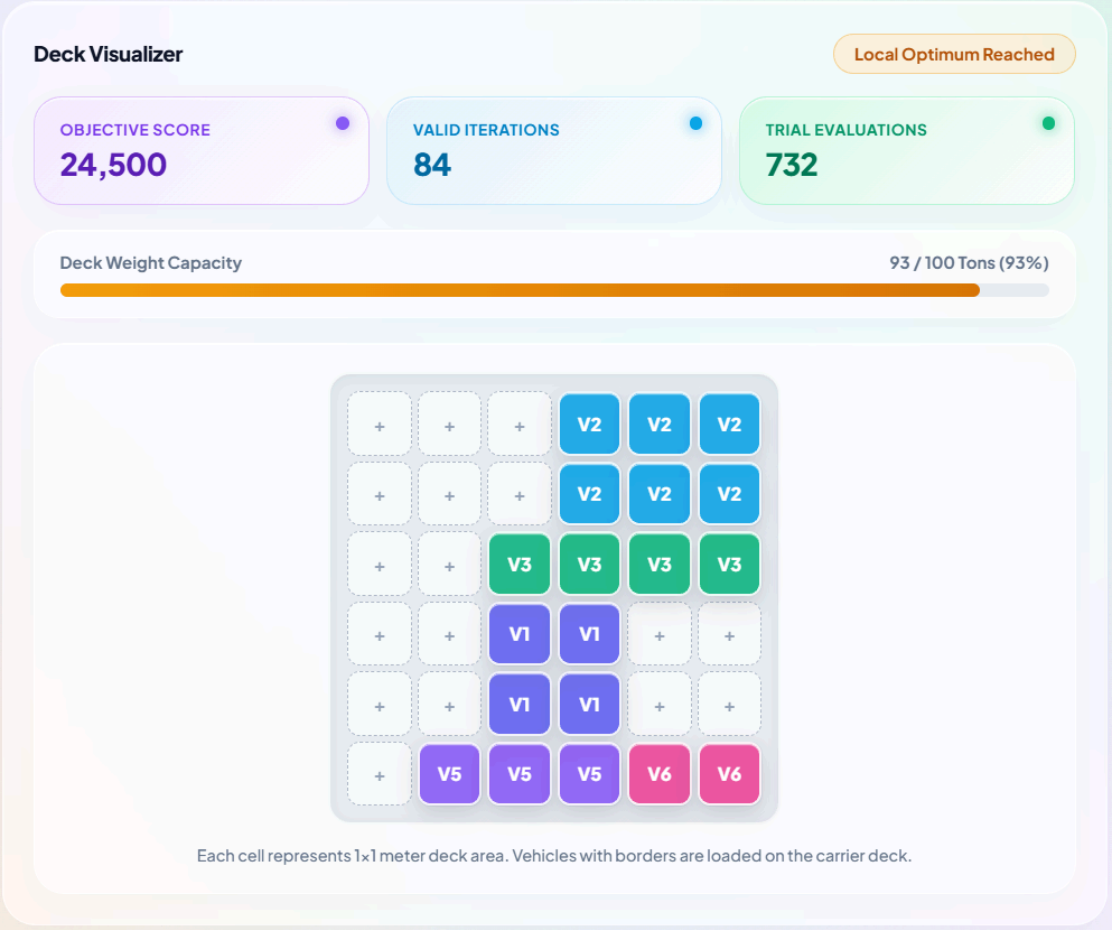
Plot probability $e^{\Delta E/T}$ over Iterations



Tabel 15. Hasil *Simulated Annealing* 3

Iterasi 3				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Temperature**	**Cooling Rate (Alpha)**	**Initial Objective Score**
2 (V1, V5)	500	1000	0,95	$10,500

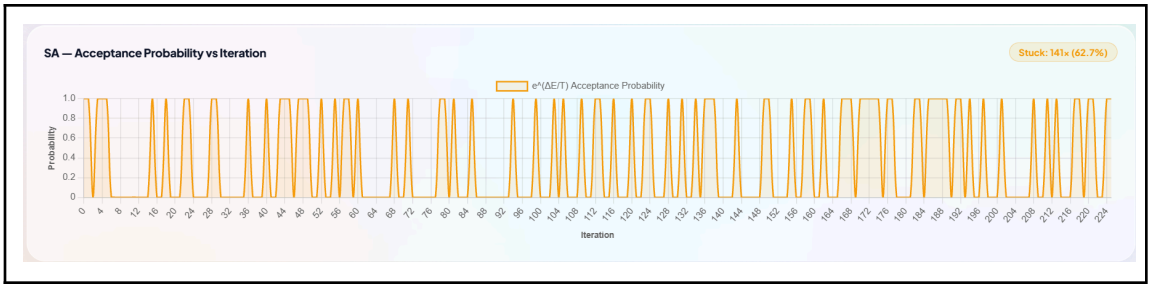
Best State



Total Kontainer yang Digunakan	Best Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
5 (V1, V2, V3, V5. V6)	$24,500	0.0149s	141 (62.7%)	84

Plot Objective Function over Iterations

Plot probability $e^{\Delta E/T}$ over Iterations



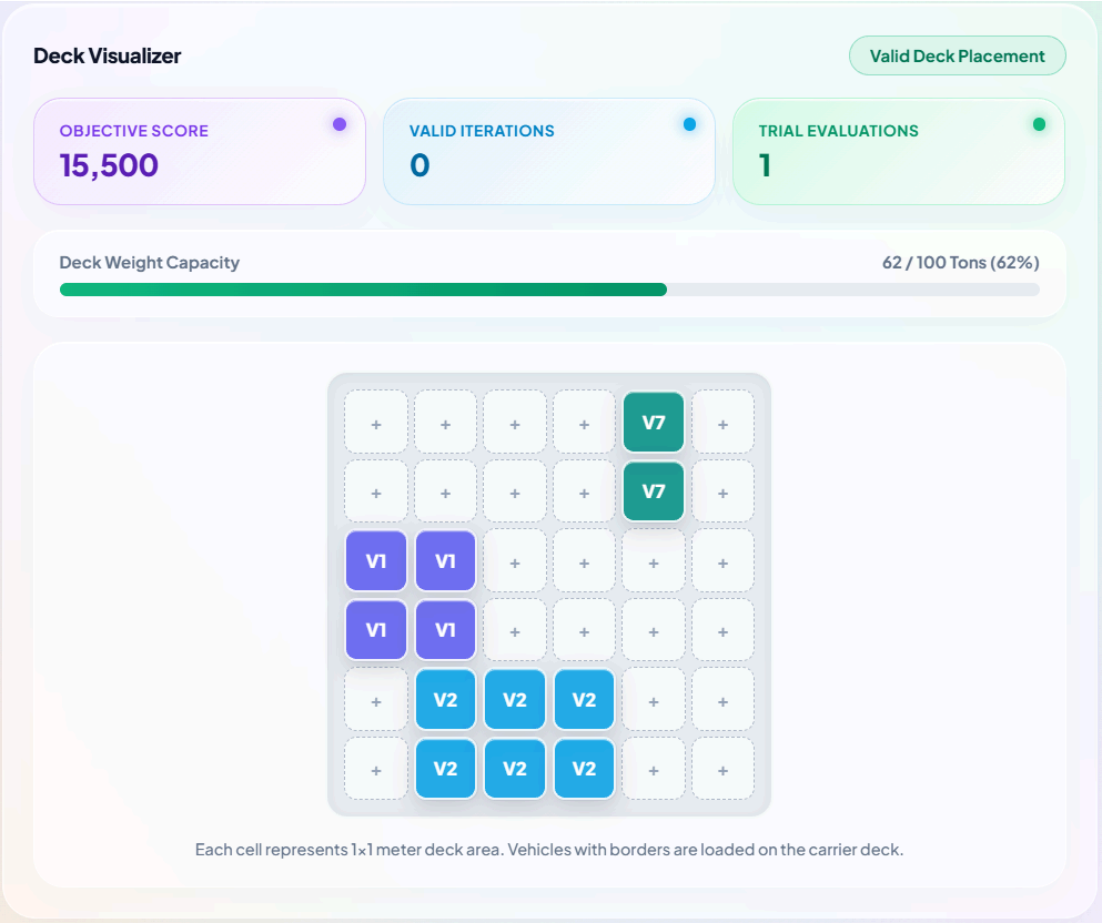
2.6.4 Hasil *Genetic Algorithm*

Eksperimen 1: Jumlah populasi sebagai kontrol dengan 3 variasi iterasi yang berbeda-beda.

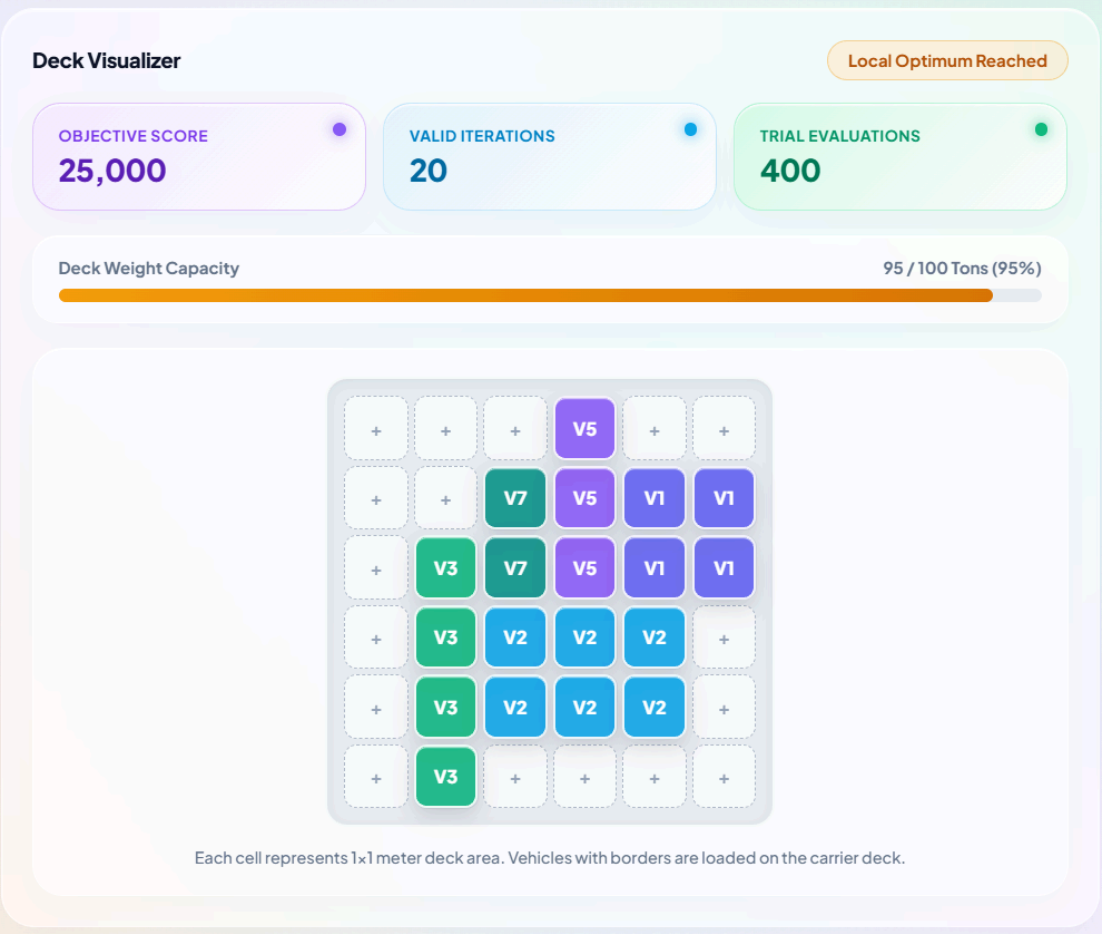
Fixed Parameters	
Population Size	20
Mutation Rate	0,15

Tabel 16. Hasil *Genetic Algorithm 1*

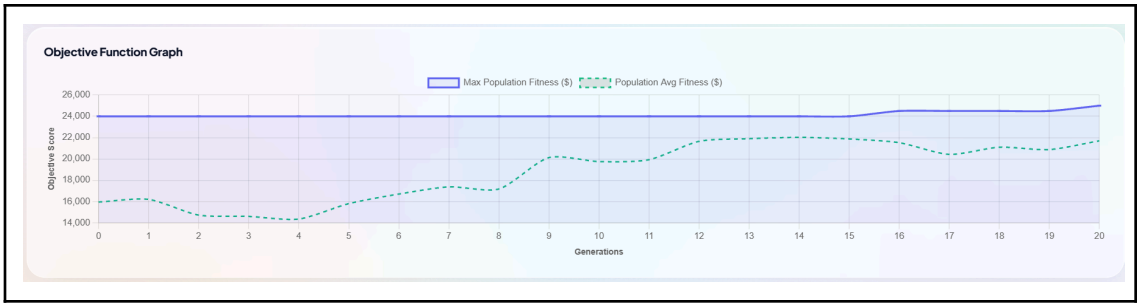
Percobaan 1 (max iterations: 20)
Initial State



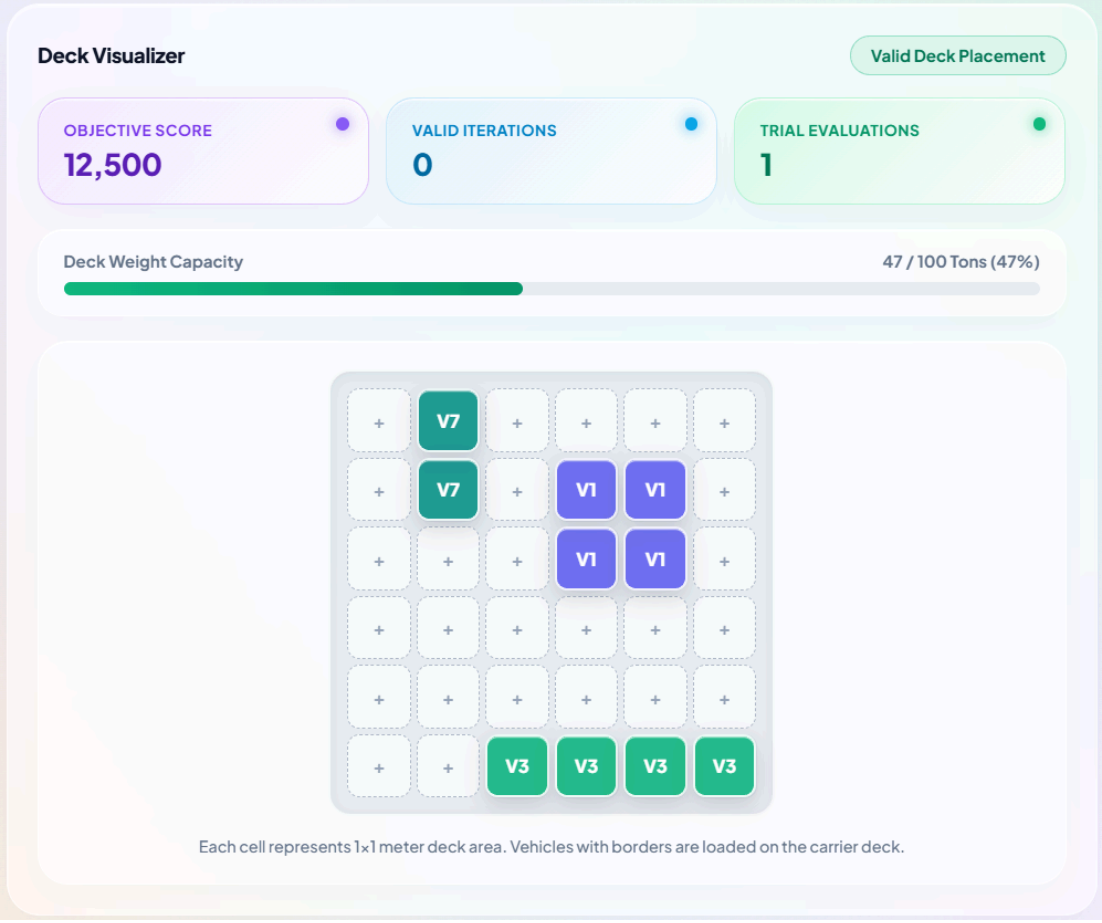
Run 1	
Total Kontainer Digunakan	**Initial Objective Score**
3 (V1, V2, V7)	$15,500
Run 2	
Total Kontainer Digunakan	**Initial Objective Score**
5 (V1, V3, V5, V7, V8)	$26,000
Run 3	
Total Kontainer Digunakan	**Initial Objective Score**
4 (V1, V4, V5, V8)	$24,500
Final State	



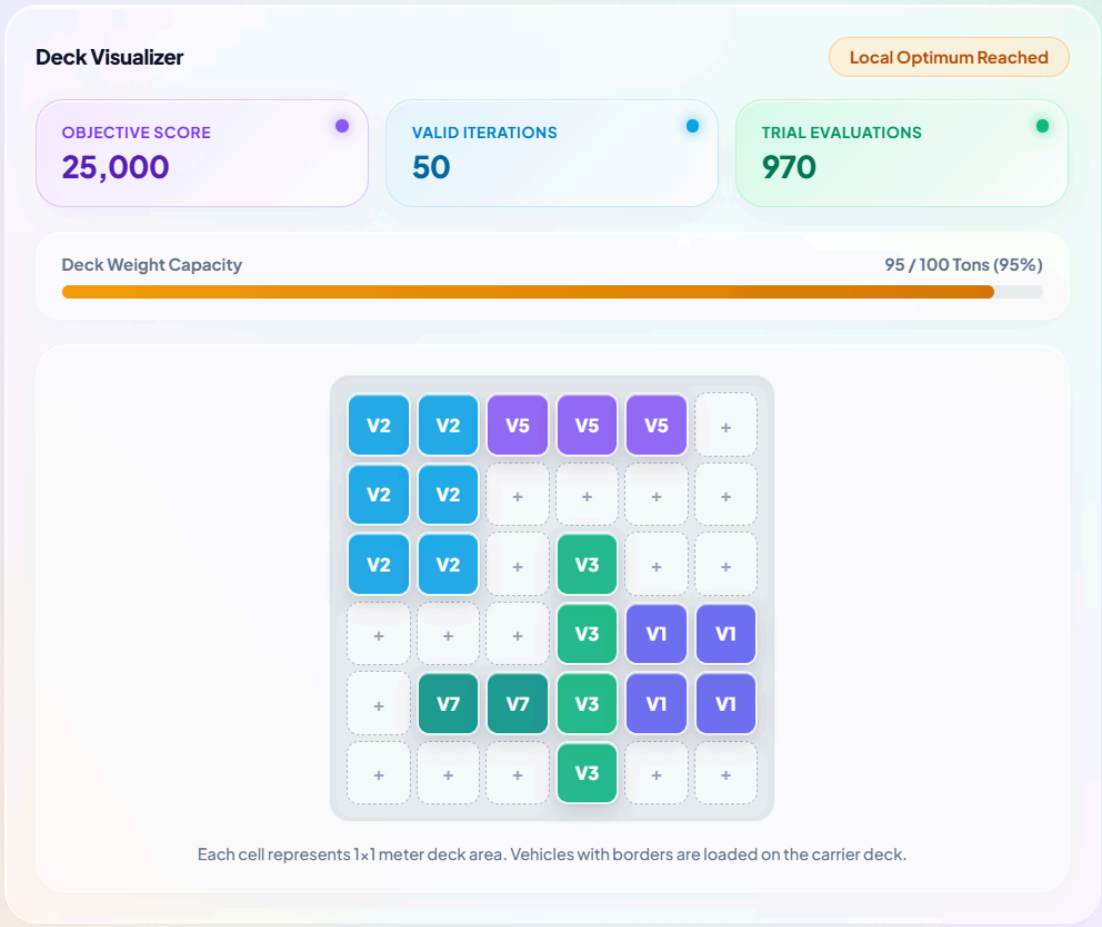
Run 1		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V1, V3, V5, V7, V8)	0.0168s	$26,000
Run 2		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
4 (V1, V4, V5, V8)	0.0150s	$24,500
Run 3		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V1, V2, V3, V5, V7)	0.0159s	$25,000
Final Plot Objective Function over Iterations		



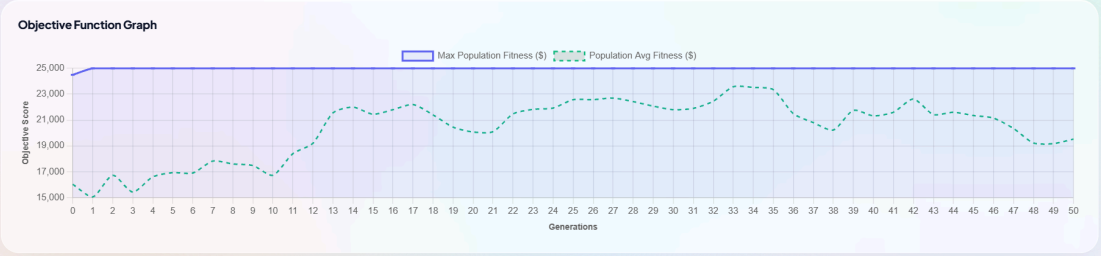
Tabel 17. Hasil *Genetic Algorithm 2*

Percobaan 2 (max iterations: 50)	
Initial State	
	
Run 1	
Total Kontainer Digunakan	**Initial Objective Score**
3 (V1, V3, V7)	$12,500

Run 2	
Total Kontainer Digunakan	**Initial Objective Score**
5 (V1, V2, V3, V5, V7)	$25,000
Run 3	
Total Kontainer Digunakan	**Initial Objective Score**
6 (V1, V3, V4, V5, V6, V7)	$27,000
Final State	

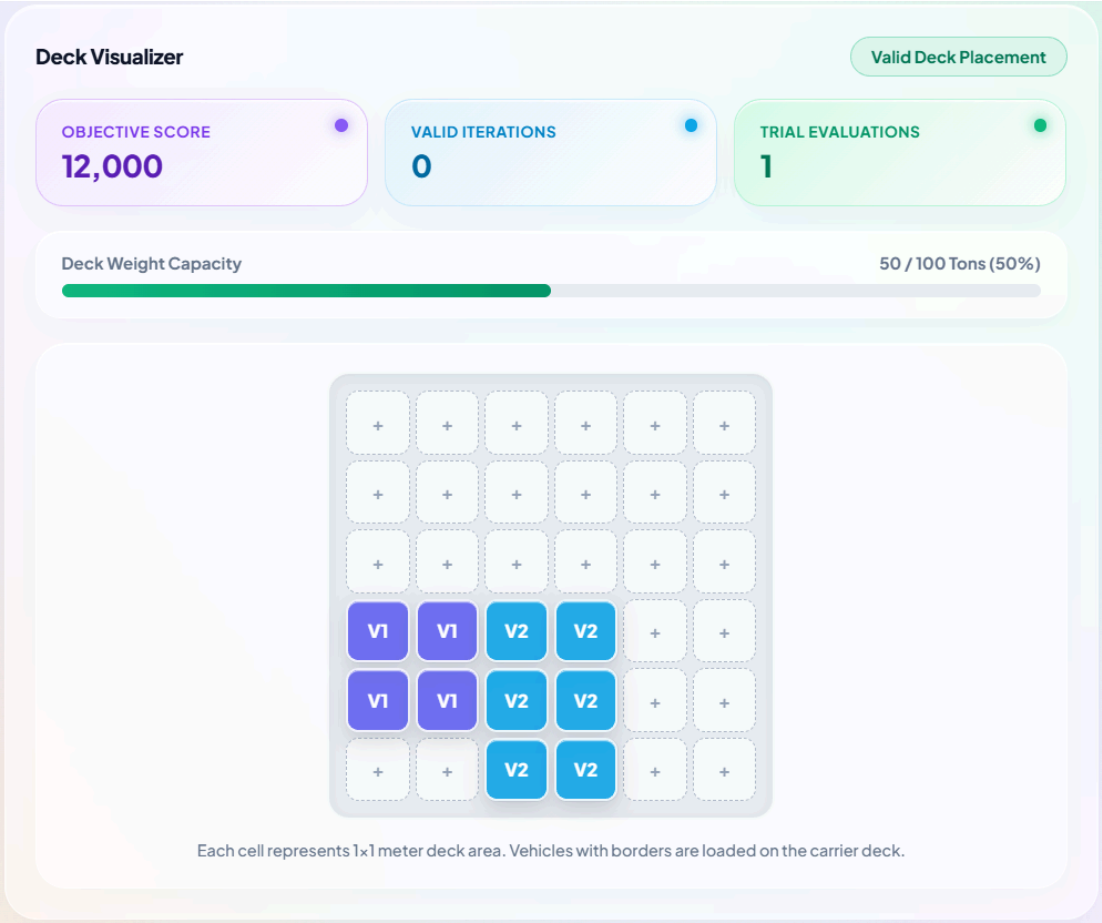


Run 1		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V1, V2, V3, V5, V7)	0.0560s	$25,000
Run 2		

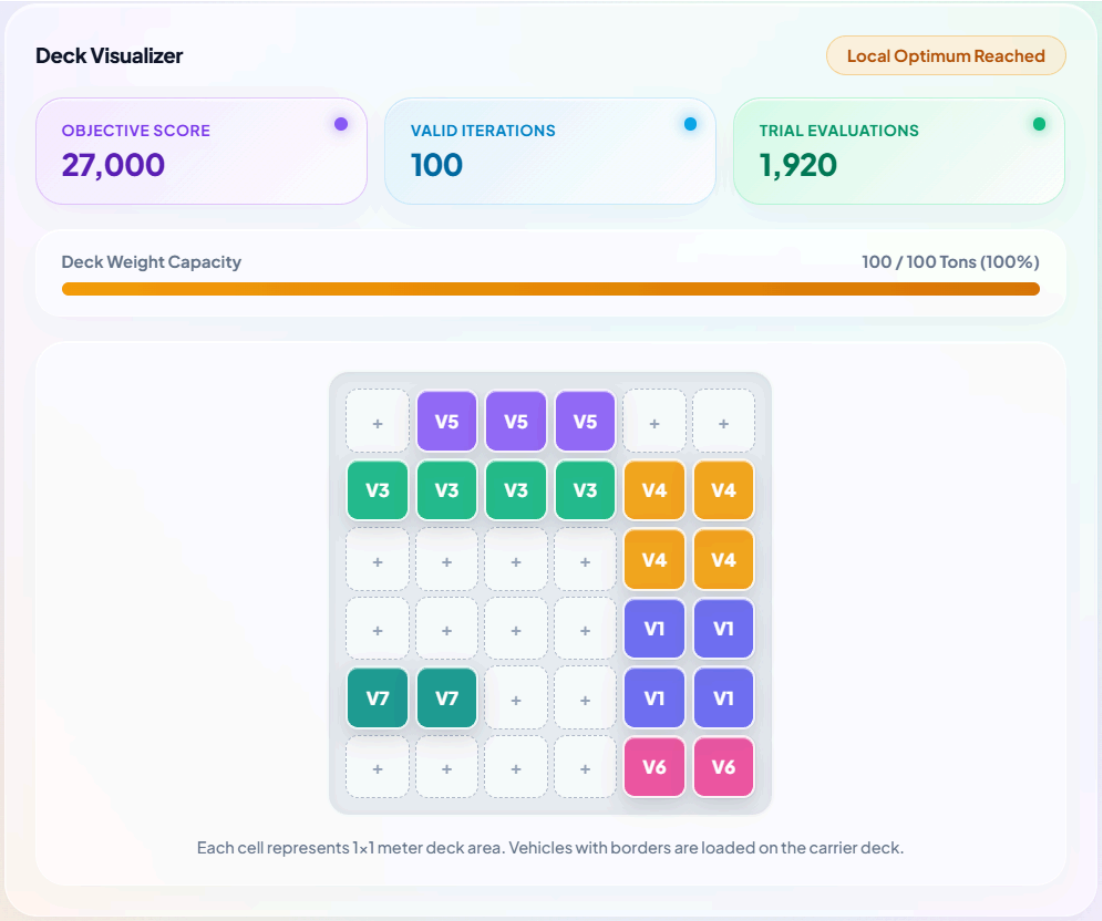
Total Kontainer yang Digunakan	Execution Time	Final Objective Score
6 (V1, V3, V4, V5, V6, V7)	0.0617s	$27,000
Run 3		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V1, V2, V3, V5, V7)	0.0415s	$25,000
Final Plot Objective Function over Iterations		
		

Tabel 18. Hasil *Genetic Algorithm 3*

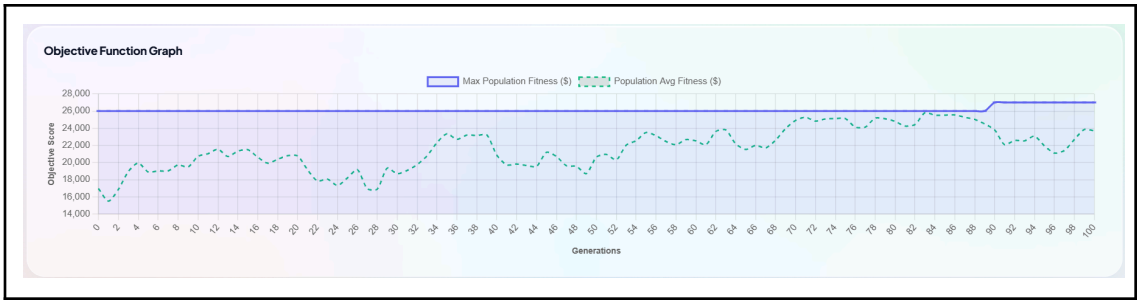
Percobaan 3 (max iterations: 100)
Initial State



Run 1	
Total Kontainer Digunakan	**Initial Objective Score**
2 (V1, V2)	$12,000
Run 2	
Total Kontainer Digunakan	**Initial Objective Score**
5 (V1, V2, V3, V5, V7)	$25,000
Run 3	
Total Kontainer Digunakan	**Initial Objective Score**
4 (V1, V2, V4, V5)	$23,500
Final State	



Run 1		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V1, V2, V3, V5, V7)	0.1207s	$25,000
Run 2		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
4 (V1, V2, V4, V5)	0.0801s	$23,500
Run 3		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
6 (V1, V3, V4, V5, V6, V7)	0.0812s	$27,000
Final Plot Objective Function over Iterations		

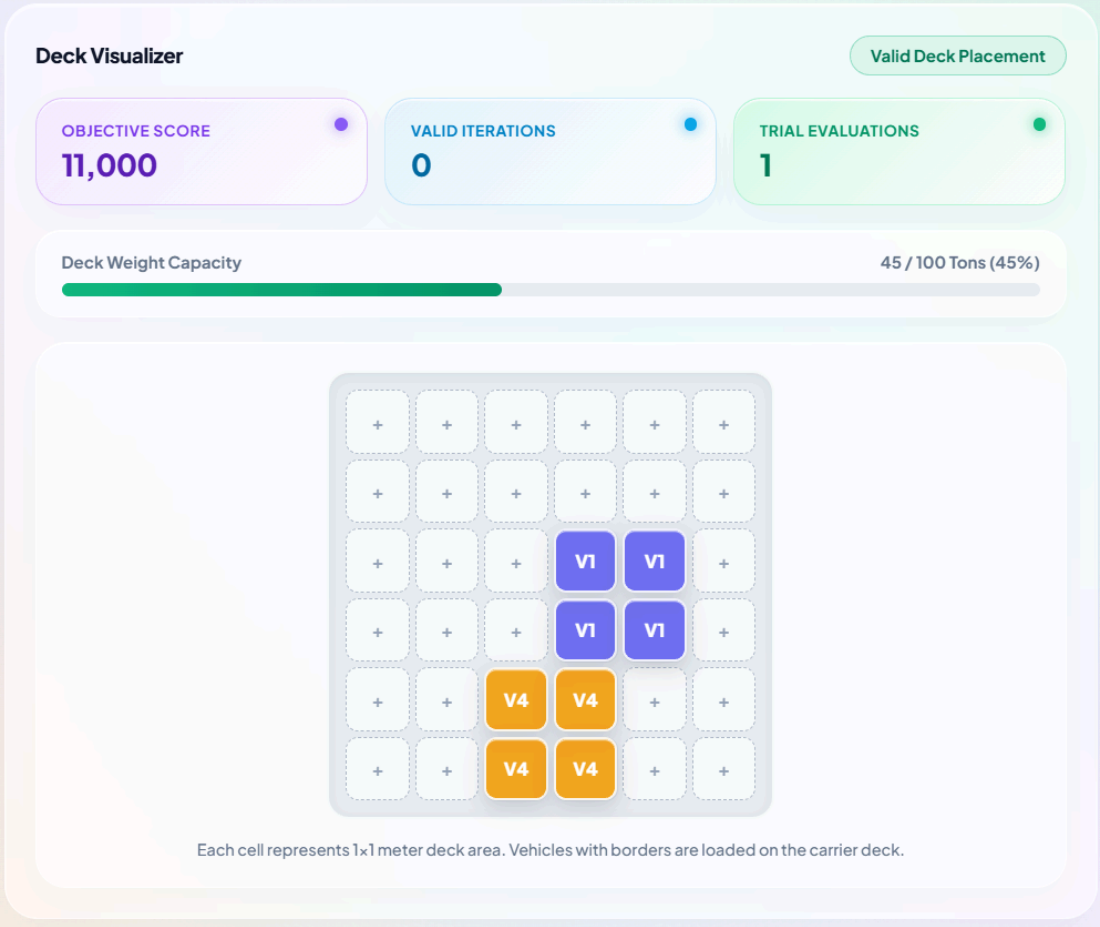


Eksperimen 2: Jumlah iterasi sebagai kontrol dengan 3 variasi jumlah populasi yang berbeda-beda.

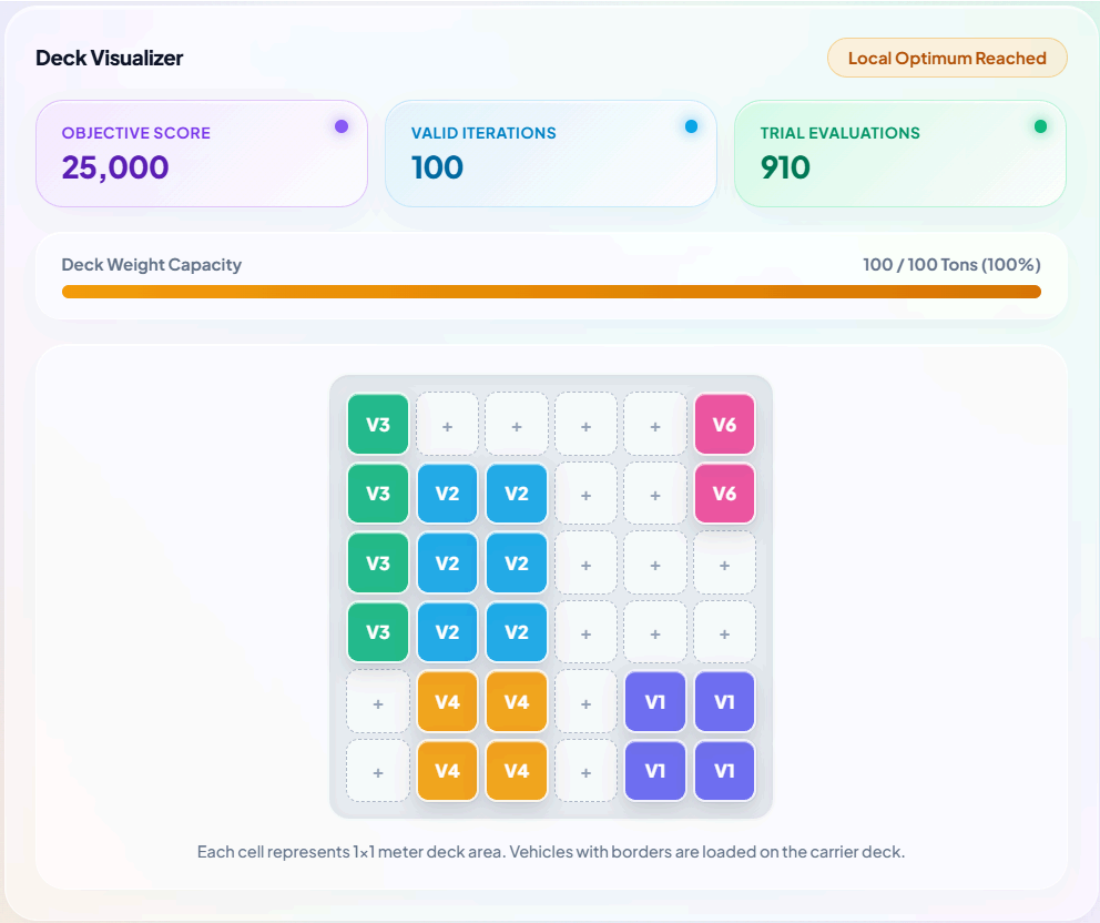
Fixed Parameters	
Max Iterations	100
Mutation Rate	0,15

Tabel 19. Hasil *Genetic Algorithm 4*

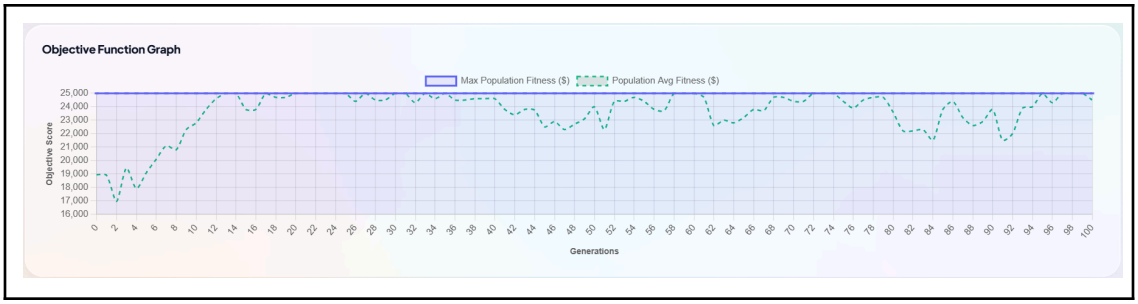
Percobaan 1 (population size: 10)
Initial State



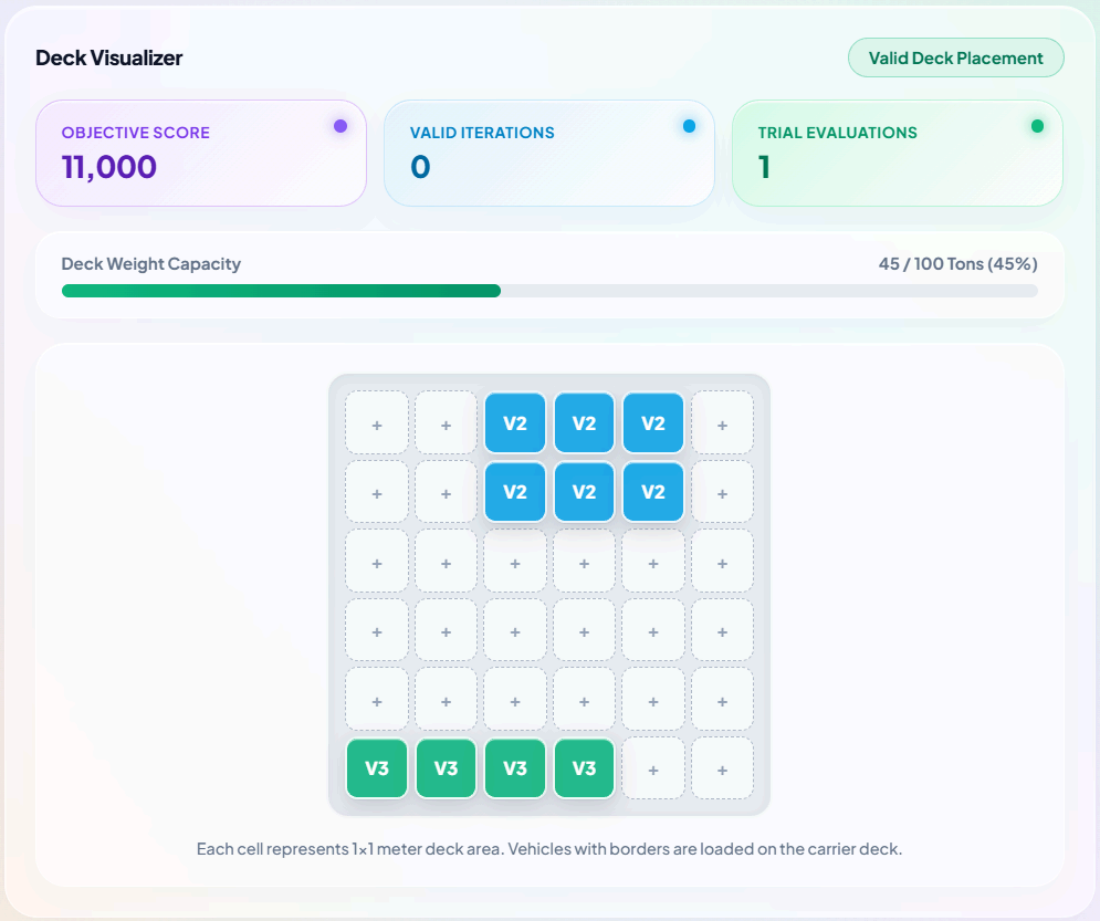
Run 1	
Total Kontainer Digunakan	**Initial Objective Score**
2 (V1, V4)	$11,000
Run 2	
Total Kontainer Digunakan	**Initial Objective Score**
5 (V2, V3, V4, V5, V6)	$25,500
Run 3	
Total Kontainer Digunakan	**Initial Objective Score**
5 (V2, V3, V4, V5, V7)	$26,000
Final State	

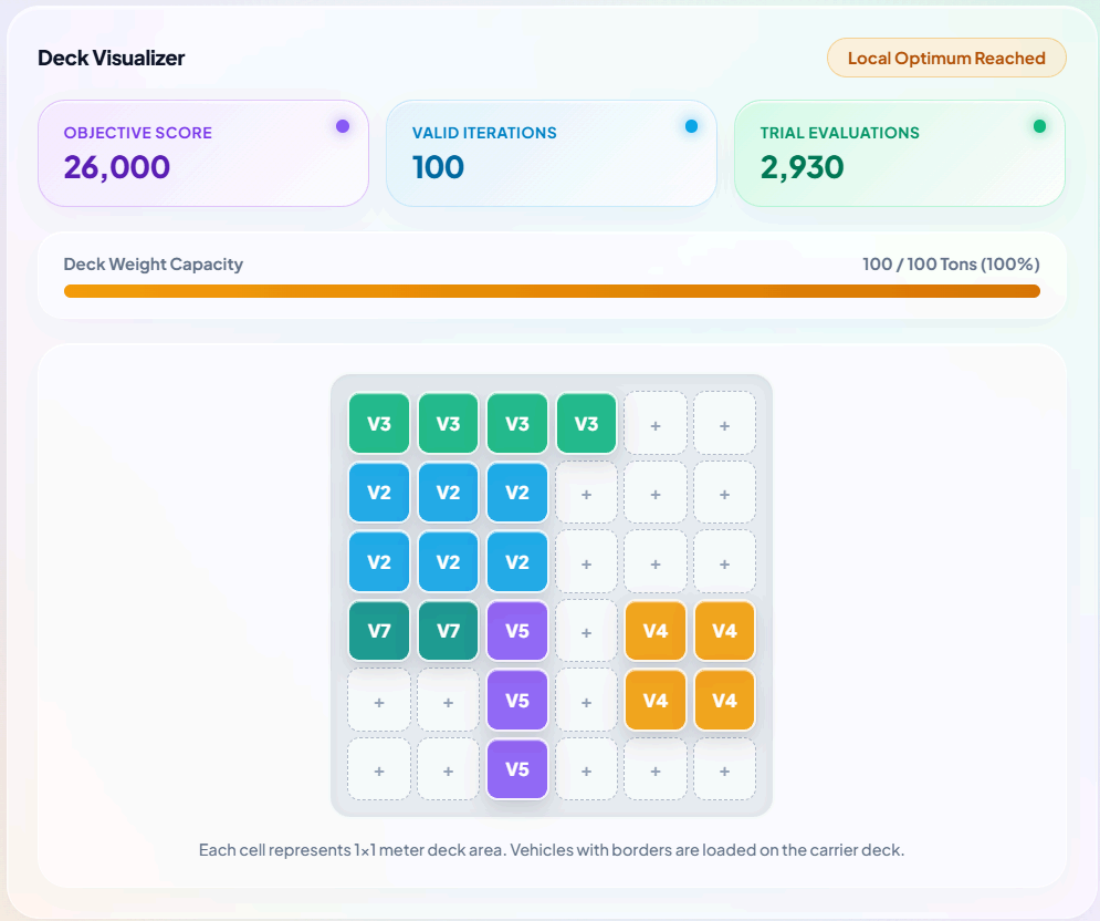


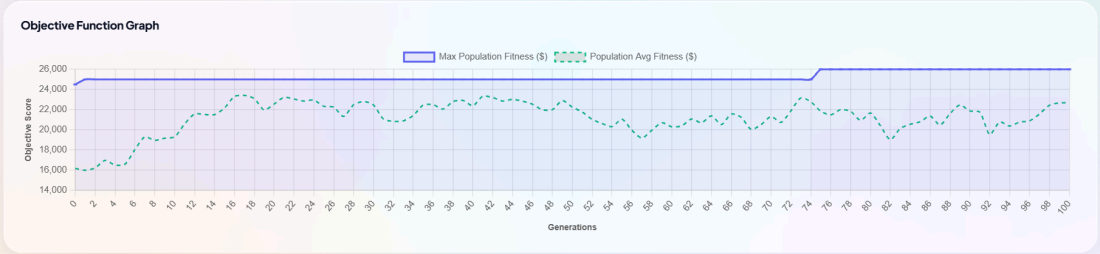
Run 1		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V2, V3, V4, V5, V6)	0.0403s	$25,500
Run 2		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V2, V3, V4, V5, V7)	0.0398s	$26,000
Run 3		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V1, V2, V3, V4, V6)	0.0420s	$25,000
Final Plot Objective Function over Iterations		



Tabel 20. Hasil *Genetic Algorithm 5*

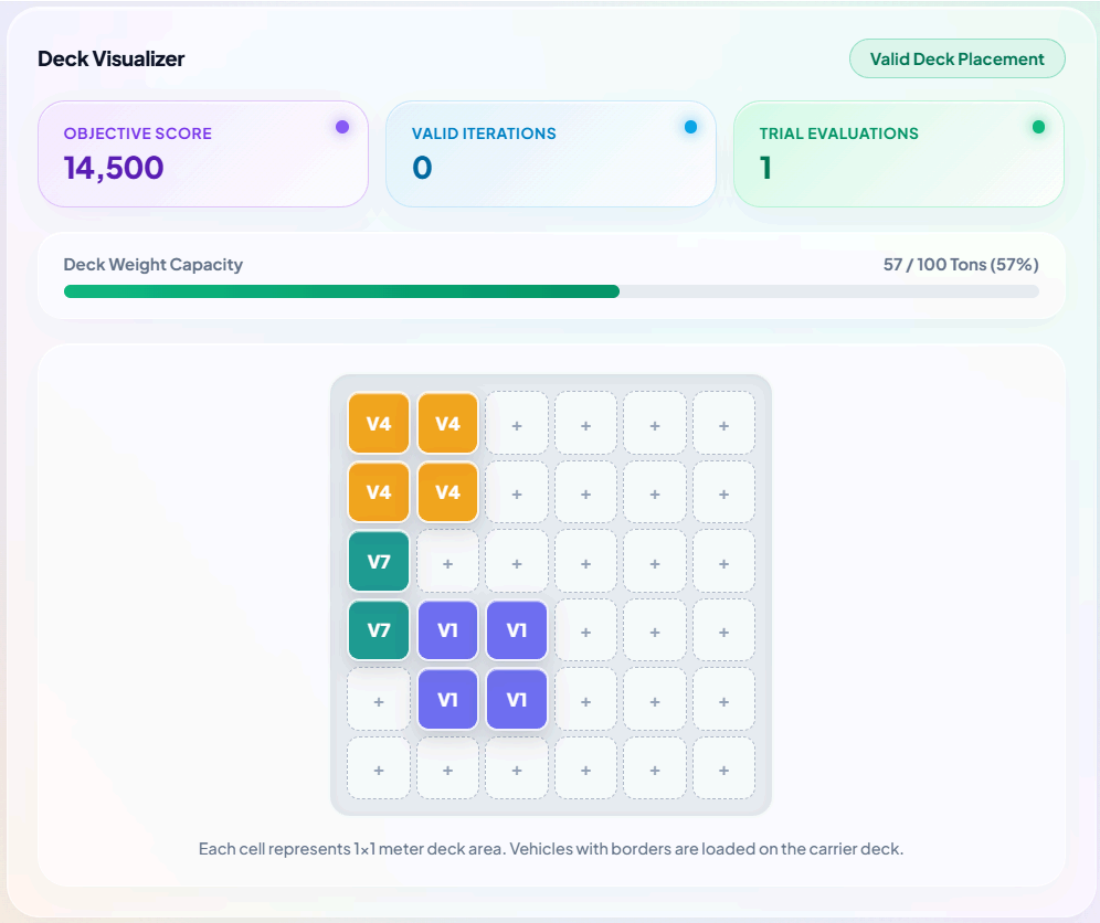
Percobaan 2 (population size: 30)	
Initial State	
	
Run 1	
Total Kontainer Digunakan	**Initial Objective Score**
2 (V2, V3)	$11,000

<table>
<tr><th colspan="2">Run 2</th></tr>
<tr><th>Total Kontainer Digunakan</th><th>Initial Objective Score</th></tr>
<tr><td>5 (V1, V2, V3, V5, V7)</td><td>$25,000</td></tr>
<tr><th colspan="2">Run 3</th></tr>
<tr><th>Total Kontainer Digunakan</th><th>Initial Objective Score</th></tr>
<tr><td>5 (V1, V2, V3, V5, V7)</td><td>$25,000</td></tr>
<tr><th colspan="2">Final State</th></tr>
<tr><td colspan="2"> 
</td></tr>
<tr><th colspan="3">Run 1</th></tr>
<tr><th>Total Kontainer yang Digunakan</th><th>Execution Time</th><th>Final Objective Score</th></tr>
<tr><td>5 (V1, V2, V3, V5, V7)</td><td>0.1229s</td><td>$25,000</td></tr>
<tr><th colspan="3">Run 2</th></tr>
</table>

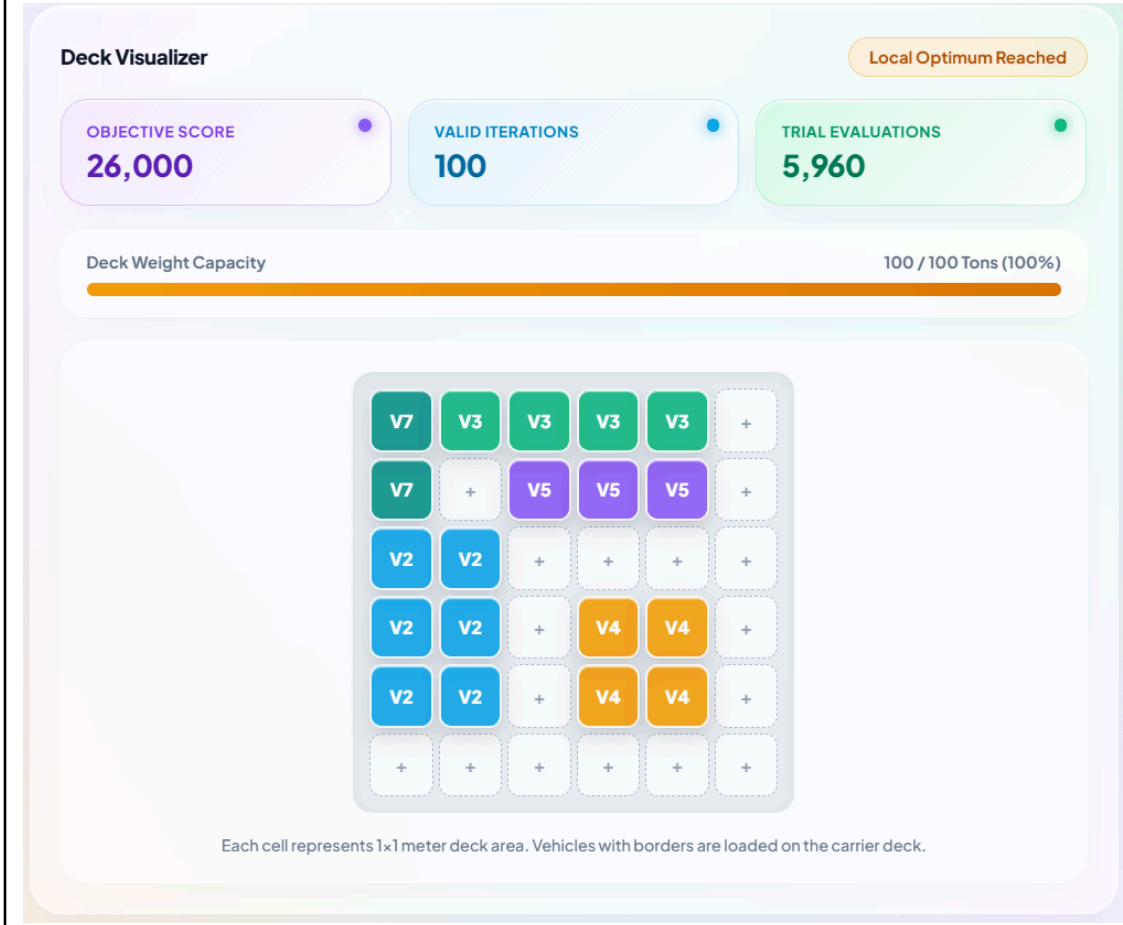
Total Kontainer yang Digunakan	Execution Time	Final Objective Score
5 (V1, V2, V3, V5, V7)	0.1187s	$25,000
Run 3		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V2, V3, V4, V5, V7)	0.1214s	$26,000
Final Plot Objective Function over Iterations		
		

Tabel 21. Hasil *Genetic Algorithm 6*

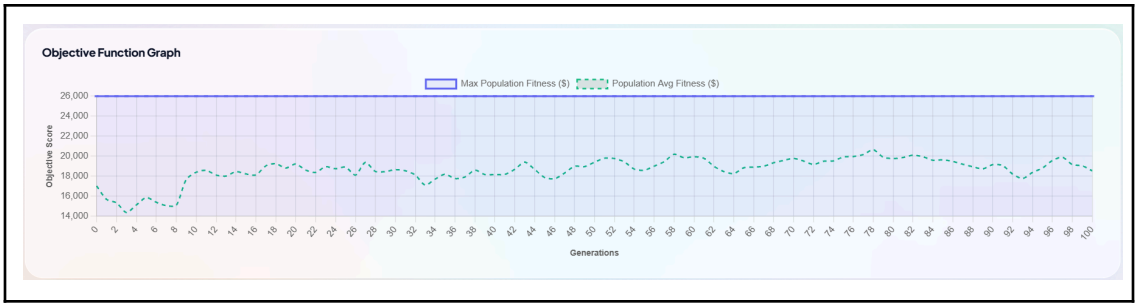
Percobaan 3 (population size: 60)
Initial State



Run 1	
Total Kontainer Digunakan	**Initial Objective Score**
3 (V1, V4, V7)	$14,500
Run 2	
Total Kontainer Digunakan	**Initial Objective Score**
5 (V1, V2, V3, V5, V7)	$25,000
Run 3	
Total Kontainer Digunakan	**Initial Objective Score**
5 (V2, V3, V4, V5, V6)	$25,500
Final State	



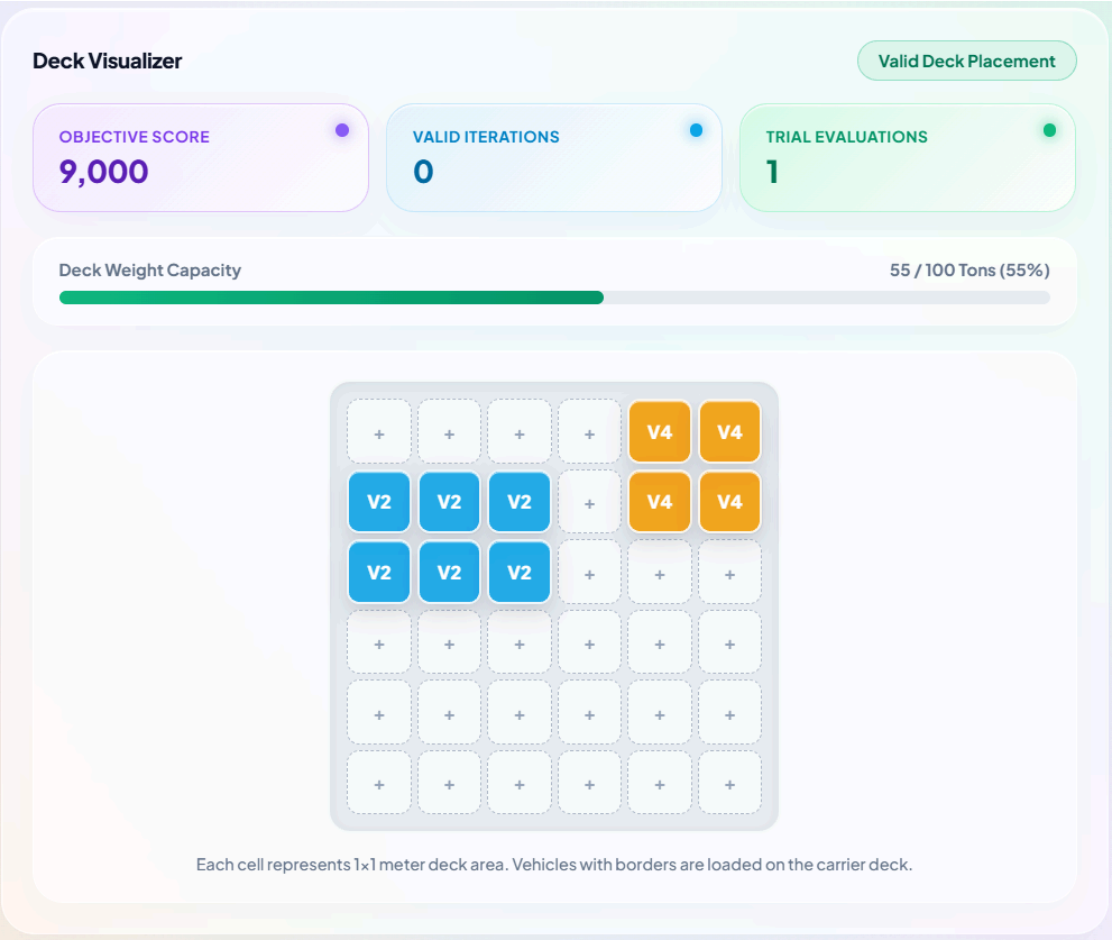
Run 1		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V1, V2, V3, V5, V7)	0.2670s	$25,000
Run 2		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V2, V3, V4, V5, V6)	0.2655s	$25,500
Run 3		
Total Kontainer yang Digunakan	**Execution Time**	**Final Objective Score**
5 (V2, V3, V4, V5, V7)	0.2689s	$26,000
Final Plot Objective Function over Iterations		

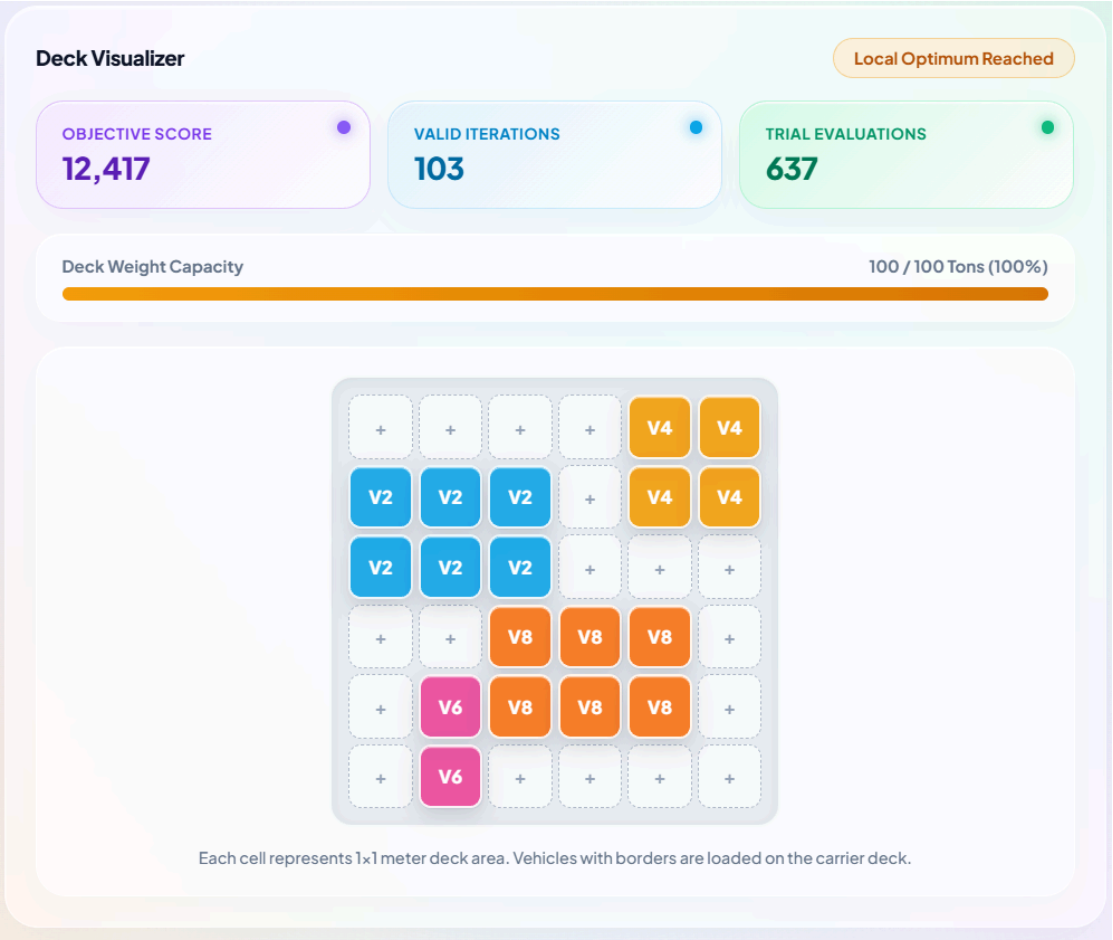


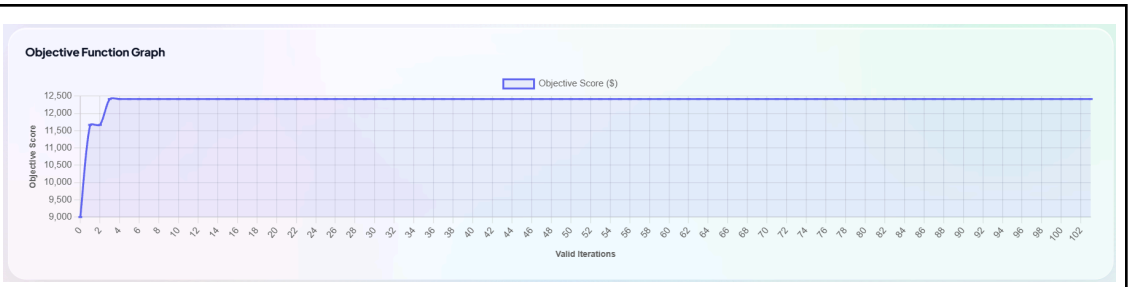
2.6.5 Hasil Fungsi Objective Alternatif

Eksperimen 1: ETA-*Weighted Fee* (ETA *Priority*)

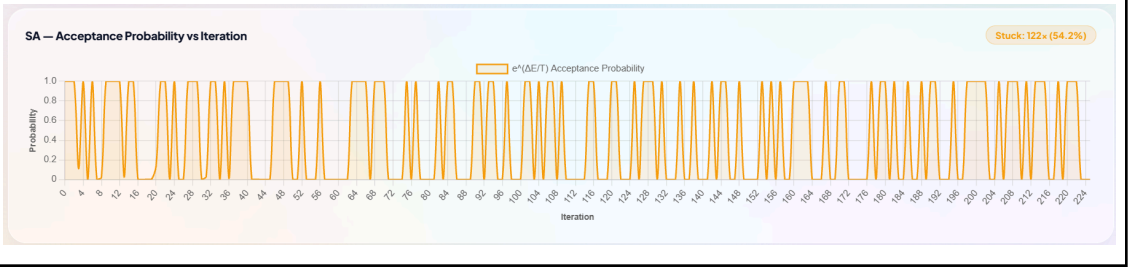
Tabel 22. Hasil ETA *Priority* dengan *Simulated Annealing* 1

Iterasi 1				
Initial State				
				
Total Kontainer	**Max Iterations**	**Initial**	**Cooling Rate**	**Initial Objective**

yang Digunakan		Temperature	(Alpha)	Score
2 (V2, V4)	500	1000	0,95	$9,000
Final State				
				
Total Kontainer yang Digunakan	**Final Objective Score**	**Execution Time**	**Frekuensi Stuck**	**Valid Iterations**
4 (V2, V4, V6, V8)	$12,417	0.0151s	122 (54.2%)	103
Plot Objective Function over Iterations				

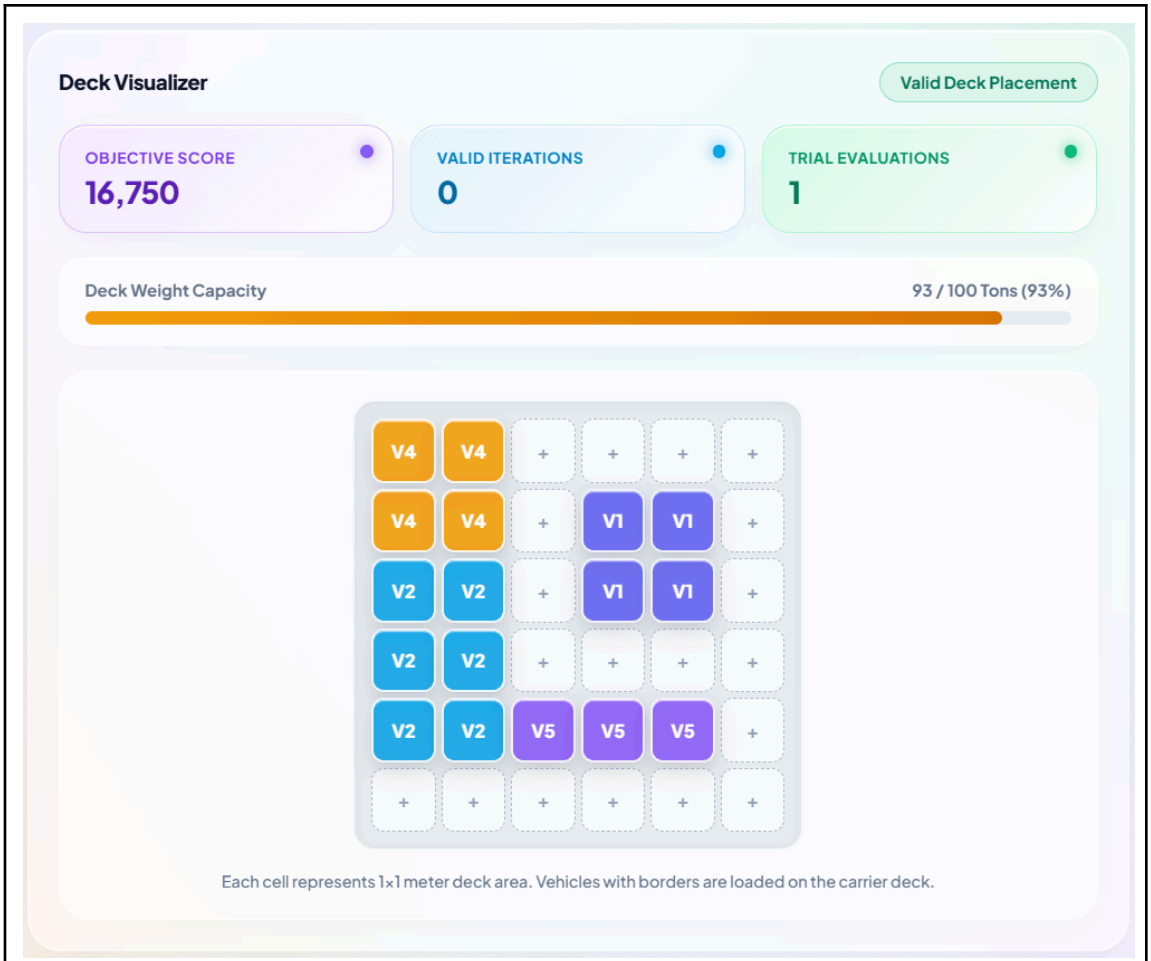


Plot probability $e^{\Delta E/T}$ over Iterations

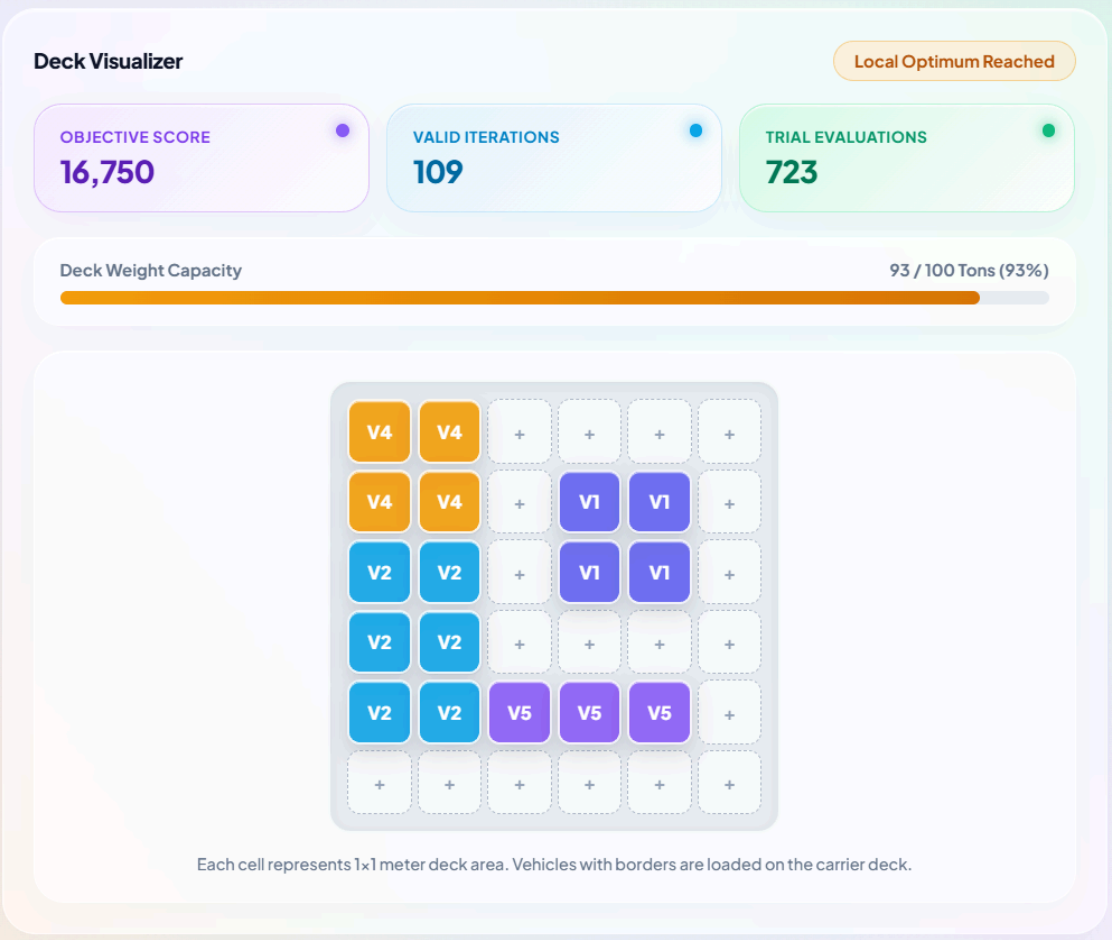


Tabel 23. Hasil ETA *Priority* dengan *Simulated Annealing* 2

Iterasi 2
Initial State



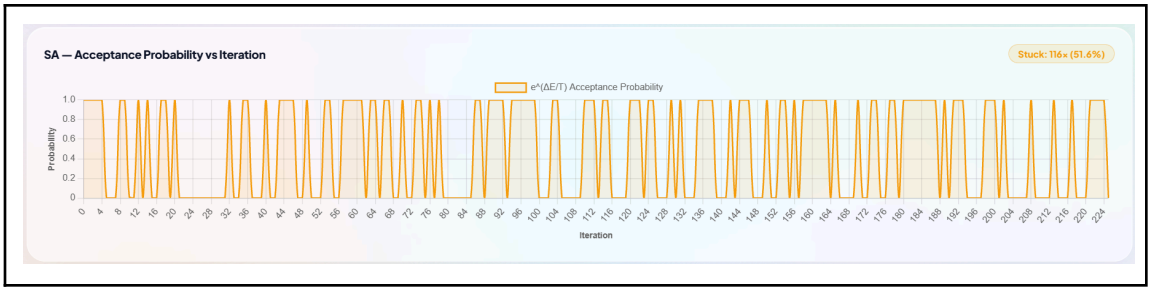
Total Kontainer yang Digunakan	Max Iterations	Initial Temperature	Cooling Rate (Alpha)	Initial Objective Score
4 (V1, V2, V4, V5)	500	1000	0,95	$16,750
Final State				



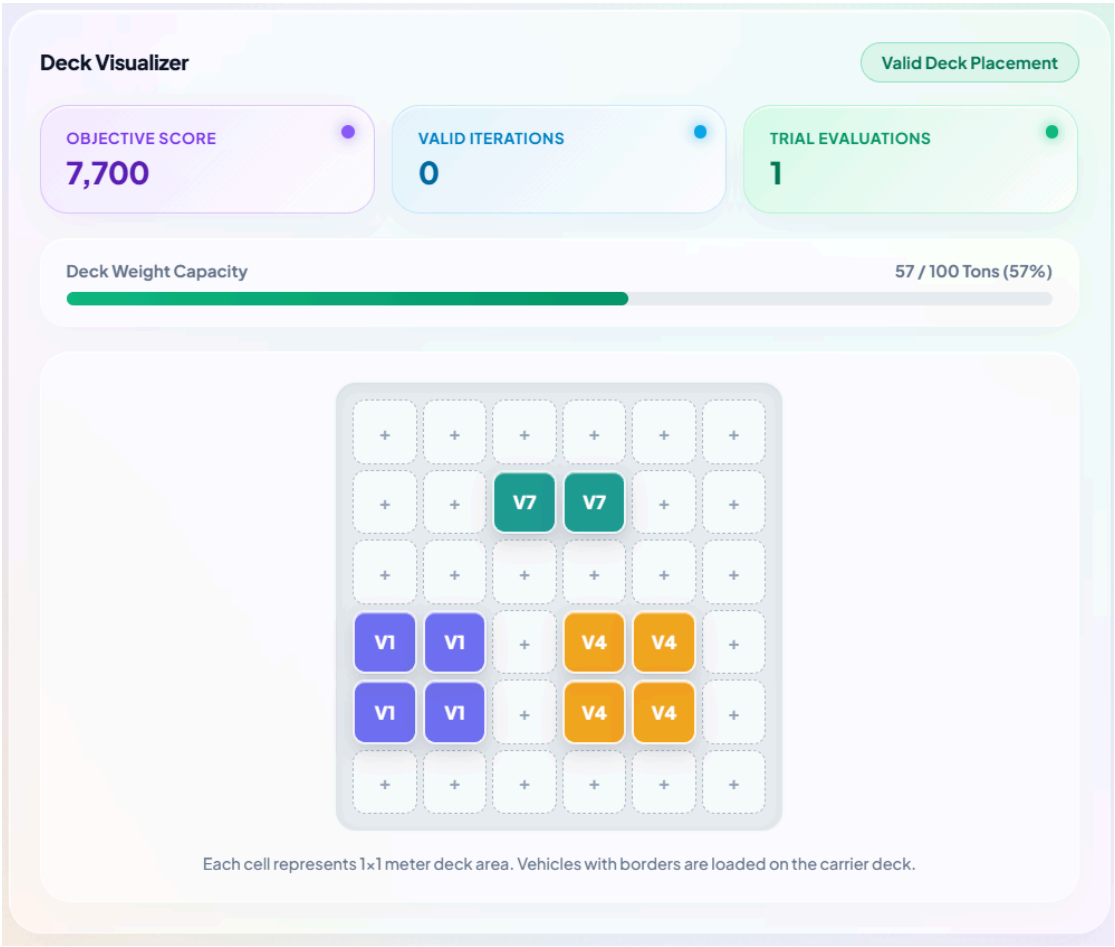
Total Kontainer yang Digunakan	Final Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
4 (V1, V2, V4, V5)	$16,750	0.0156s	116 (51.6%)	109

Plot Objective Function over Iterations

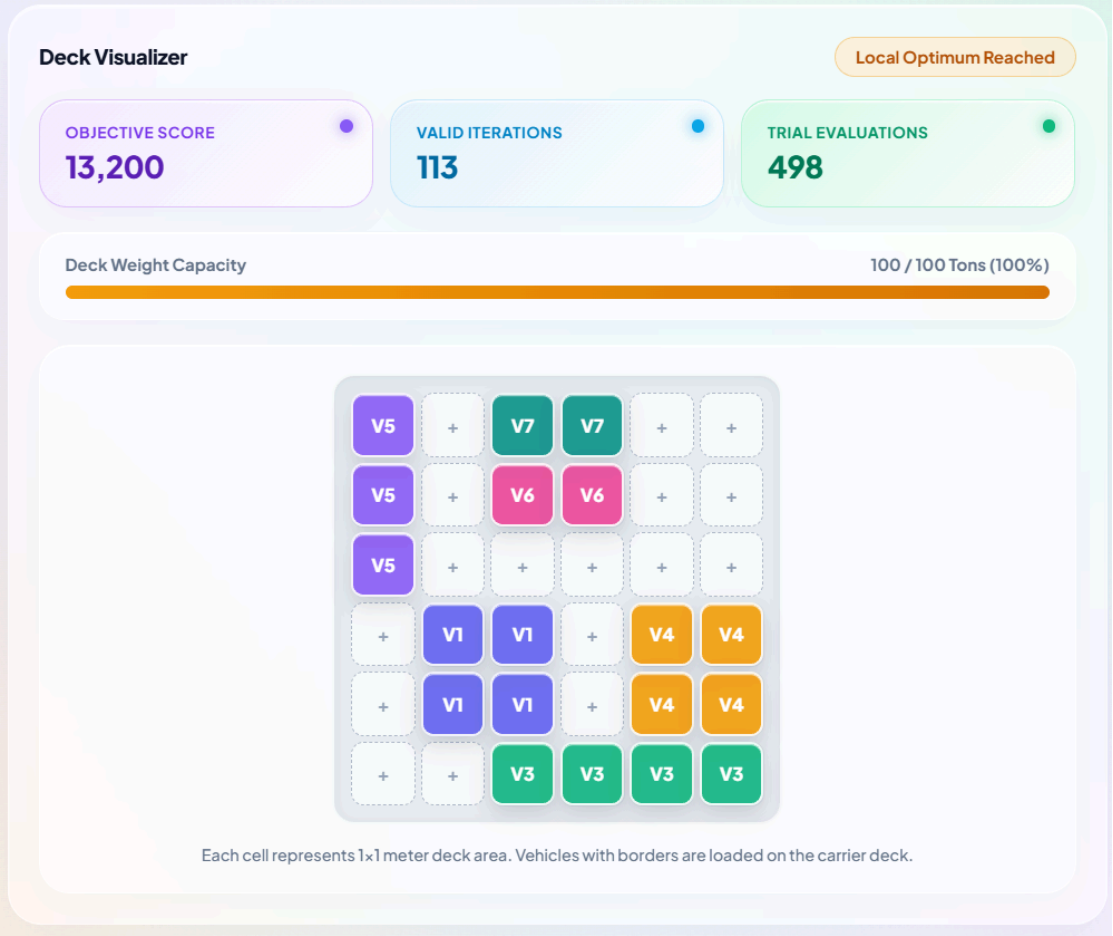
Plot probability $e^{\Delta E/T}$ over Iterations



Tabel 24. Hasil ETA *Priority* dengan *Simulated Annealing* 3

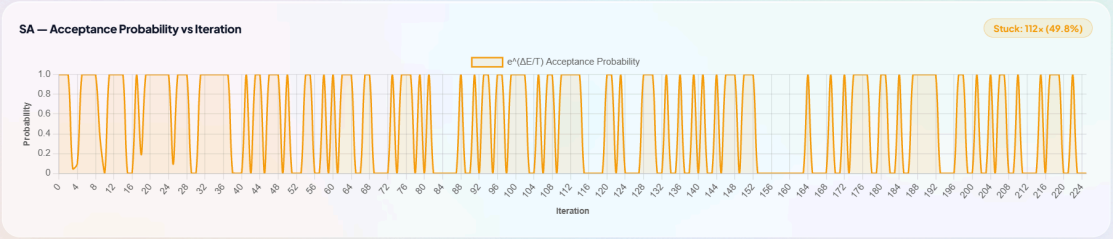
Iterasi 3				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Temperature**	**Cooling Rate (Alpha)**	**Initial Objective Score**
3 (V1, V4, V7)	500	1000	0,95	$7,700

Final State

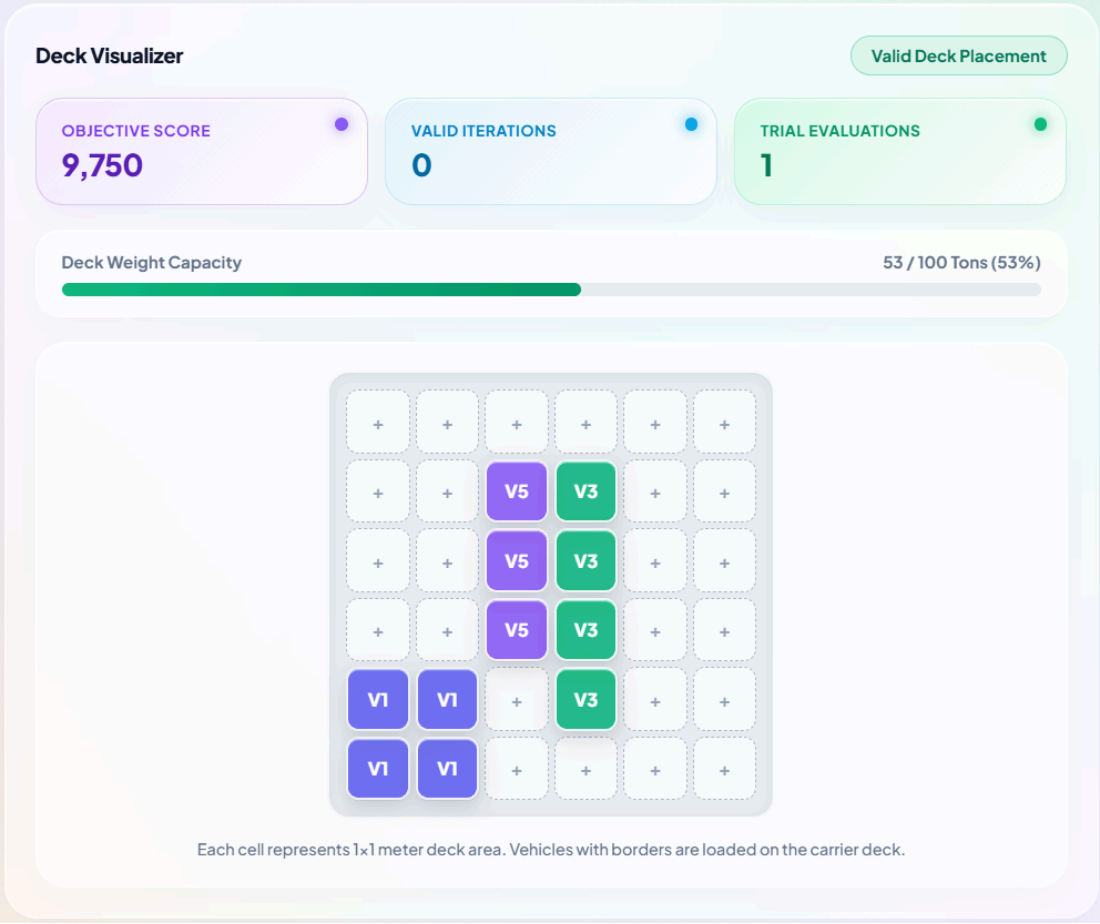


Total Kontainer yang Digunakan	Final Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
6 (V1, V3, V4, V5, V6, V7)	$13,200	0.0139s	112 (49.8%)	113

Plot Objective Function over Iterations

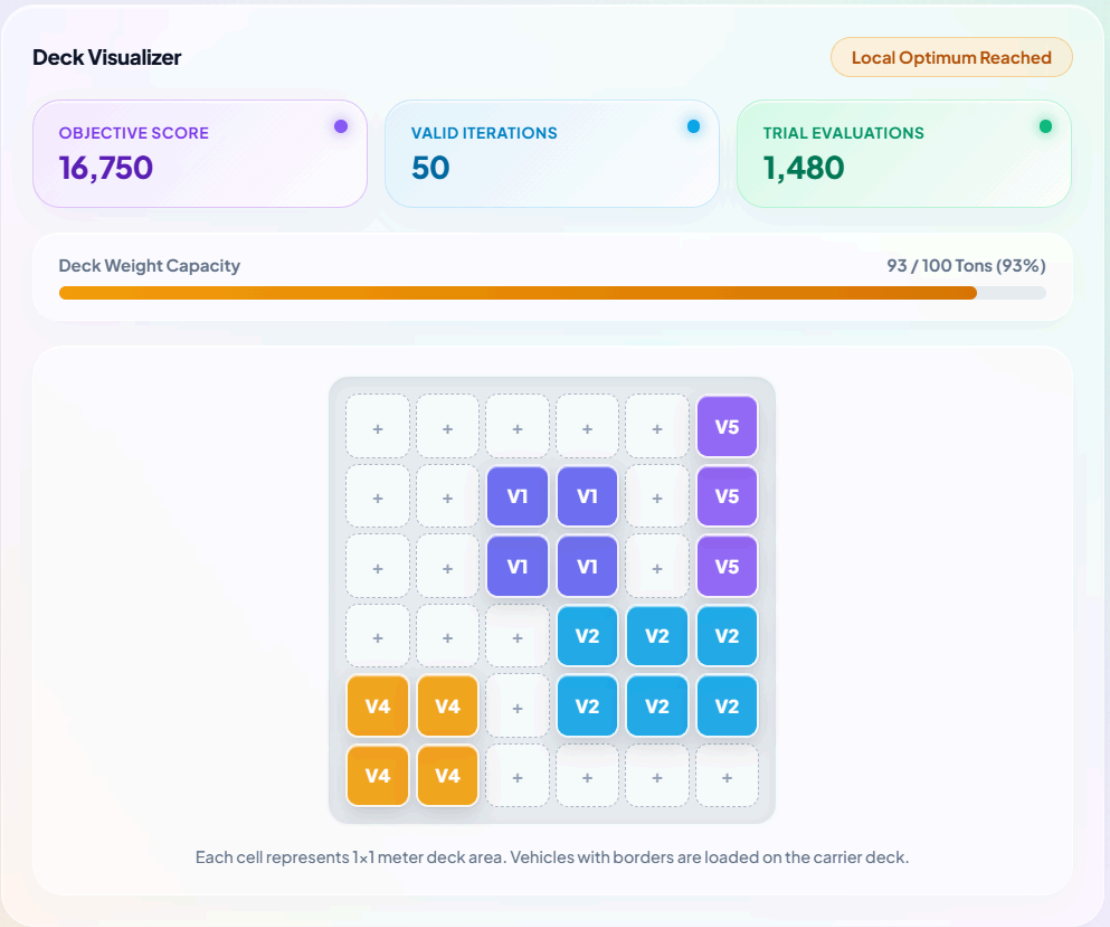
Plot probability $e^{\Delta E/T}$ over Iterations


Tabel 25. Hasil ETA *Priority* dengan *Genetic Algorithm 1*

Iterasi 1				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Population Size**	**Mutation Rate**	**Initial Objective Score**

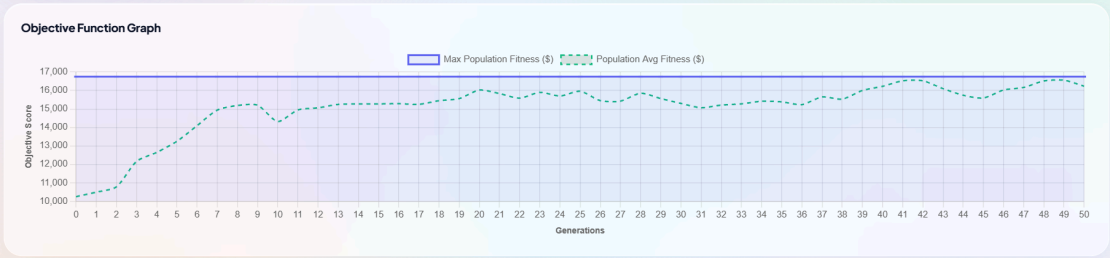
3 (V1, V3, V5)	50	30	0,15	$9,750

Final State



Total Kontainer yang Digunakan	Execution Time	Final Objective Score
4 (V1, V2, V4, V5)	0.0735s	$16,750

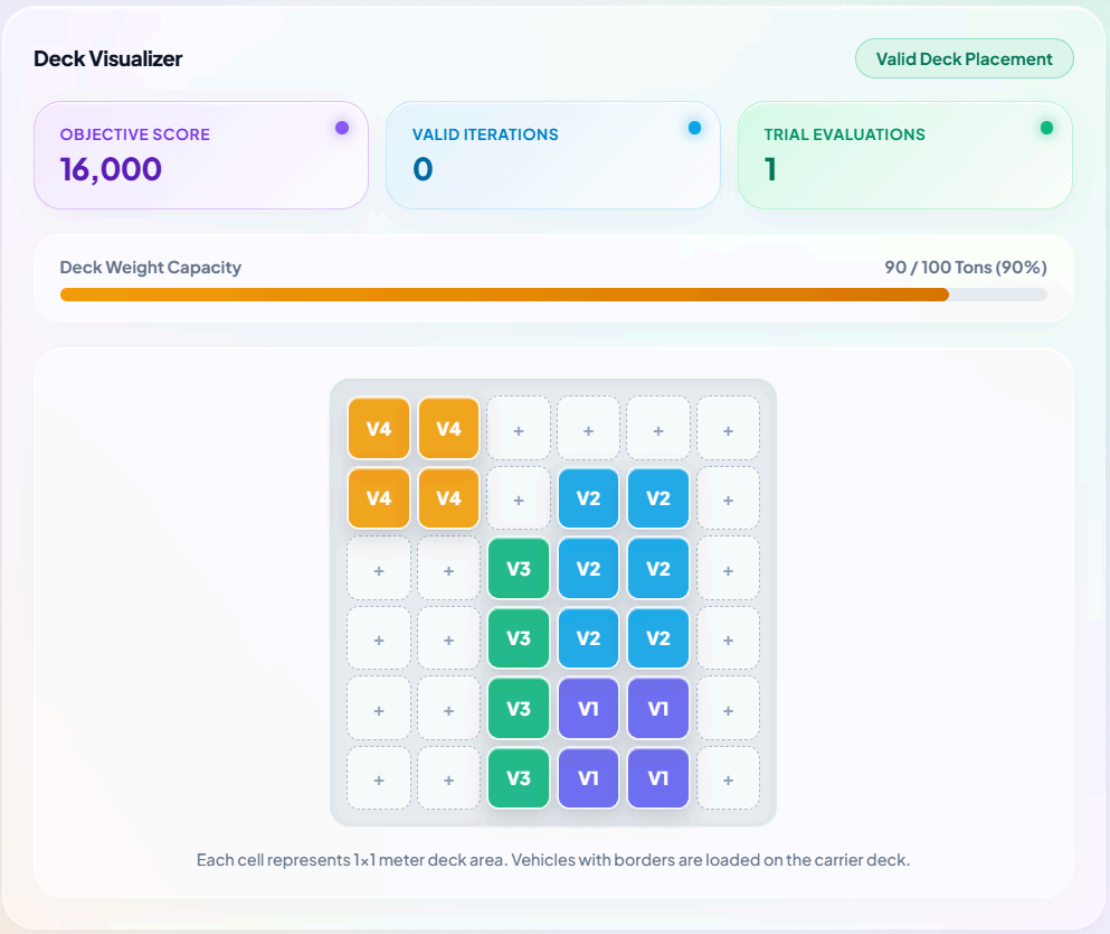
Final Plot Objective Function over Iterations



Tabel 26. Hasil ETA *Priority* dengan *Genetic Algorithm 2*

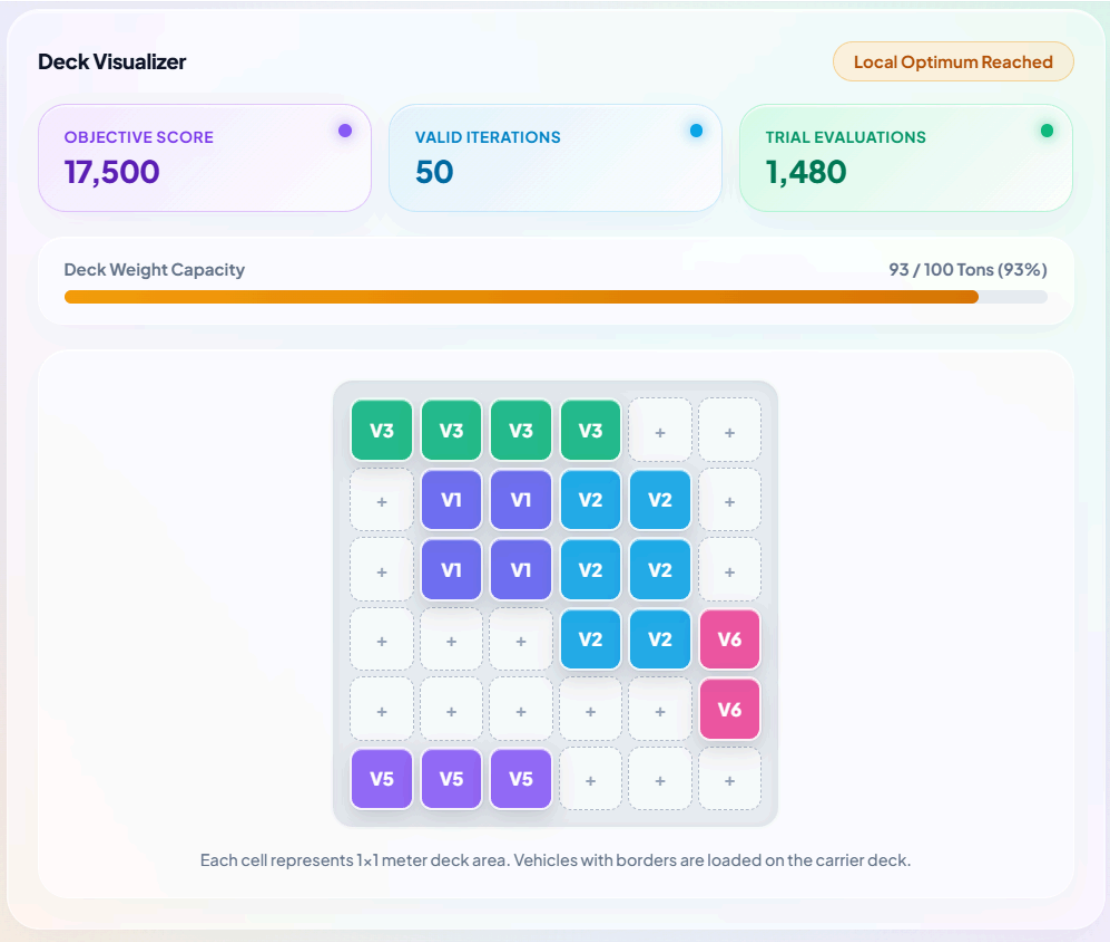
Iterasi 2

Initial State

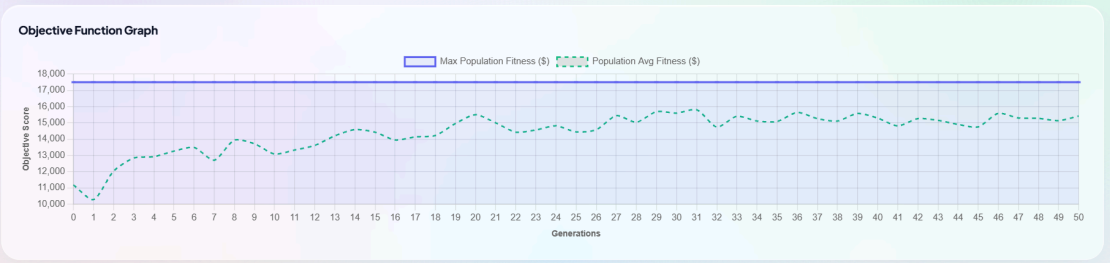


Total Kontainer yang Digunakan	Max Iterations	Population Size	Mutation Rate	Initial Objective Score
4 (V1, V2, V3, V4)	50	30	0,15	$16,000

Final State



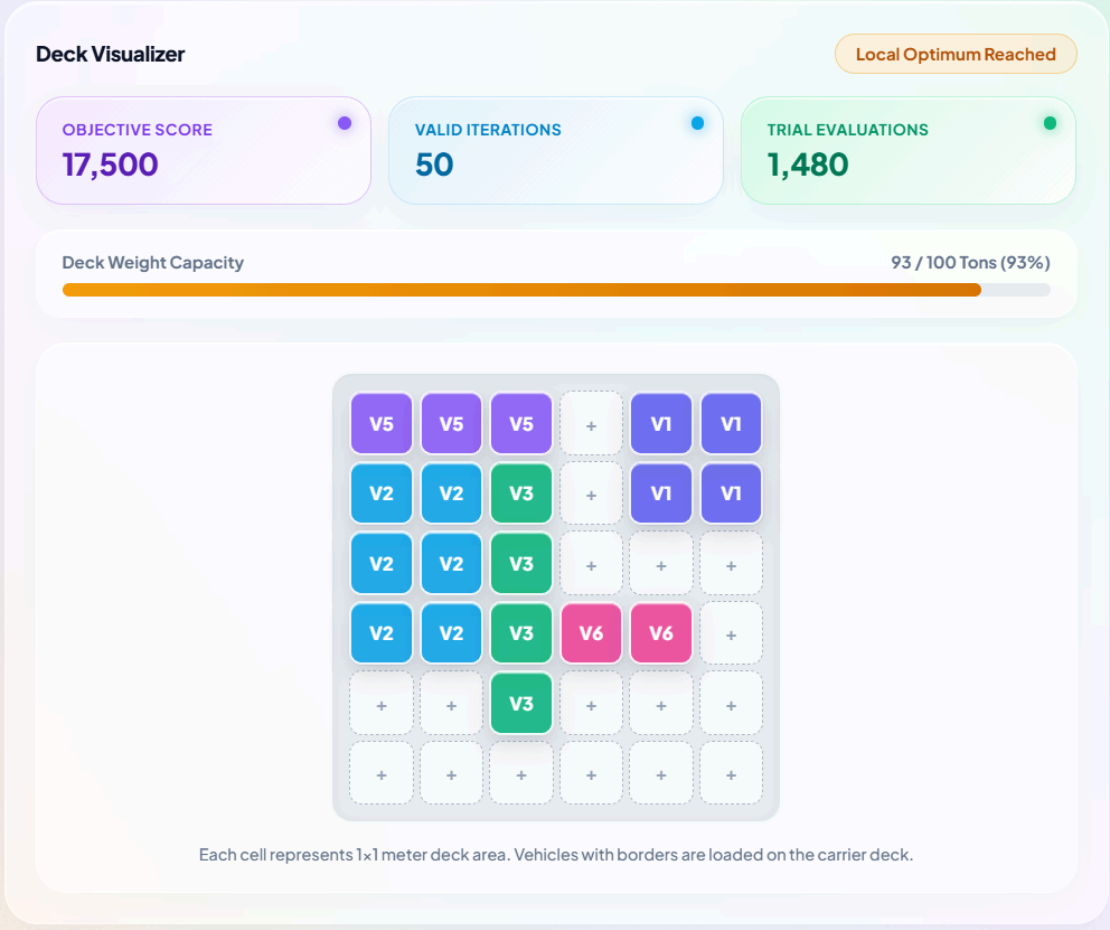
Total Kontainer yang Digunakan	Execution Time	Final Objective Score
5 (V1, V2, V3, V5, V6)	0.0616s	$17,500
Final Plot Objective Function over Iterations		



Tabel 27. Hasil ETA *Priority* dengan *Genetic Algorithm 3*

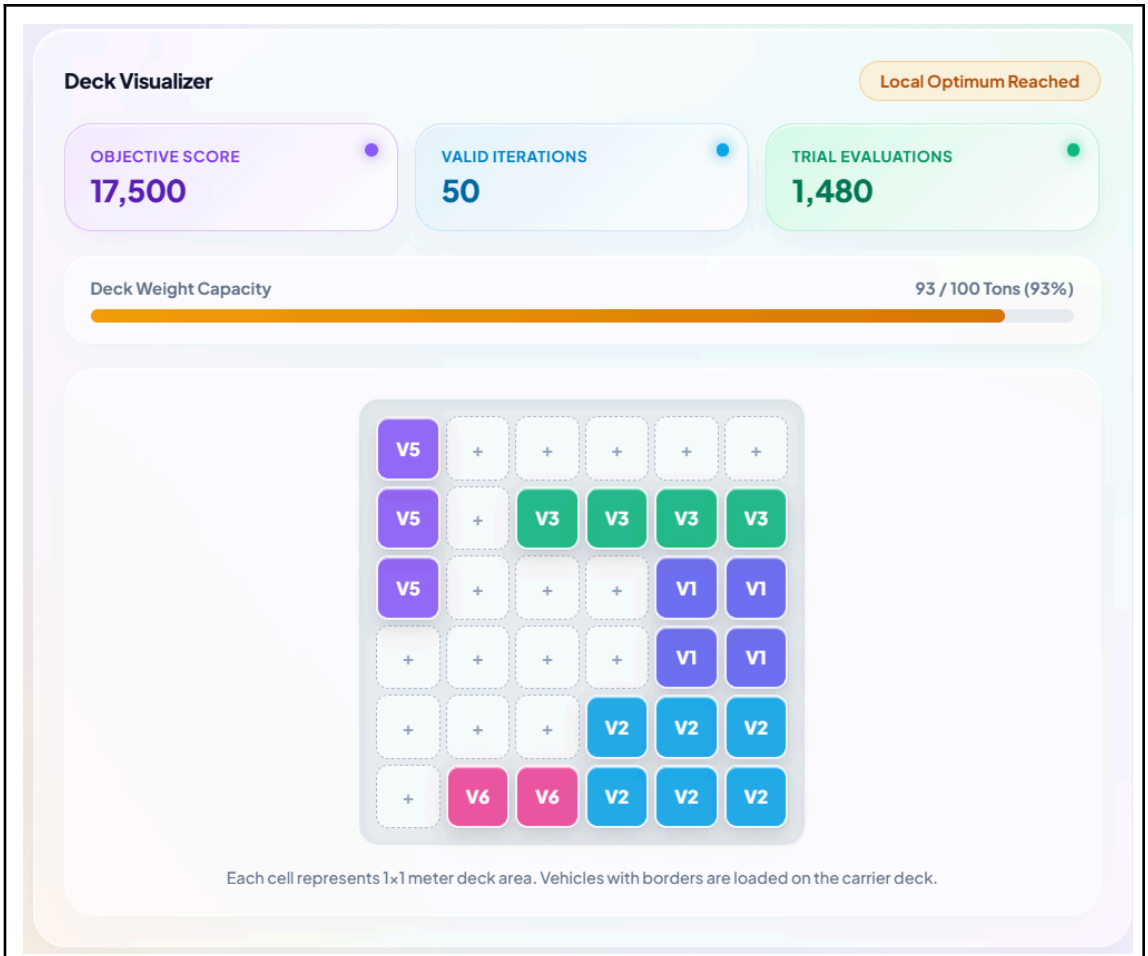
Iterasi 3

Initial State



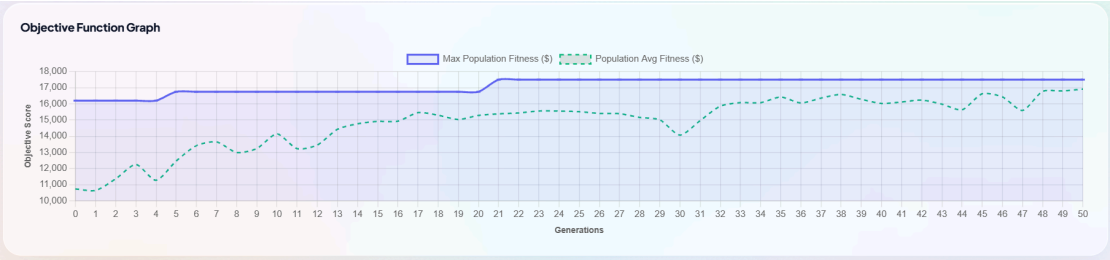
Total Kontainer yang Digunakan	Max Iterations	Population Size	Mutation Rate	Initial Objective Score
5 (V1, V2, V3, V5, V6)	50	30	0,15	$17,500

Final State



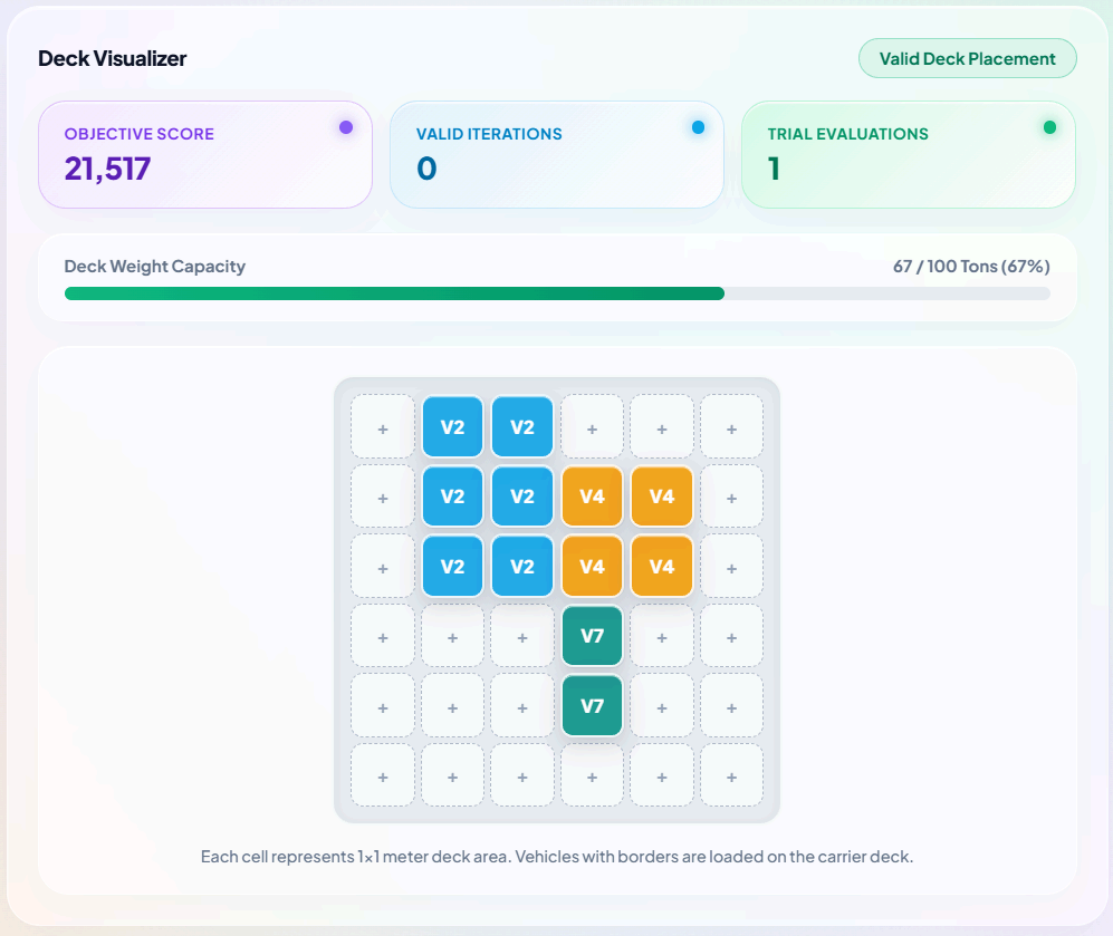
Total Kontainer yang Digunakan	Execution Time	Final Objective Score
5 (V1, V2, V3, V5, V6)	0.0643s	$17,500

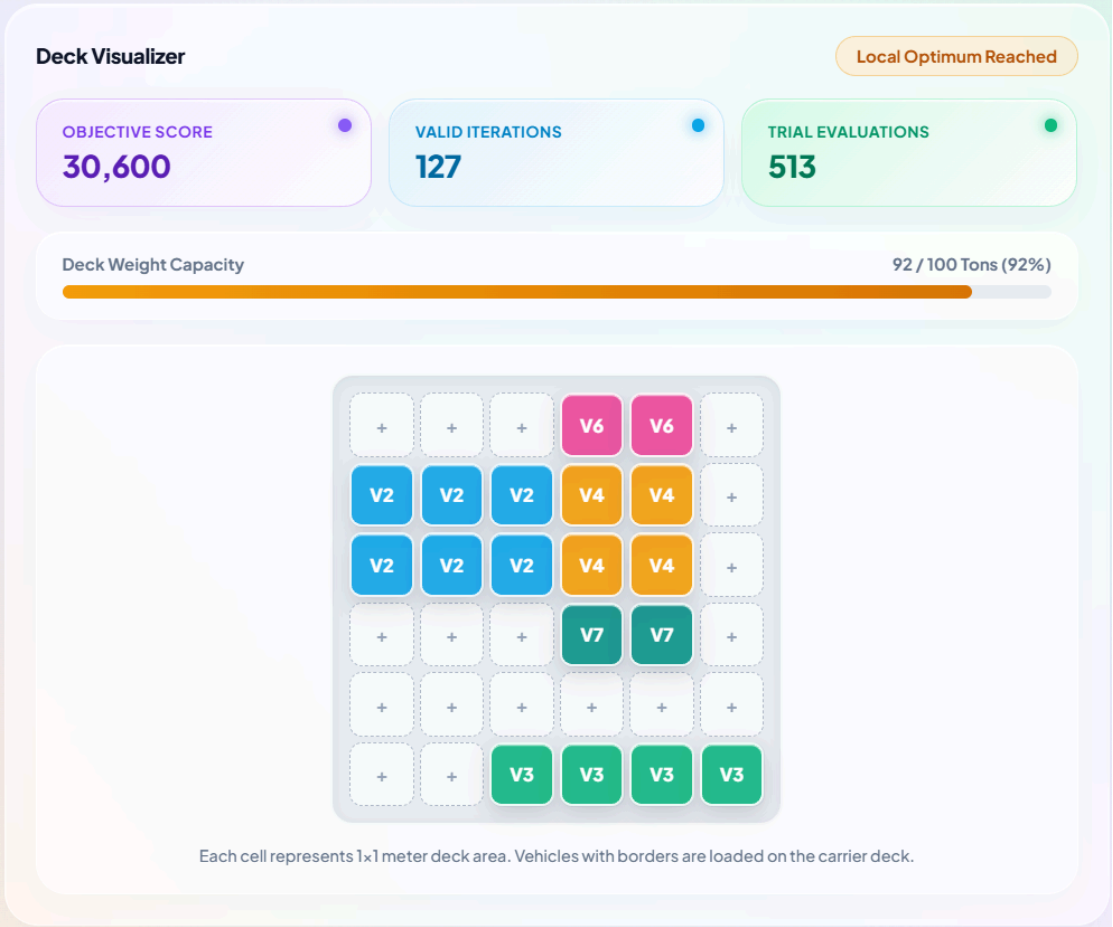
Final Plot Objective Function over Iterations



Eksperimen 2: *Capacity Utilization (Capacity Efficiency)*

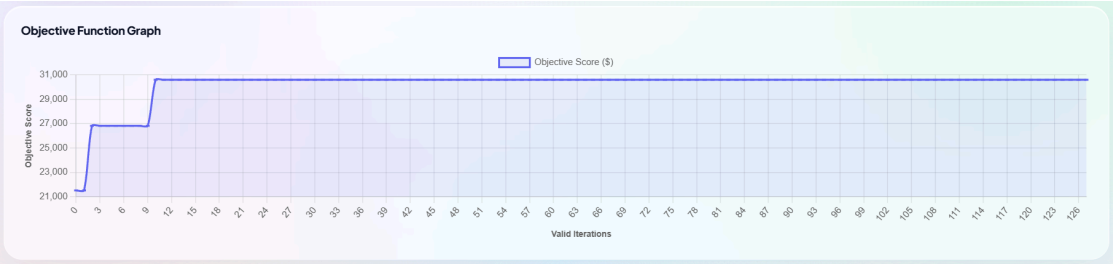
Tabel 28. Hasil *Capacity Efficiency* dengan *Simulated Annealing* 1

Iterasi 1				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Temperature**	**Cooling Rate (Alpha)**	**Initial Objective Score**
3 (V2, V4, V7)	500	1000	0,95	$21,517
Final State				

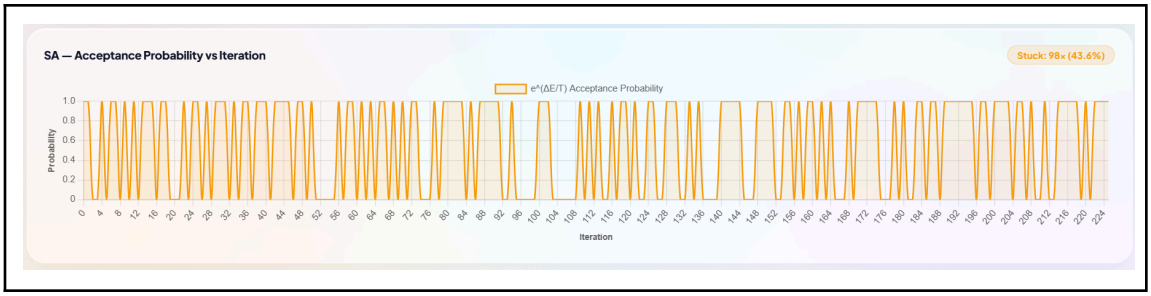


Total Kontainer yang Digunakan	Final Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
5 (V2, V3, V4, V6, V7)	$30,600	0.0134s	98 (43.6%)	127

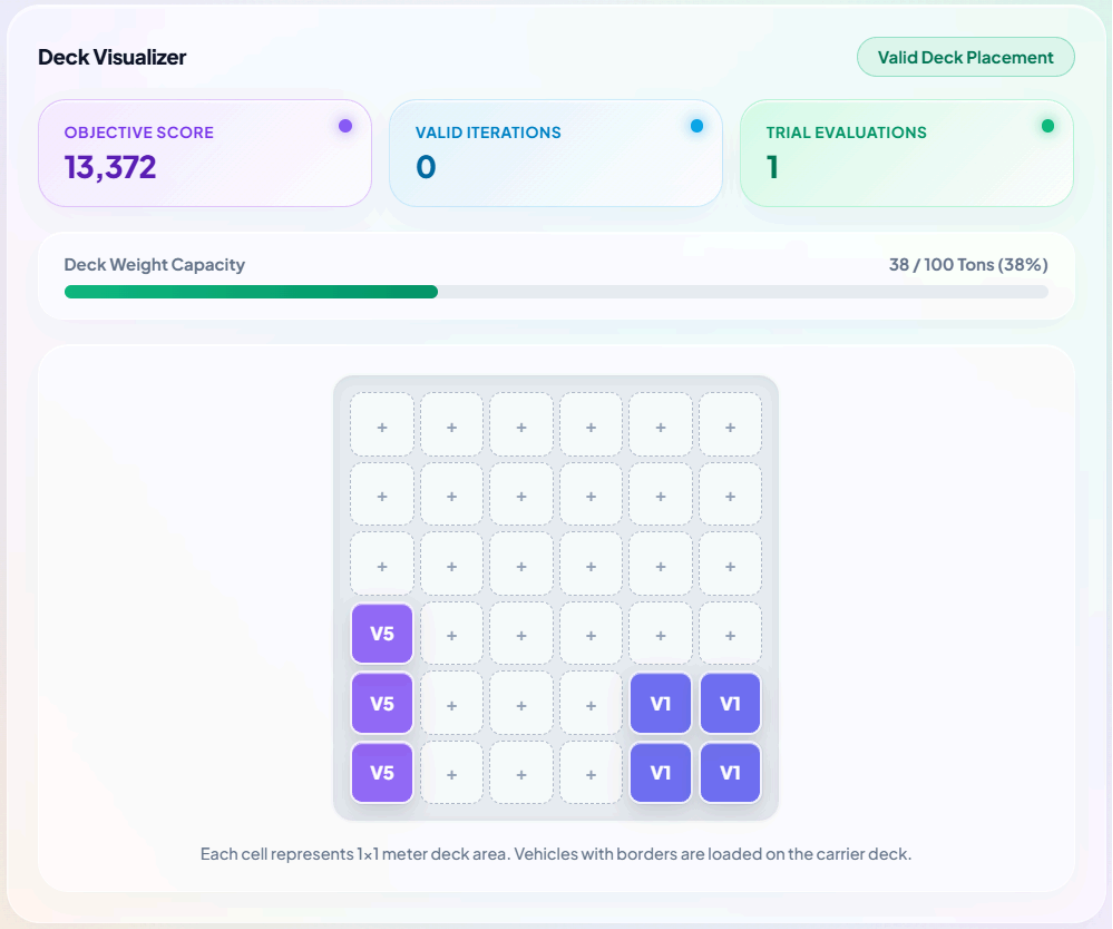
Plot Objective Function over Iterations



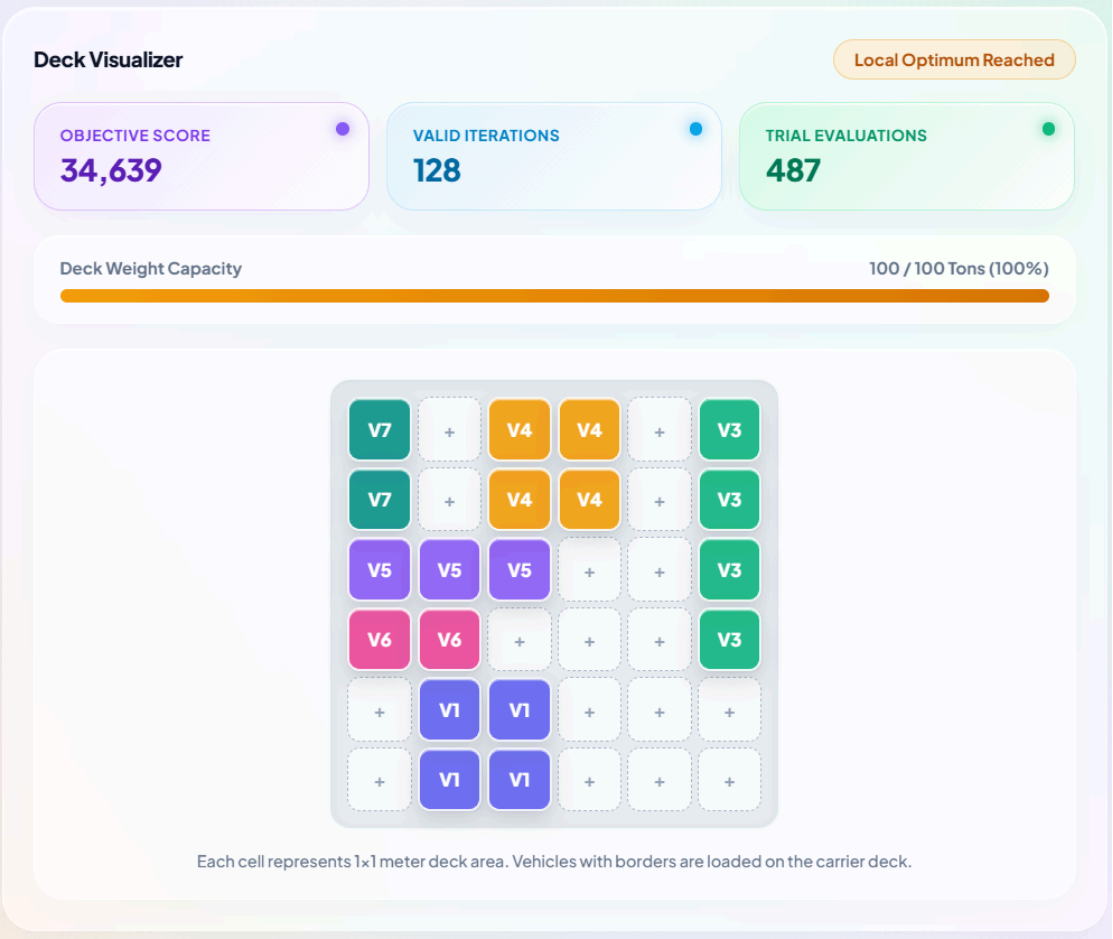
Plot probability $e^{\Delta E/T}$ over Iterations



Tabel 29. Hasil *Capacity Efficiency* dengan *Simulated Annealing* 2

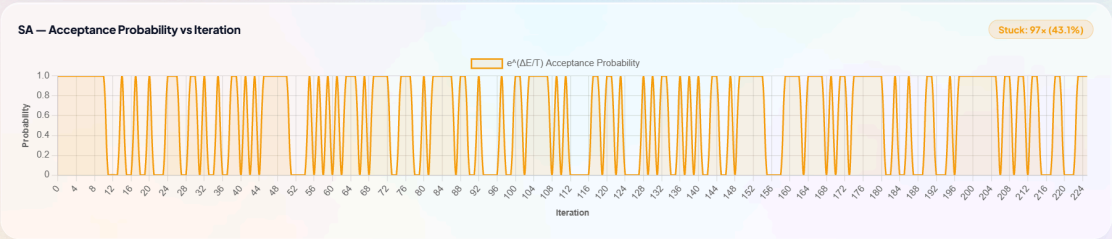
Iterasi 2				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Temperature**	**Cooling Rate (Alpha)**	**Initial Objective Score**
2 (V1, V5)	500	1000	0,95	$13,372

Final State

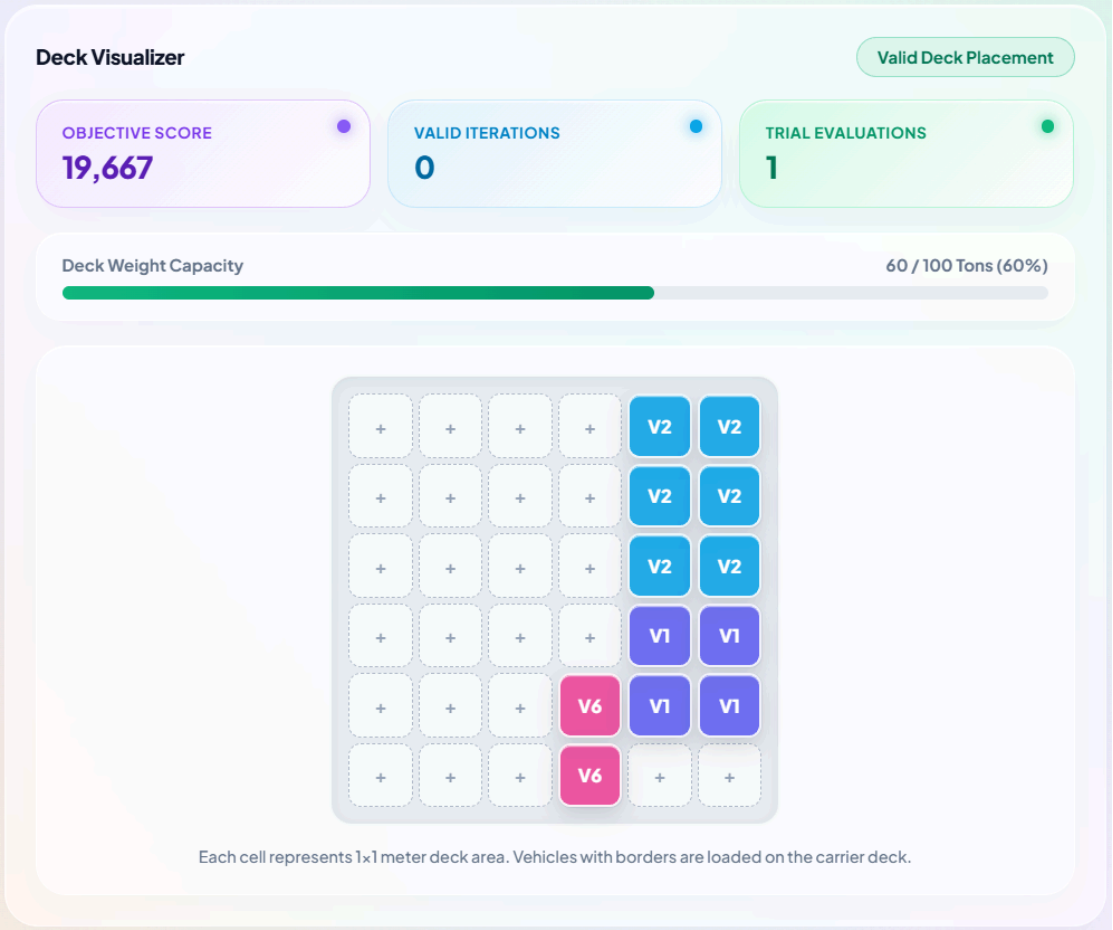


Total Kontainer yang Digunakan	Final Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
6 (V1, V3, V4, V5, V6, V7)	$34,639	0.0142s	97 (43.1%)	128

Plot Objective Function over Iterations

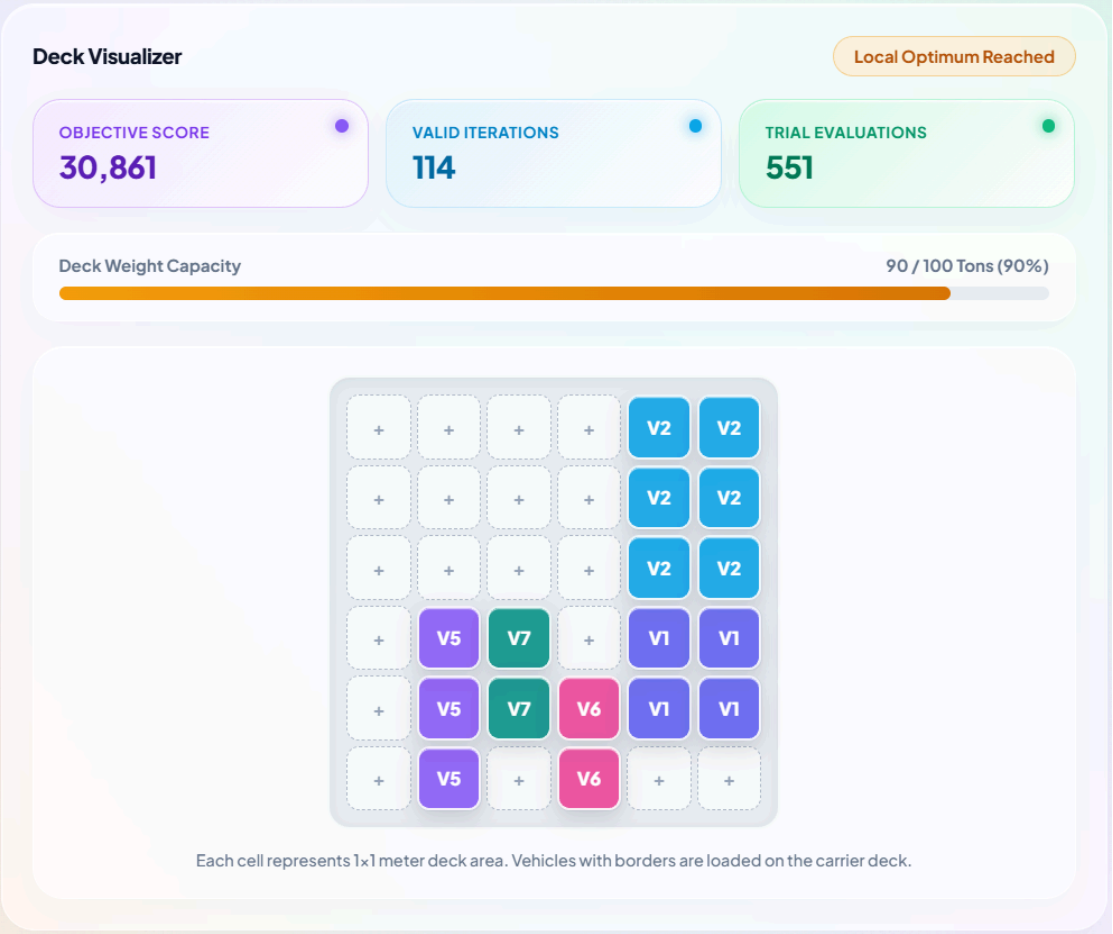
Plot probability $e^{\Delta E/T}$ over Iterations


Tabel 30. Hasil *Capacity Efficiency* dengan *Simulated Annealing* 3

Iterasi 3				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Initial Temperature**	**Cooling Rate (Alpha)**	**Initial Objective Score**

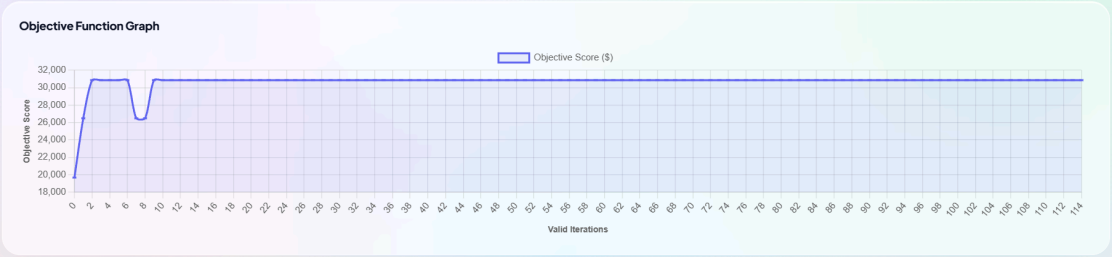
3 (V1, V2, V6)	500	1000	0,95	$19,667

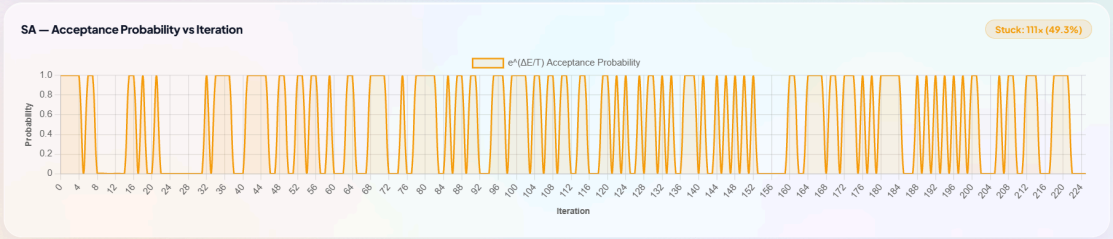
Final State



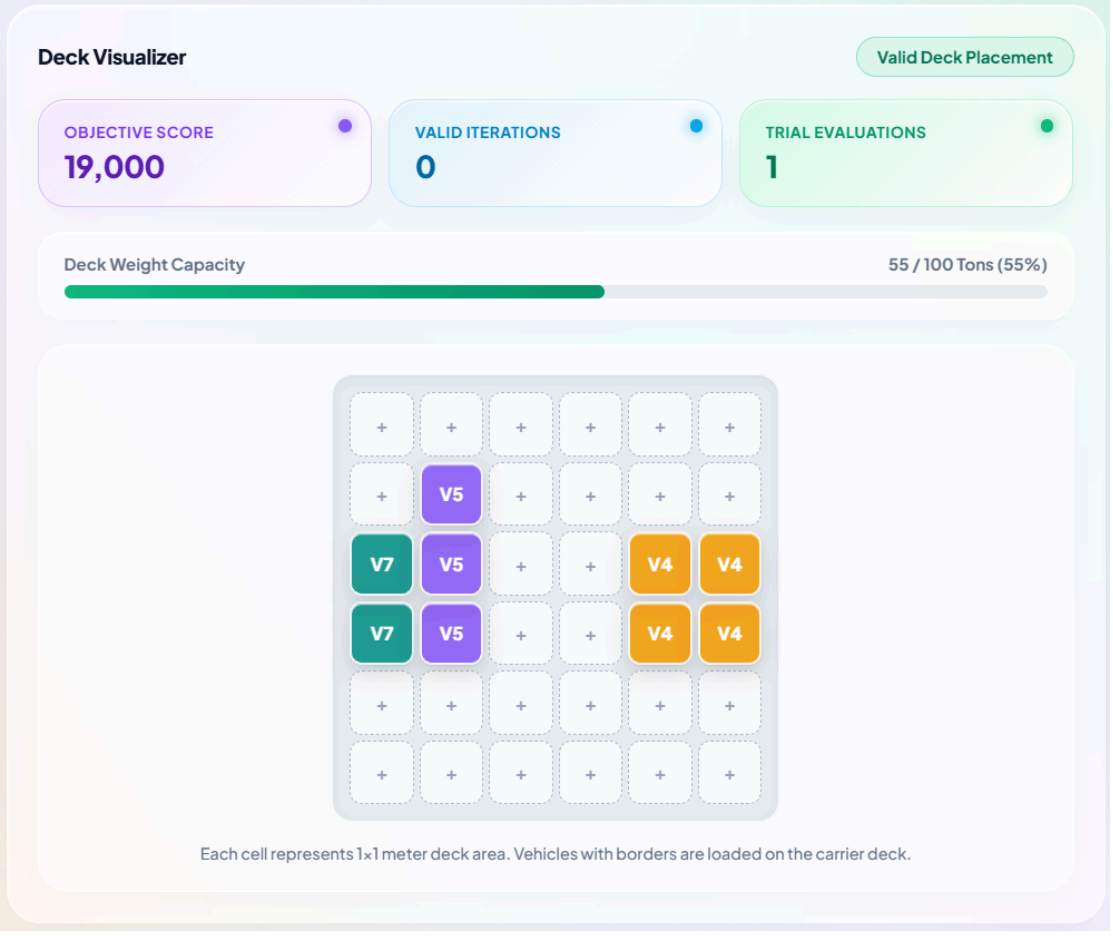
Total Kontainer yang Digunakan	Final Objective Score	Execution Time	Frekuensi Stuck	Valid Iterations
5 (V1, V2, V5, V6, V7)	$30,861	0.0171s	111 (49.3%)	114

Plot Objective Function over Iterations



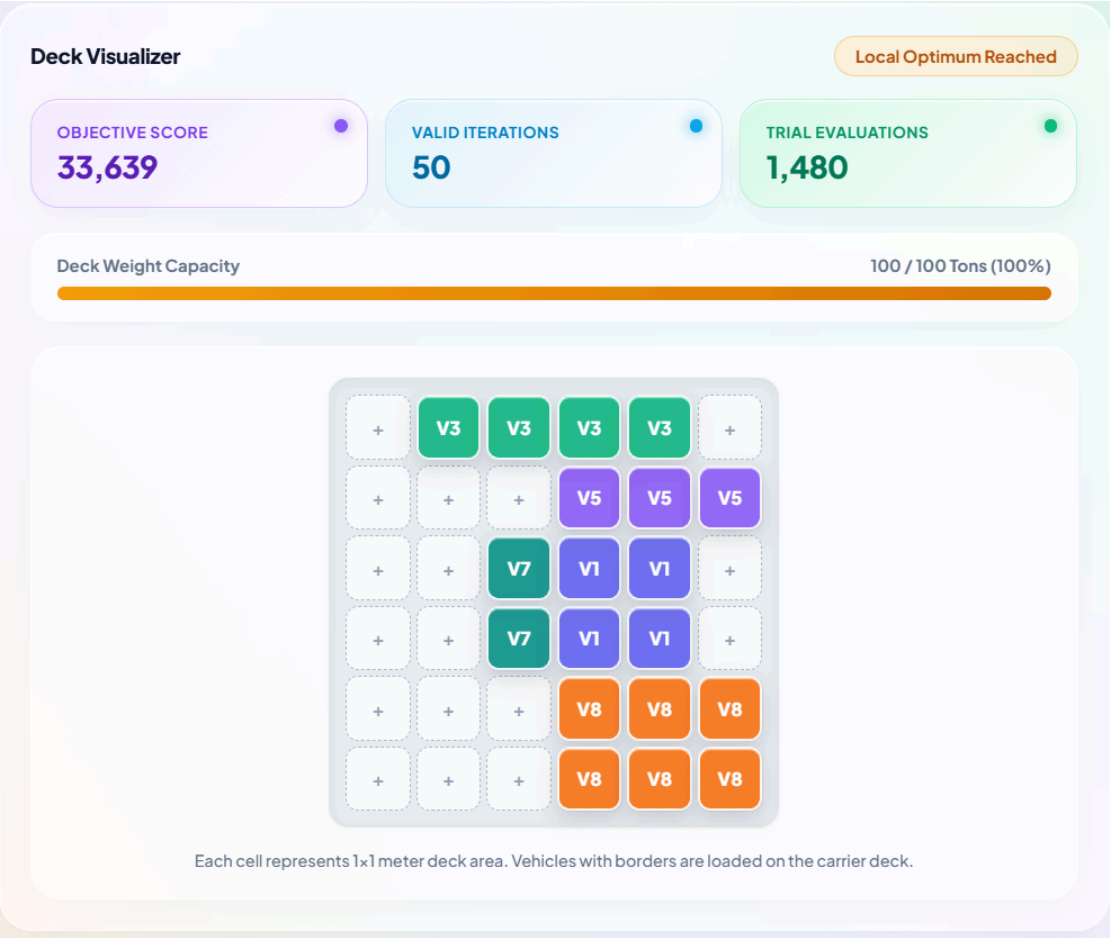
Plot probability $e^{\Delta E/T}$ over Iterations


Tabel 31. Hasil *Capacity Efficiency* dengan *Genetic Algorithm 1*

Iterasi 1				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Population Size**	**Mutation Rate**	**Initial Objective**

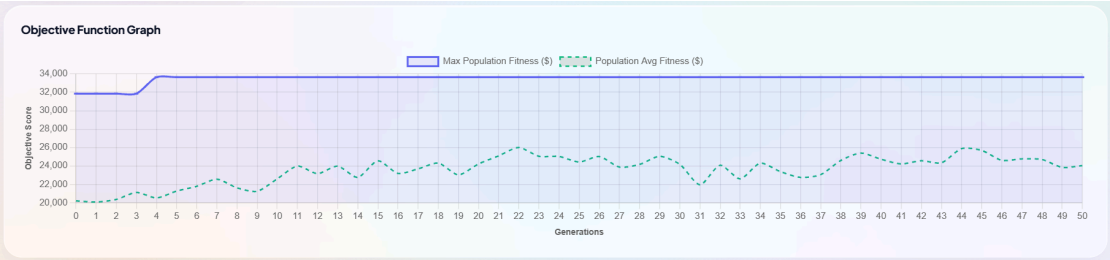
				Score
3 (V4, V5, V7)	50	30	0,15	$19,000

Final State

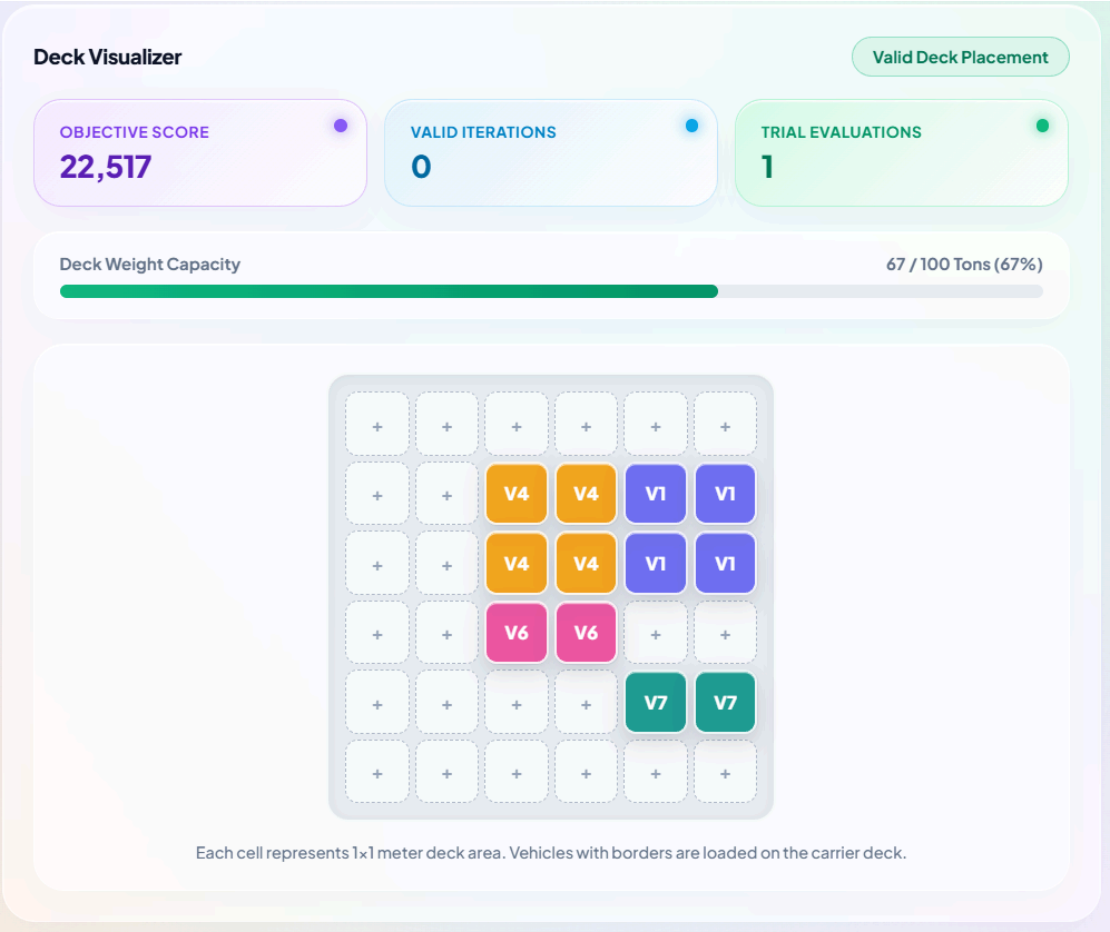


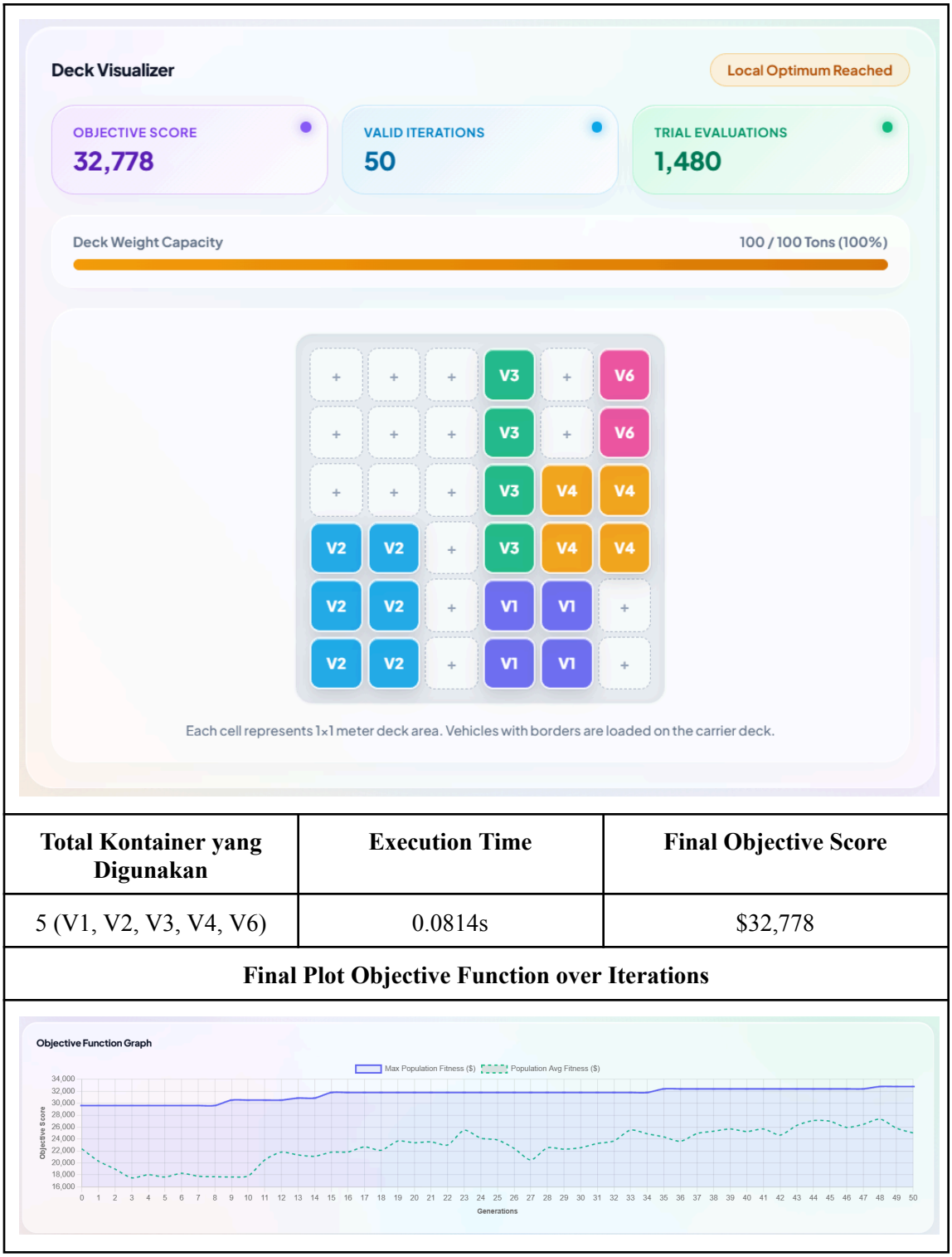
Total Kontainer yang Digunakan	Execution Time	Final Objective Score
5 (V1, V3, V5, V7, V8)	0.0666s	$33,639

Final Plot Objective Function over Iterations



Tabel 32. Hasil *Capacity Efficiency* dengan *Genetic Algorithm 2*

Iterasi 2				
Initial State				
				
Total Kontainer yang Digunakan	**Max Iterations**	**Population Size**	**Mutation Rate**	**Initial Objective Score**
4 (V1, V4, V6, V7)	50	30	0,15	$22,517
Final State				

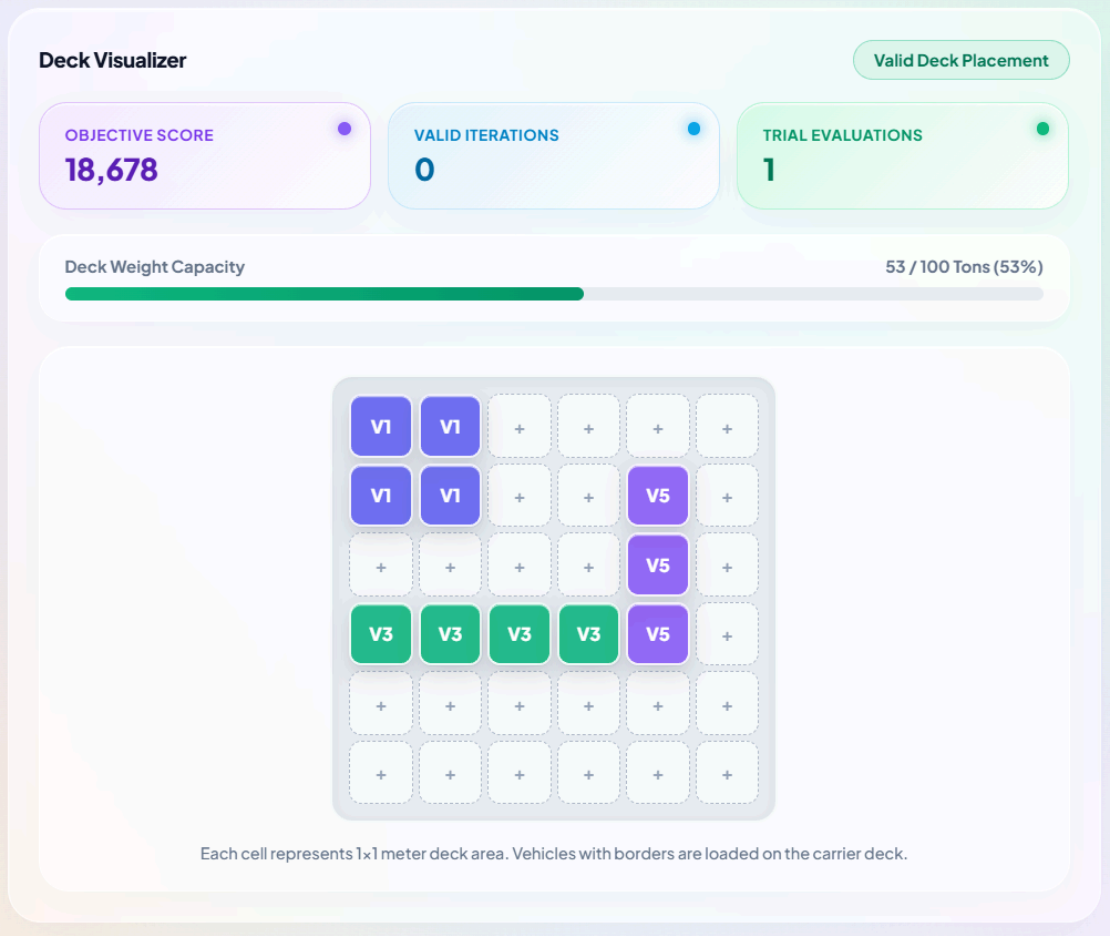


Total Kontainer yang Digunakan	Execution Time	Final Objective Score
5 (V1, V2, V3, V4, V6)	0.0814s	$32,778
Final Plot Objective Function over Iterations		

Tabel 33. Hasil *Capacity Efficiency* dengan *Genetic Algorithm 3*

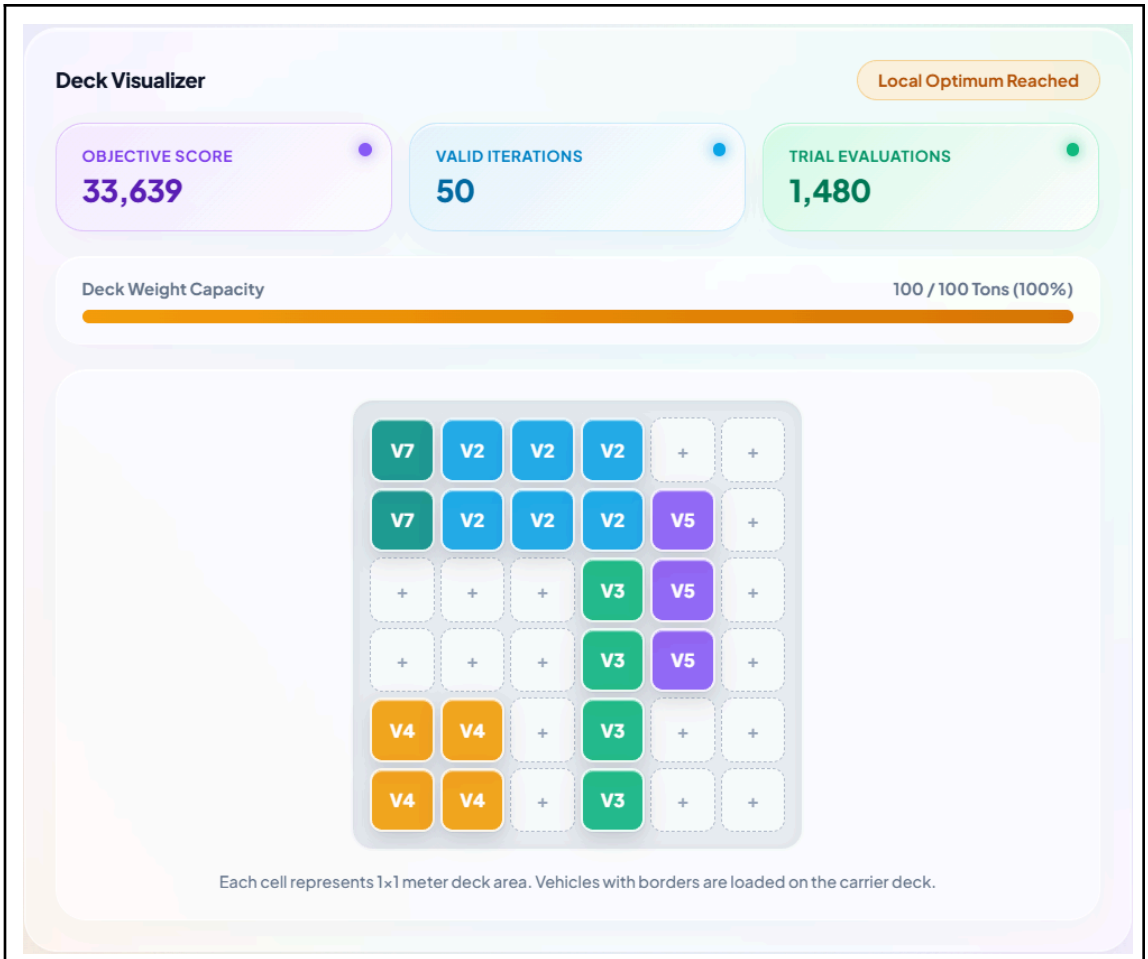
Iterasi 3

Initial State



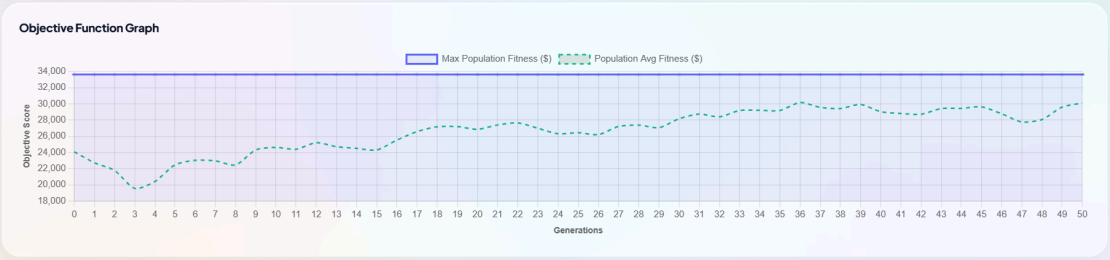
Total Kontainer yang Digunakan	Max Iterations	Population Size	Mutation Rate	Initial Objective Score
3 (V1, V3, V5)	50	30	0,15	$18,678

Final State

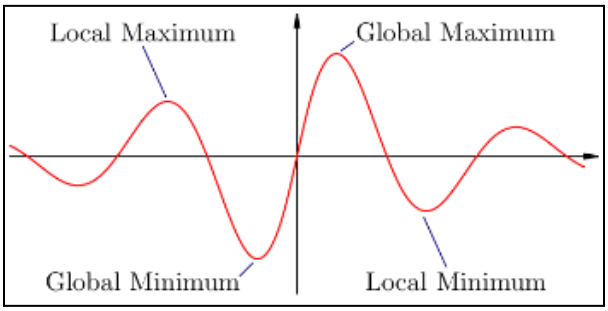


Total Kontainer yang Digunakan	Execution Time	Final Objective Score
5 (V2, V3, V4, V5, V7)	0.0781s	$33,639

Final Plot Objective Function over Iterations



2.6.6 Analisis Hasil Algoritma dengan Global Optima



Gambar 1. Global Optima

Penggunaan algoritma *local search*, seperti *Steepest Ascent Hill-Climbing, Hill-Climbing with Sideways Move, Random Restart Hill-Climbing, Stochastic Hill-Climbing, Simulated Annealing, dan Genetic Algorithm,* dalam konteks pendekatannya terhadap *global optima* menunjukkan hasil yang cukup berbeda. Pada *Steepest Ascent Hill-Climbing* dan *Hill-Climbing with Sideways Move*, suatu hasil yang mengindikasikan *local optima* dicapai dengan cukup cepat. Hal ini dikarenakan kedua algoritma ini mencari seluruh kemungkinan *neighbor* dari *current state*. Namun, hal ini juga bisa jadi suatu kelemahan karena hasil algoritma kerap kali *stuck* di *local optima*, apalagi tanpa adanya kondisi terminasi yang bisa membantu mencari *state* lain yang lebih memungkinkan untuk meraih *global optima*. Apalagi pada *Hill-Climbing with Sideways Move*, ada kemungkinan *limit sideways move* terlalu kecil sehingga *shoulder* belum ter-*sweep* sehingga tidak bisa meraih kondisi yang lebih optimum.

Kelemahan ini membuat *Random Restart Hill-Climbing* lebih unggul dalam mencari *global optima* karena kemampuannya dalam mencari suatu *random state* baru yang bisa menjadi *successor* dalam meraih solusi yang lebih dekat dengan *global optima*. Meski begitu, apabila *limit restart* dari algoritma ini terlalu kecil nilainya, ada kemungkinan seluruh *moves* yang dibuat oleh algoritma ini *bad* sehingga nilai objektif pun tidak semakin dekat ke *global optima*. Sedang apabila angka *limit restart*-nya besar, kemungkinan dilakukannya *good move* pun lebih besar. *Stochastic Hill-Climbing* memilih tetangga acak di tiap iterasi, membuat algoritma ini lebih fleksibel dalam melakukan eksplorasi. Namun, apabila *state* awal kurang bagus, tetangga yang mungkin di-*generate* oleh algoritma ini pun bisa jadi kurang baik sehingga hasil pun tidak bisa

mendekati *global optima*. Apalagi, jika angka *limit* untuk *random neighbor searching*-nya terlalu kecil, *neighbor* yang bisa dijangkau pun tidak bisa terlalu banyak sehingga kemungkinan untuk mendekati *global optima* juga semakin kecil.

Sedangkan pada algoritma *Simulated Annealing*, kekurangan yang dimiliki *Hill-Climbing algorithm* terkait keterjebakannya di *local optima* bisa teratasi. Hal ini bisa teratasi karena kemampuannya untuk mengambil tetangga yang lebih buruk, selama nilai probabilitasnya masih di atas *threshold*, yang memungkinkan algoritma ini untuk mencari tetangga yang bisa jadi *turning point* untuk meraih *global optimum*. Namun, terdapat banyak faktor yang bisa memengaruhi algoritma ini. Ada kemungkinan hasil akhir yang diberikan algoritma ini malah lebih jelek. Hal ini dikarenakan variabel *temperature* yang selalu berubah tiap dijalankan. Agar ini teratasi, *initial temperature* dari algoritma ini di-*set* pada angka yang tinggi sehingga fleksibilitas dari algoritma ini pun lebih besar, memungkinkannya untuk meraih nilai-nilai objektif yang lebih baik dari *current state*.

Pada algoritma *Genetic Algorithm*, dengan melakukan seleksi populasi hingga *crossover* dan mutasi, dimunculkan populasi baru dengan hasil solusi yang sangat bervariasi. Hal ini bisa jadi *a two-sided sword* karena di satu sisi, ada kemungkinan hasil yang didapat bisa sangat unik dan dekat dengan *global optima*, tetapi tidak menutup kemungkinan bahwa hasil mutasi sangat buruk dan malah jauh dari hasil yang mau diraih. Dengan itu, algoritma-algoritma ini memiliki kelebihan dan kekurangannya masing-masing dalam konteks kedekatannya terhadap *global optima*. Pemilihan algoritma pun diserahkan kepada pengguna, sesuai dengan *drawbacks* apa yang bisa ditolerir.

2.6.7 Analisis Perbandingan Hasil Pencarian Algoritma

Pada pengujian ini setiap algoritma diuji sebanyak tiga kali, lalu *final state*-nya dirata-ratakan. *Objective function* yang digunakan adalah total *fee*, jadi semakin besar nilainya maka hasilnya akan semakin bagus.

Pertama, ***Steepest Ascent Hill-Climbing*** memperoleh hasil paling rendah, rata-rata *objective score* nya $23,167. Dua percobaan berhenti di $24,000 dan percobaan lainnya berhenti di $21,000, jadi percobaan ini tidak pernah mencapai $27,000. Algoritma ini hanya butuh 1-2 valid *iterations* dan berhenti dalam waktu 0.0032 s -

0.0075 s. Proses pencariannya cepat, namun dapat langsung terjebak di kondisi local optimum karena hanya mengambil *neighbor* dengan nilai terbaik dan tidak memiliki cara keluar.

Kedua, ***Sideways Move Hill-Climbing*** rata-rata *objective score*-nya $25,333. Dua percobaan pertama berhenti di $24,500, sedangkan percobaan ketiga mencapai $27,000. Perpindahan ke *neighbor* dengan nilai sama atau lebih baik dari *current state* membantu melewati kondisi *plateau* atau *local optimum*, namun waktu pemrosesannya paling lama, rata-rata waktunya 0.3258 s dengan 101-102 valid *iterations*. Hasilnya bergantung pada *initial state*. Percobaan ketiga dimulai dari $20,000 dan memperoleh *objective score* $27,000, sedangkan percobaan 1 dan 2 dimulai dari $11,000 dan $16,500 dan hanya menghasilkan *objective score* sebesar $24,500.

Ketiga, ***Stochastic Hill-Climbing*** rata-ratanya $26,333 dengan waktu pemrosesan 0,0040 s yang tercepat diantara semuanya. Dua percobaannya bisa mencapai *objective score* $27,000 dengan 6 kontainer (V1, V3, V4, V5, V6, V7), satu percobaannya berhenti di $25,000 dengan 1 valid *iterations*. Pemilihan *neighbor* dilakukan secara acak yang membantunya keluar dari kondisi *local optimum*.

Keempat, ***Random Restart Hill-Climbing*** rata-rata *objective score* tertinggi dengan nilai $26,500. Dua percobaannya mencapai *objective score* $27,000 dan satu percobaannya berhenti di $25,500. Makin tinggi nilai *max restart*, maka jumlah valid *iterations* akan ikut naik dengan waktu 0.0117 s - 0.0212 s. Namun, *restart* yang lebih banyak tidak selalu lebih baik. Dari percobaan, 10 *restart* menghasilkan *objective score* $25,500, sedangkan 5 *restart* menghasilkan $27,000 yang hasilnya lebih baik. Hal ini terjadi karena *initial state* diacak.

Kelima, ***Simulated Annealing*** rata-ratanya $25,000 ($23,500, $27,000, $24,500). Parameternya sama di ketiga percobaan (500 iterasi, suhu awal 1000, alpha 0.95). Namun, hasil yang diperoleh berbeda-beda karena pengaruh faktor acak yang besar. Frekuensi *stuck*-nya tinggi, rata-rata sekitar 122 kali (45.8% - 62.7% dari terasi), artinya lebih dari separuh iterasi tidak menghasilkan perpindahan *state*. *Stuck* 62,7% menghasilkan *objective score* $24,500, sedangkan *stuck* 53,8% menghasilkan *objective score* $27,000. Waktu pemrosesan algoritma tetap cepat, yaitu 0.015 s - 0.023 s.

Keenam, ***Genetic Algorithm*** rata-ratanya $25,389 dari enam konfigurasi (tiap konfigurasi 3 *run*) dan dua kali memperoleh *objective score* sebesar $27,000. Pada eksperimen pertama (populasi 20), rata-rata untuk 20, 50, dan 100 iterasi adalah $25,167, $25,667, dan $25,167. Pada eksperimen ke dua (100 iterasi), tiga ukuran populasi menghasilkan $25,500, $25,333, dan $25,500. Iterasi dan populasi yang lebih besar tidak membantu memperbaiki hasil, sementara waktunya naik sampai 0.27 s pada populasi 60. Jadi parameter GA yang lebih besar tidak sebanding dengan tambahan waktunya.

Selisih kualitas antar algoritma tidak besar, hanya sekitar $3,300 antara rata-rata terendah dan tertinggi, dan hampir semua algoritma pernah mencapai *objective score* $27,000. Yang berbeda adalah waktu dan konsistensinya. *Steepest Ascent* paling mudah terjebak di kondisi *local optimum*, sedangkan algoritma dengan unsur acak atau *restart* (*Stochastic Hill-Climbing, Random Restart Hill-Climbing, Sideways Move Hill-Climbing)* hasilnya lebih baik. SA dan GA tidak otomatis hasilnya lebih baik, kemungkinan karena ruang solusi yang kecil (V1-V8). Kombinasi V1, V3, V4, V6, V7 dengan *fee* $27,000 muncul di beberapa algoritma, jadi kemungkinan besar itu adalah nilai optimum globalnya.

Urutan berdasarkan kualitas hasil (rata-rata *fee*):

1. *Random Restart Hill-Climbing:* $26,500
2. *Stochastic Hill-Climbing:* $26,333
3. *Genetic Algorithm:* $25,389
4. *Sideways Move Hill-Climbing:* $25,333
5. *Simulated Annealing:* $25,000
6. *Steepest Ascent Hill-Climbing:* $23,167

Urutan berdasarkan waktu eksekusi (tercepat ke terlama):

1. *Stochastic Hill-Climbing:* 0.0040 s
2. *Steepest Ascent Hill-Climbing:* 0.0054 s
3. *Random Restart Hill-Climbing:* 0.0178 s
4. *Simulated Annealing:* 0.0185 s
5. *Genetic Algorithm:* 0.0868 s
6. *Sideways Move Hill-Climbing:* 0.3258 s

Untuk kasus ini, *Random Restart Hill-Climbing* dan *Stochastic Hill-Climbing* paling seimbang antara kualitas dan waktu, karena keduanya bisa mencapai *objective score* $27,000 dalam waktu di bawah 0.03 s. SA dan GA bisa mencapai hasil yang sama, namun tidak konsisten dan waktunya lebih lama, jadi kelebihannya baru terasa ketika ruang pencarian lebih besar. *Steepest Ascent* cukup digunakan sebagai *baseline*. *Sideways Move Hill-Climbing* dapat mengatasi masalah *plateau* tapi dengan waktu paling lama.

2.6.8 Analisis Perbandingan Durasi Algoritma

Tabel 34. Durasi Setiap Algoritma

Algoritma	Iterasi	Durasi
Steepest Ascent Hill-Climbing	1	0.0032s
	2	0.0056s
	3	0.0075s
Stochastic Hill-Climbing	1	0.0045s
	2	0.0046s
	3	0.0029s
Random-Restart Hill-Climbing	1	0,0117s
	2	0,0204s
	3	0,0212s
Hill-Climbing w/ Sideways Move	1	0.2716s
	2	0.2993s
	3	0.4066s
Simulated Annealing	1	0.0225s
	2	0.0180s
	3	0.0149s
Genetic Algorithm	1	0.0168s, 0.0150s, 0.0159s

		2	0.0560s, 0.0617s, 0.0415s
		3	0.1207s, 0.0801s, 0.0812s
	2	1	0.0403s, 0.0398s, 0.0420s
		2	0.1229s, 0.1187s, 0.1214s
		3	0.2670s, 0.2655s, 0.2689s

Dari tabel di atas, terlihat bahwa Stochastic Hill-Climbing dan Steepest Ascent Hill-Climbing memiliki durasi pencarian paling cepat, yaitu sekitar 0.0029 - 0.0046 detik dan 0.0032 - 0.0075 detik. Stochastic Hill-Climbing bekerja dengan memilih tetangga secara acak, sedangkan Steepest Ascent Hill-Climbing memeriksa seluruh tetangga untuk menentukan langkah terbaik. Durasi keduanya relatif singkat, tetapi masih berisiko terjebak pada local optima.

Selanjutnya, Random-Restart Hill-Climbing dan Simulated Annealing memiliki durasi yang sedikit lebih lama, masing-masing sekitar 0.0117 - 0.0212 detik dan 0.0149 - 0.0225 detik. Durasi Random-Restart dipengaruhi oleh jumlah pengulangan yang dilakukan, sedangkan Simulated Annealing menggunakan temperatur yang menurun secara bertahap untuk membantu pencarian keluar dari local optima.

Genetic Algorithm menunjukkan durasi yang lebih bervariasi, dari 0.0150 detik hingga 0.2670 detik. Hal ini disebabkan oleh proses seleksi, crossover, dan mutasi yang dilakukan pada setiap generasi. Sementara itu, Hill-Climbing with Sideways Move menjadi algoritma dengan durasi paling lama, yaitu 0.2716 - 0.4066 detik, karena algoritma ini dapat melakukan perpindahan berulang kali pada area plateau sebelum menemukan solusi yang lebih baik.

Berdasarkan hasil tersebut, Stochastic Hill-Climbing dan Steepest Ascent Hill-Climbing unggul dari segi kecepatan. Namun, Simulated Annealing dapat menjadi pilihan yang lebih seimbang karena waktu eksekusinya tetap singkat dan memiliki mekanisme untuk mengatasi local optima. Meski demikian, pemilihan algoritma terbaik juga perlu mempertimbangkan kualitas solusi yang dihasilkan, bukan hanya durasi pencariannya.

2.6.9 Analisis Konsistensi Hasil Algoritma

Analisis konsistensi menunjukkan bahwa setiap algoritma memiliki tingkat stabilitas hasil yang berbeda. Steepest Ascent Hill-Climbing (SAHC) dan Hill-Climbing with Sideways Move (HCSM) menunjukkan pengulangan skor pada beberapa percobaan. Namun, hasil keduanya tetap dapat berbeda antar percobaan, sehingga konsistensinya tidak dapat disimpulkan hanya berdasarkan sifat algoritma.

Sementara itu, Stochastic Hill-Climbing (SHC) menunjukkan variasi hasil yang dipengaruhi oleh pemilihan neighbor secara acak. Random-Restart Hill-Climbing (RRHC) juga menghasilkan skor yang bervariasi, tetapi mampu mencapai hasil terbaik secara berulang melalui pencarian dari beberapa initial state. Adapun Simulated Annealing (SA) memiliki variasi hasil karena mekanisme probabilitasnya memungkinkan algoritma mengeksplorasi solusi yang berbeda selama proses pencarian.

Genetic Algorithm (GA) memiliki hasil yang bervariasi akibat proses selection, crossover, dan mutation yang melibatkan unsur acak. Hasilnya juga dipengaruhi oleh konfigurasi parameter, seperti ukuran populasi, jumlah iterasi, dan mutation rate.

Secara keseluruhan, konsistensi hasil setiap algoritma dipengaruhi oleh mekanisme pencarian dan unsur acak yang terlibat. Oleh karena itu, pengujian berulang diperlukan untuk menilai kestabilan hasil, sedangkan pemilihan algoritma sebaiknya mempertimbangkan konsistensi dan kualitas solusi yang dihasilkan.

2.6.10 Analisis Variasi Parameter *Genetic Algorithm*

Pengujian variasi parameter *Genetic Algorithm* dilakukan untuk melihat pengaruh jumlah generasi dan ukuran populasi terhadap kualitas solusi serta waktu eksekusi. Seluruh pengujian menggunakan *mutation rate* tetap sebesar 0,15.

- Pengaruh jumlah generasi (*Population Size* = 20)

  Pengujian dilakukan dengan 20, 50, dan 100 generasi. Pada 20 generasi, waktu eksekusi rata-rata hanya 0,016 detik, tetapi solusi terbaik masih berada pada kisaran $25.000 - $26.000. Ketika jumlah generasi ditingkatkan menjadi 50 dan 100, algoritma berhasil mencapai nilai objektif tertinggi sebesar $27.000. Namun, waktu eksekusi meningkat hingga 0,120 detik pada 100 generasi. Hasil ini menunjukkan bahwa jumlah generasi yang lebih besar dapat membantu

algoritma menemukan solusi yang lebih baik, meskipun membutuhkan waktu lebih lama.

- Pengaruh ukuran populasi (*Max Iterations* = 100)

  Pengujian dilakukan menggunakan populasi berukuran 10, 30, dan 60 individu. Populasi 10 individu memiliki waktu eksekusi sekitar 0,040 detik, tetapi skor yang diperoleh cenderung berada pada kisaran $25.000 - $26.000. Populasi yang lebih besar memberikan lebih banyak variasi solusi untuk dieksplorasi, tetapi juga meningkatkan waktu komputasi hingga 0,267 detik pada populasi 60 individu.

Berdasarkan hasil pengujian, konfigurasi *Population Size* sebesar 20 dan *Max Iterations* sebanyak 50 menjadi pilihan yang paling efisien karena berhasil mencapai nilai objektif $27.000 dengan waktu eksekusi rata-rata 0,061 detik dan rata-rata *fitness* populasi sebesar $25.000. Penambahan generasi atau populasi tidak meningkatkan nilai objektif tertinggi tersebut, tetapi membutuhkan waktu eksekusi lebih lama. Dengan demikian, konfigurasi 20 individu dan 50 generasi memberikan keseimbangan yang baik antara kualitas solusi dan waktu komputasi untuk pengujian ini.

2.6.11 Analisis Perbandingan Penggunaan Fungsi Objektif Alternatif

Pengujian fungsi objektif alternatif dilakukan menggunakan *Simulated Annealing* dan *Genetic Algorithm* untuk melihat pengaruh perubahan tujuan optimasi terhadap solusi yang dihasilkan. *Simulated Annealing* mencari solusi melalui satu jalur pencarian dan terkadang menerima solusi yang lebih buruk, sedangkan *Genetic Algorithm* mengembangkan beberapa solusi sekaligus melalui proses seleksi dan persilangan. Kedua algoritma dipilih untuk melihat perbedaan hasil pencarian tanpa perlu menguji ulang seluruh algoritma *Hill-Climbing*.

- Fungsi f_2 (ETA *Priority*) memberikan skor lebih besar kepada kendaraan dengan *shipping Fee* tinggi dan ETA kecil. Hasil pengujian menunjukkan bahwa *Best State* cenderung memilih kendaraan V1 dan V2 yang memiliki ETA = 1 serta V3 dan V5 yang memiliki ETA = 2. Sebaliknya, kendaraan seperti V7 dengan ETA = 5 kurang diprioritaskan. Hasil ini menunjukkan bahwa ETA memengaruhi

pemilihan kendaraan, meskipun total *shipping fee* yang dihasilkan belum tentu paling besar.

- Penambahan bonus berdasarkan luas geladak dan kapasitas berat menghasilkan skor objektif sekitar $30,000 hingga $34.639. Fungsi ini mendorong algoritma untuk mempertimbangkan penggunaan ruang dan kapasitas kapal, termasuk dengan memilih kendaraan berukuran kecil hingga sedang. Namun, total luas kendaraan yang dimuat belum tentu menunjukkan penggunaan ruang yang optimal karena masih ada kemungkinan terdapat celah yang sulit diisi.
- Pada *Simulated Annealing*, penggunaan f_3 (*Capacity Efficiency*) menghasilkan persentase kondisi *stuck* yang lebih rendah dibandingkan fungsi objektif utama. Hal ini menunjukkan bahwa penambahan bonus utilisasi dapat memengaruhi proses pencarian solusi. Sementara itu, pada *Genetic Algorithm*, nilai *fitness* populasi cenderung stabil. Hasil ini mengindikasikan bahwa penambahan komponen bonus membantu menjaga keanekaragaman dan kualitas populasi secara konsisten di setiap run independen.

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian algoritma pencarian solusi pada permasalahan penataan kendaraan di geladak kapal (*Cargo Carrier Deck Optimization*), diperoleh beberapa kesimpulan sebagai berikut.

- **Kinerja Algoritma**

  Stochastic Hill-Climbing dan *Steepest Ascent Hill-Climbing* memiliki waktu eksekusi yang cepat, tetapi lebih mudah terjebak pada solusi lokal. *Hill-Climbing with Sideways Move* dapat membantu pencarian melewati kondisi ketika nilai objektif tidak berubah, meskipun membutuhkan waktu komputasi lebih lama. Sementara itu, *Random Restart Hill-Climbing* dan *Simulated Annealing* memberikan kesempatan lebih besar untuk menemukan solusi yang lebih baik. *Genetic Algorithm* juga mampu mengeksplorasi berbagai kombinasi solusi dan menghasilkan nilai objektif tertinggi berdasarkan hasil pengujian.

- **Pengaruh Parameter *Genetic Algorithm***

  Population size dan *max iterations* memengaruhi kualitas solusi dan waktu eksekusi. Berdasarkan pengujian, konfigurasi *Population Size* = 20 dan *Max Iterations* = 50 menjadi pilihan yang efisien karena mampu mencapai nilai objektif sebesar $27.000 dengan waktu eksekusi rata-rata 0,061 detik. Penambahan ukuran populasi atau jumlah generasi belum memberikan peningkatan nilai objektif tertinggi pada pengujian tersebut.

- **Pengaruh Fungsi Objektif Alternatif**

  Fungsi f_2 (ETA *Priority*) mengarahkan pemilihan kendaraan berdasarkan tarif pengiriman dan waktu kedatangan, sehingga kendaraan dengan ETA lebih kecil cenderung memperoleh prioritas. Sementara itu, fungsi f_3 (*Capacity Efficiency*) menambahkan bonus berdasarkan penggunaan luas geladak dan kapasitas berat kapal. Fungsi ini mendorong pemuatan kendaraan kecil hingga sedang sebagai *filler* ruang kosong, membantu dalam menstabilkan *fitness* populasi pada *Genetic Algorithm*, serta mengurangi frekuensi *stuck* pada *Simulated Annealing*.

3.2 Saran

Pengembangan lebih lanjut dapat dilakukan dengan menguji beberapa nilai konstanta bonus pada fungsi objektif f_3 (*Capacity Efficiency*) untuk mendapatkan keseimbangan antara total *shipping fee*, penggunaan luas geladak, dan kapasitas berat kapal. Selain itu, parameter algoritma juga dapat disesuaikan selama proses pencarian, seperti penggunaan *cooling schedule* adaptif pada *Simulated Annealing* serta perubahan mutation rate dan crossover rate pada *Genetic Algorithm.* Penggabungan beberapa algoritma juga dapat dicoba, misalnya menggunakan *Hill-Climbing* untuk memperbaiki solusi awal sebelum diproses oleh *Genetic Algorithm.* Pendekatan tersebut dapat diuji lebih lanjut untuk mengetahui pengaruhnya terhadap kualitas solusi dan waktu eksekusi.

BAB IV
PEMBAGIAN TUGAS ANGGOTA KELOMPOK

Berikut adalah pembagian kerja dalam pengerjaan Tugas Besar IF3070 Dasar Intelegensi Artifisial 1, yaitu sebagai berikut:

Tabel 35. Pembagian Tugas

NIM	Nama	Bagian Pengerjaan
18224015	Salma Az Zahra	1. Pembuatan *Source Code Random Restart Hill-Climbing* 2. Pembuatan *Source Code Stochastic Hill-Climbing* 3. Pengerjaan Laporan Bagian Pemilihan *Objective Function* 4. Pengerjaan Laporan Bagian Penjelasan Pembangkitan *State* Awal (*Initial State*) 5. Pengerjaan Laporan Bagian Penjelasan Algoritma *Steepest Ascent Hill-Climbing, Stochastic Hill-Climbing* dan *Random Restart Hill-Climbing* 6. Pengerjaan Laporan Bagian Hasil Eksperimen 7. Pengerjaan Laporan Bagian Analisis 8. Finalisasi Laporan Akhir
18224051	Belva Charissa Putri Have	1. Pembuatan *Source Code Hill-Climbing with Sideways Move* 2. Pembuatan *Source Code Simulated Annealing* 3. Pengerjaan Laporan Bagian 2D Vehicle Problem 4. Pengerjaan Laporan Bagian Permasalahan 5. Pengerjaan Laporan Bagian Algoritma *Hill-Climbing with Sideways Move, Simulated Annealing* dan *Genetic Algorithm* 6. Pengerjaan Laporan Bagian Dataset 7. Pengerjaan Laporan Bagian Hasil Eksperimen 8. Pengerjaan Laporan Bagian Analisis 9. Finalisasi Laporan Akhir
18224075	Sherry Eunike	1. Pembuatan *Source Code Steepest Ascent Hill-Climbing* 2. Pembuatan *Source Code Genetic Algorithm* 3. Melakukan *Testing* 4. Mengintegrasikan Seluruh Algoritma 5. Pengembangan Fungsi Objektif Tambahan 6. Pengerjaan README 7. Melakukan *Testing* dan *Debugging* 8. Pengerjaan Laporan Bagian Hasil Eksperimen 9. Pengerjaan Laporan Bagian Analisis

		10. Pengerjaan Laporan Bab 3 11. Finalisasi Laporan Akhir

REFERENSI

Google. (n.d.). Packing. Google for Developers. https://developers.google.com/optimization/pack

Katoch, S., Chauhan, S. S., & Kumar, V. (2021). A review on genetic algorithm: Past, present, and future. Multimedia Tools and Applications, 80, 8091–8126. https://doi.org/10.1007/s11042-020-10139-6

Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

University of California, Berkeley, Department of Electrical Engineering and Computer Sciences. (n.d.). 1.5 Local search. Introduction to Artificial Intelligence. https://inst.eecs.berkeley.edu/~cs188/textbook/search/local.html

Diyar Waysi, et al. "Optimization by Nature: A Review of Genetic Algorithm Techniques." *Indonesian Journal of Computer Science*, vol. 14, no. 1, 7 Feb. 2025, https://doi.org/10.33022/ijcs.v14i1.4596.

Eglese, R.W. "Simulated Annealing: A Tool for Operational Research." *European Journal of Operational Research*, vol. 46, no. 3, June 1990, pp. 271–281, https://doi.org/10.1016/0377-2217(90)90001-r.

GeeksforGeeks. "Difference between Hill-Climbing and Simulated Annealing Algorithm." www.geeksforgeeks.org/machine-learning/difference-between-hill-climbing-and-simulated-annealing-algorithm/.

"Genetic Algorithms." *GeeksforGeeks*, 29 June 2017, www.geeksforgeeks.org/dsa/genetic-algorithms/.

"Hill-Climbing in Artificial Intelligence." GeeksforGeeks, 12 Dec. 2017, www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/.

McKee, Amberle. "Genetic Algorithm: Complete Guide with Python Implementation." *Datacamp.com*, DataCamp, www.datacamp.com/tutorial/genetic-algorithm-python.

Tahsin Soyak. "Hill Climbing Algorithm: A Comprehensive Guide - Tahsin Soyak - Medium." medium.com/@tahsinsoyakk/hill-climbing-algorithm-a-comprehensive-guide-46e33f1ecc02.

LAMPIRAN

Gambar 2. Kelompok GARUDA

Gambar 3. Perjuangan Mengerjakan Tubes DAI

Repository Github: https://github.com/gasukabawang/IF3070_G16_GARUDA

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Belva Charissa Putri Have / 18224051
Salma Az Zahra / 18224015
Sherry Eunike / 18224075

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Mata Kuliah : IF3070

Nama Mata Kuliah : Dasar Intelegensi Artificial

Judul Tugas/Proposal : Optimasi Penataan Kendaraan Geladak Kapal Kargo PT Intim Line Menggunakan Algoritma Local Search

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, QuillBot, GrammarlyGO, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Rendah
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Bantuan Pembuatan Kode: menggunakan tools seperti ChatGPT, GitHub Copilot, Google Gemini, atau sejenisnya untuk membantu penulisan, perbaikan, dan penjelasan kode program.	Ya (file app.py dan index.html)	Sedang

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026

Belva Charissa Putri Have
18224051